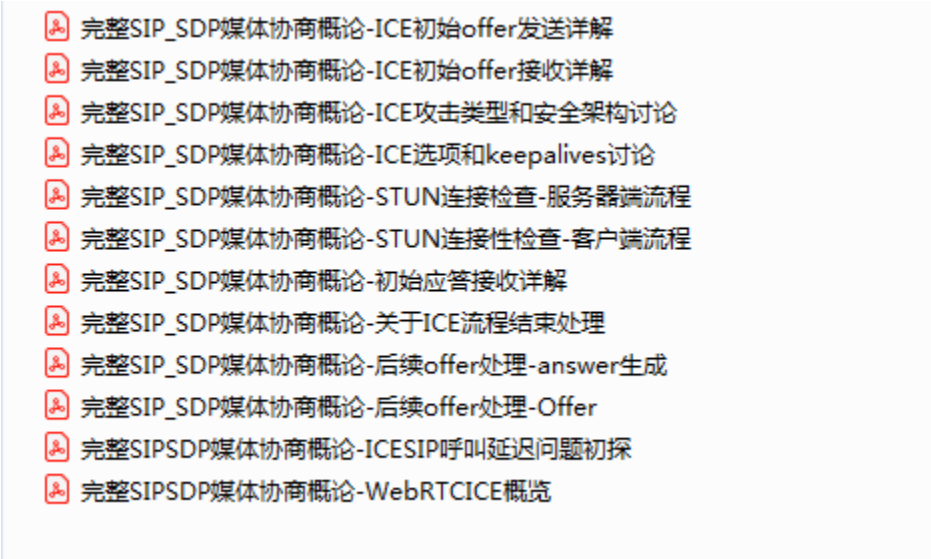
ICE/RFC5245中文详解

Interactive Connectivity Establishment (ICE):

A Protocol for Network Address Translator (NAT) Traversal for Offer/Answer Protocols

根据RFC5245 和RFC8445 翻译结合具体应用完成的完整中文详解（大概150 页），RFC5245是WebRTC技术的核心协议，RTC/WebRTC开发人员必须了解的协议。



关注微信公众号：asterisk-cn， 获得更多有价值的通信/SIP协议和技术分享。
访问 www.asterisk.org.cn 获得更多关于 Asterisk/FreePBX/SIP/完整开源通信平台技术文档
中国领先的 IP 通信解决方案提供商–深圳鼎信通达股份有限公司：www.dinstar.cn

完整SIP/SDP媒体协商概论-WebRTC/ICE概览

原创 james.zhu Asterisk开源派

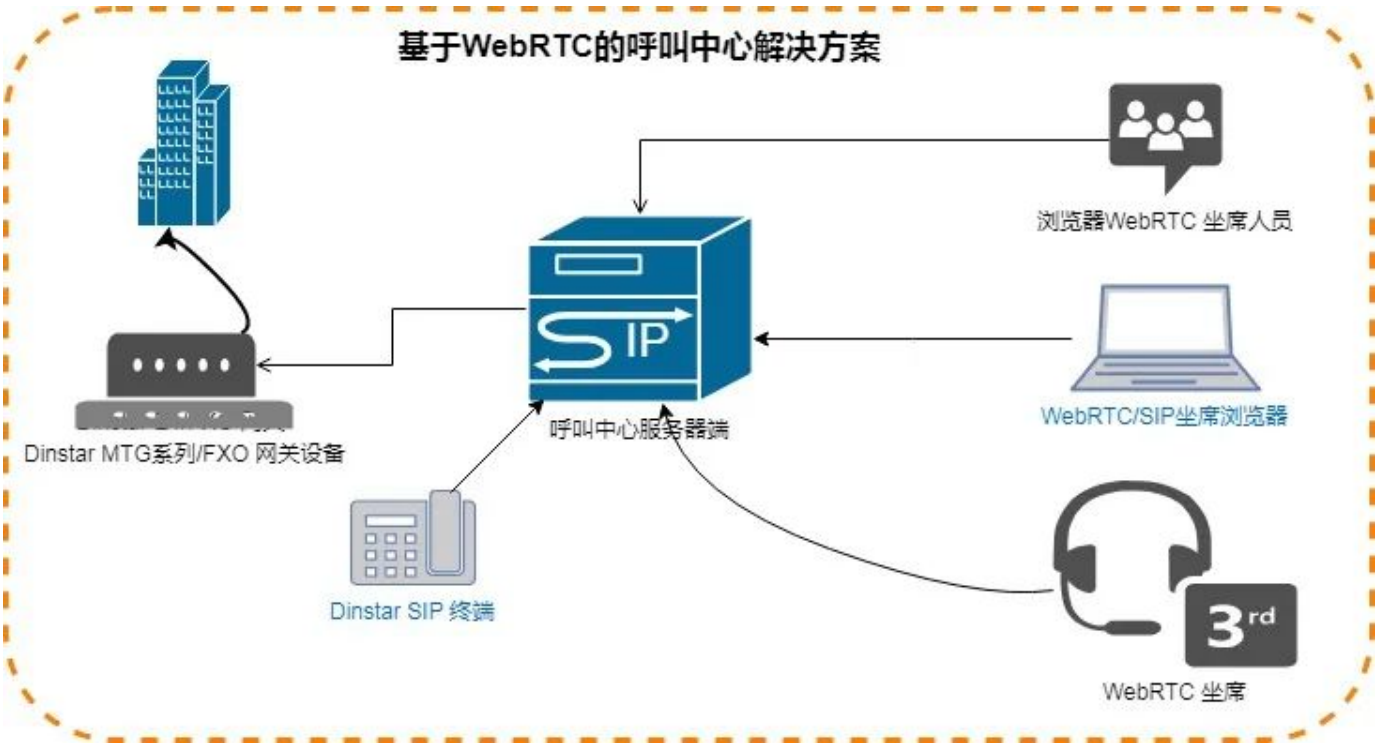
在前面的章节中，我们完整介绍了SIP中SDP的offer/answer交互流程。接下来，我们重点介绍关于SDP在WebRTC中的交互方式以及使用ICE来支持NAT处理的内容。一些读者可能注意到了，WebRTC技术虽然具有非常大的市场前景，但是因为本身和浏览器等其他工具的兼容性问题，发展的速度仍然没有想象的那么快，一些应用场景也不是太完善。很多针对ICE的规范也快速进行更迭。最近的一次更迭就是针对ICE的RFC5245规范已废除，使用了RFC8445。因为，RFC8445是2018年发布的最新的规范，因此，可能一些厂家的产品还没有完全实现对此规范版本的支持。所以，我们的讨论尽量按照RFC5245的规范来进行，也同时兼顾RFC8445的框架。笔者提前说明，WebRTC中关于ICE的规范流程非常庞杂，笔者需要分成多个章节来解释。所以，在WebRTC/ICE章节讨论中我们需要花费一定的时间，如果读者对ICE没有兴趣的话或者暂时没有使用WebRTC技术的话，可以跳过关于WebRTC/ICE的介绍直接阅读SIP/SDP的部分内容，或者，如果读者仅想了解WebRTC技术和基本的工作原理的话，可以阅读：

完整WebRTC技术及应用概要

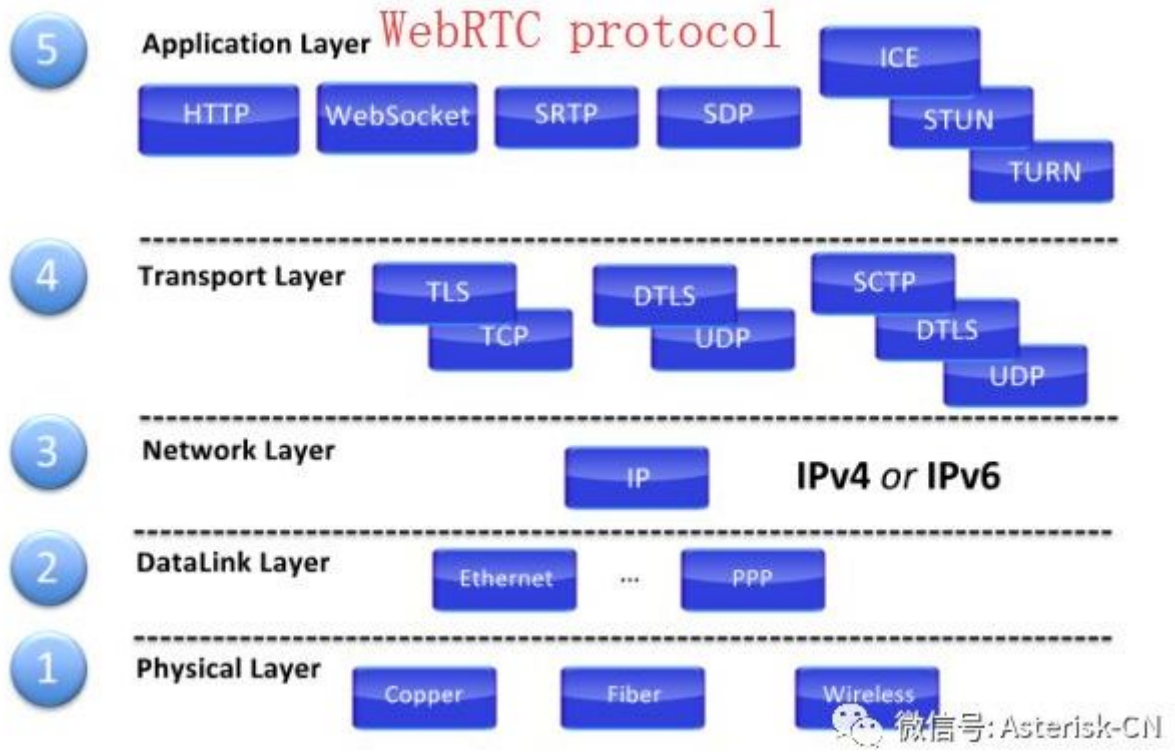
在本章节和后续章节的讨论中，笔者将根据RFC5245/RFC8445的规范架构，结合笔者的这篇历史文章来讨论WebRTC/ICE的详细技术内容。希望通过笔者完整的讨论，读者可以非常清楚了解WebRTC的流程，特别是关于ICE的处理。笔者在文章中使用的一些本规范的专有名词（例如，check 或者check list，事实上，在本规范中它具有特定的含义），可能没有直接以中文的名称来介绍，这样做的目的是为了保证不会产生歧义，所以笔者尽量使用规范中的专有名词来解释。所以，请读者在阅读时一定要注意。

说明：1）一些专有名词以前的章节中已经发布，这里不再介绍。2）笔者本人水平有限，文章中所使用的专有名词中文命名或解释可能和其他网上的的有所不同，建议读者参考RFC原文理解或者发邮件给规范起草人获得支持。

广告



笔者会在接下来的内容中重点讨论关于ICE的背景介绍，ICE概览，ICE使用/ICE候选地址采集和交互，ICE候选对象流程处理，执行连接性检查，完成ICE创建，轻量级ICE使用介绍等话题。



1 背景介绍

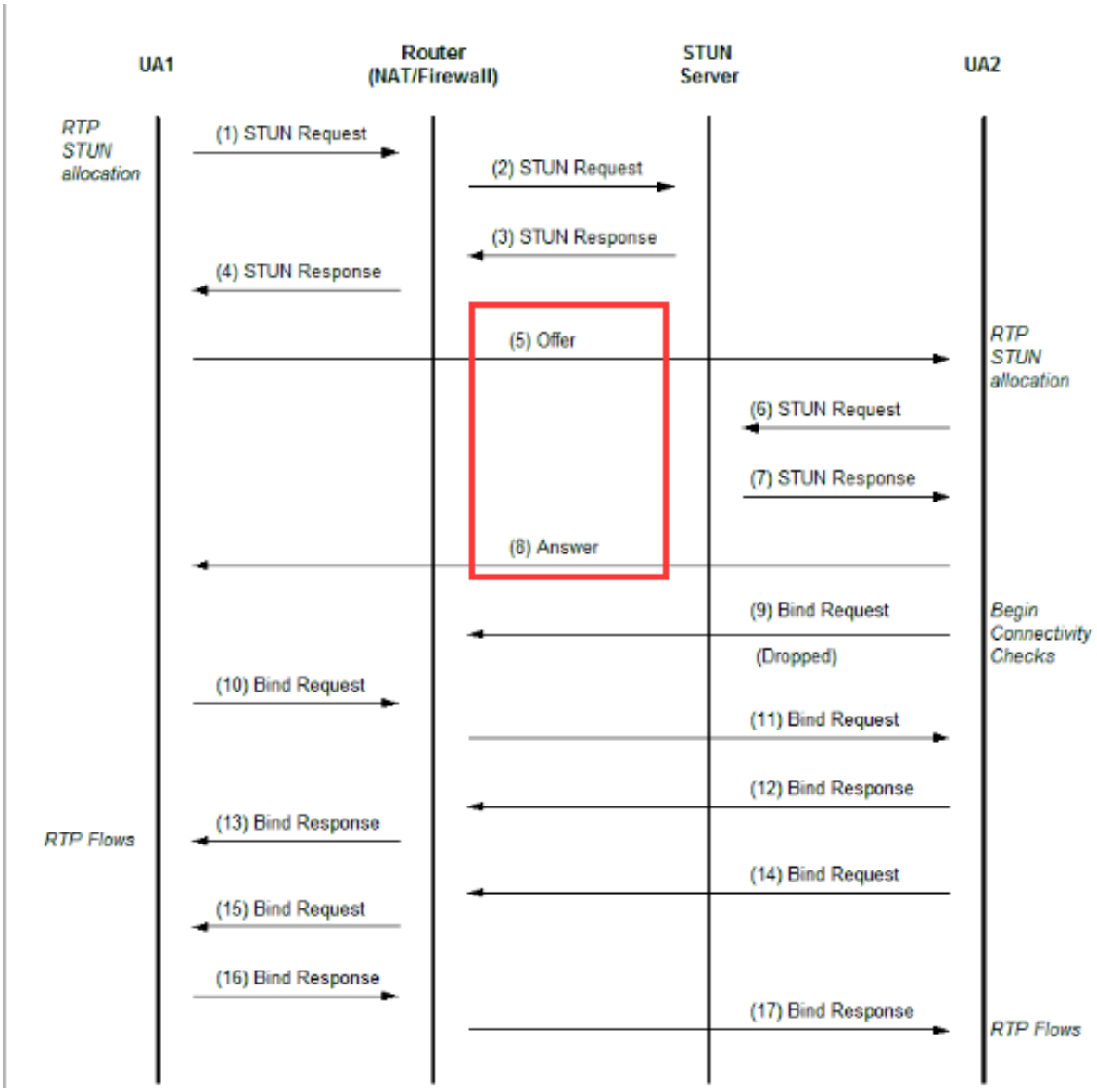
如果读者看过前面的SDP全解的读者可能已经了解，SIP使用了offer/answer结合模式，通过SDP消息来实现媒体传输，其最终目的是实现媒体流之间的创建和完整传输。ICE英文全名是Interactive Connectivity Establishment。RFC5245(更新的RFC6336)对ICE做了规定。笔者推荐的一般简单的定义是：**ICE=STUN+TURN+协商机制+协商路径。**

但是，因为网络越来越复杂，终端的环境也发生了根本变化，因此NAT问题也越来越多。RFC3235规范针对NAT（地址转换）发布了一个指导。很多相关的协议希望通过媒体流之间的点对点传输解决媒体本身的问题（例如，低时延，降低丢包，降低部署成本），但是在涉及到NAT环境时，通常会遇到一些更加复杂的问题，导致实际部署的难度大大增加。为了解决NAT的问题，很多针对性的技术规范增加了对NAT环境的支持，常见的规范包括：

- Application Layer Gateways (ALGs）
- Middlebox Control Protocol (RFC 3303)
- STUN(RFC 3489)
- Realm Specific IP(RFCs 3102和3103)
- SDP拓展支持(RFC 4566和RFC 3605）

随着技术本身的不断发展，这些针对NAT支持的规范也带来了很多突出的问题，它们都存在各自的优缺点。这样的话，网络管理就会增加很多的不确定性，给系统管理带来很多问题。为了解决这些问题，一些相关规范组织希望使用一种统一的解决方式或规范，并且这种协议可以提供灵活性来满足目前网络环境对NAT的支持。目前，大家一致的**共识**就是使用ICE技术（Interactive Connectivity Establishment）。此规范通过offer/answer模式定义ICE来支持基于UDP的媒体流NAT问题（当然，ICE也可以通过拓展支持ICE-TCP传输协议）。

ICE是offer/answer模式的一种拓展形式，通过在SDP的offer/answer消息中包含多个地址和端口，使用这些地址端口和其交互消息来检测点对点的连接性。在SDP中的地址和端口，以及其连接性检测是通过STUN来完成。



图片来自于互联网资源

注意，关于STUN的最新规范已经再次更新（更新时间为2020年），如果读者有兴趣做进一步研究，可参考RFC5389和RFC8489。除了STUN以外，ICE也使用了STUN的另外一个拓展协议-TURN（RFC5766）实现穿越转发。另外，因为ICE对每个媒体流进行了多地址和端口交互，它也允许地址选择支持多宿主和双栈主机（IPv4/IPv6），因此也不再支持RFC4091和RFC4092。

2 ICE概览

在一个比较典型的ICE部署环境中，至少需要两个终端需要互相进行通信。终端之间可以通过一些信令结合SDP的offer/answer模式来实现，例如，我们前面讲的SIP协议。读者需要注意，ICE的目的不是为了某些协议（例如SIP）的NAT穿越，关于SIP相关的NAT穿越是通过RFC5626规定的，如果读者有兴趣的话，可以查阅此规范。这里，ICE假设终端之间可以创建

协议连接。具体来说，在ICE流程处理开始阶段，agents就忽略了它们本身的技术属性。终端也可能在或不在NAT后（例如后面将的轻量级部署终端），ICE允许终端获取到技术属性的足够信息，然后找到一个或多个潜在的路径，创建数据会话实现相互连接。

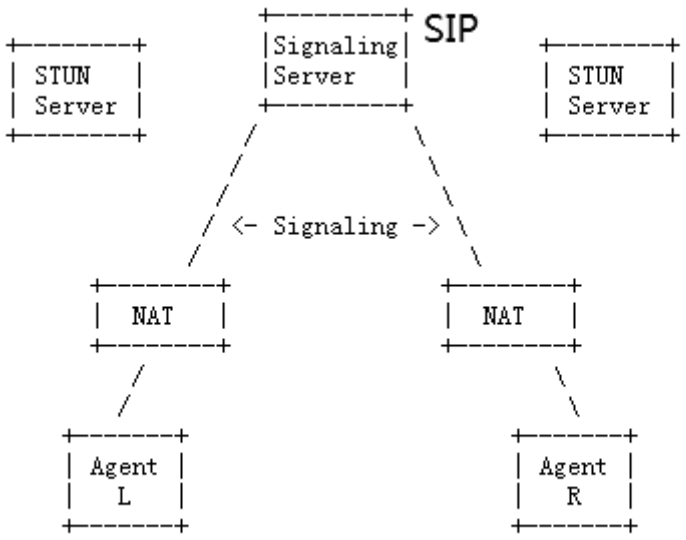
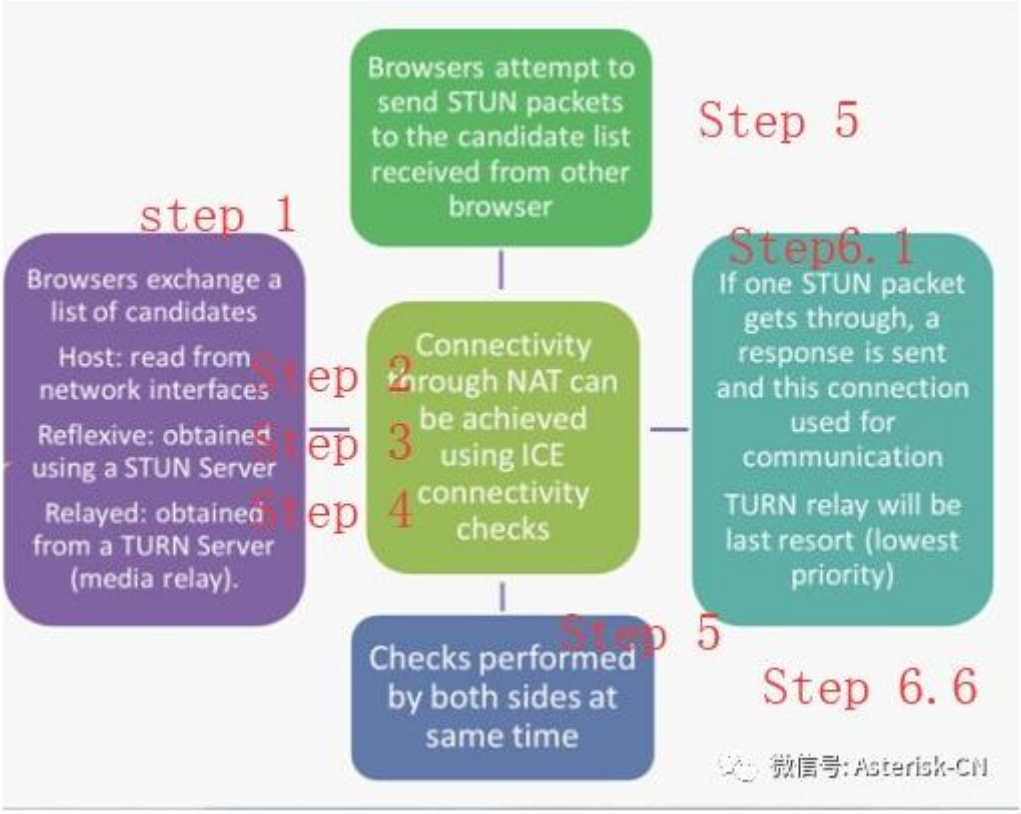


Figure 1: ICE Deployment Scenario

图片均来自于互联网资源

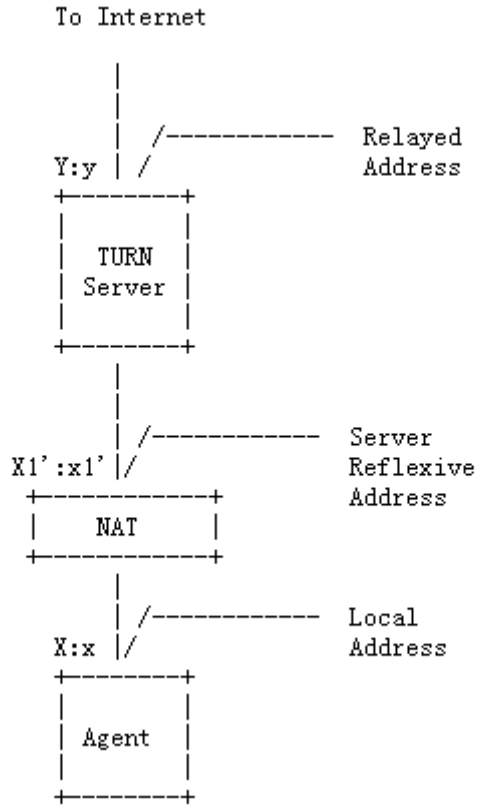
在以上示例是一个非常典型的ICE部署场景示例。两个终端分别标识为L（左边）和R（右边）。两个终端都有各自的NAT环境，双方也不清楚对端NAT的状态属性，左右双方终端都有意愿通过ICE候选对象交互实现通信。很多时候，双方的交互可能都使用SIP协议。对于双方终端，SIP服务器和NAT来说，在网络中，ICE经常使用STUN和TURN来保证所有对象的协同。每个终端agents可以支持自己独立的STUN和TRUN服务器，也可以同一STUN和TURN同一服务器。基本的ICE工作理念是: 针对传输协议（这里重点讨论UDP）每个agent都有各种候选地址（绑定了IP地址和端口），通过这种候选地址和对端agent实现通信。候选地址可能包括：附加到网络接口的传输地址（server reflexive 地址，其地址是STUN发现的地址，在NAT外），NAT中公网侧的转换后的传输地址，从TURN分配到的传输地址（转发地址）。客观上存在这种可能，任何L侧的候选传输地址都用来和任何R侧的候选传输地址进行通信。但是，在实际场景中，过多的候选传输地址组合可能不能工作。例如，如果L侧和R侧双方都在NAT后的话，直接附加的网络接口地址可能不会直接通信，因此这里需要ICE介入。这里，ICE的目的是发现何种候选地址配对可以工作。ICE的工作方式是通过系统地尝试所有可能的候选配对（排序处理后），直到ICE找到一组或者多组可以工作的候选配对。ICE对Peers的检测配对需要经过几个核心步骤:



3 采集候选地址

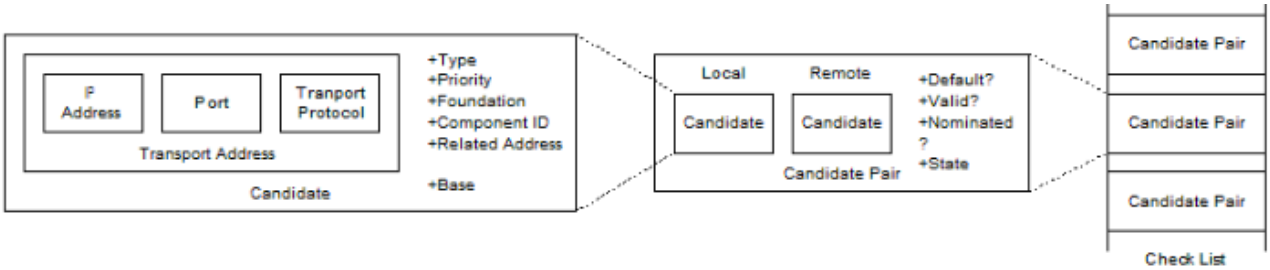
如果需要执行ICE的话，agent首先需要确认候选地址。候选地址是一种特别的地址，它是由IP地址和端口的组合构成，其目的是支持传输协议（我们这里仅讨论UDP）。RFC8445定义了三种候选地址，一种是来自于**物理网络接口**，另外一种来自于**网络的逻辑接口**，还有一种是通过**STURN或者TURN发现的候选地址**。第一种类别的候选地址支持的传输地址直接从本地网络接口获得。这样的候选地址我们称之为“host candidate”，这样类型的本地地址可以从本地网络，WIFI，远端网络或者VPN的方式获得。对agent来说，这样的地址都可以通过分配获得，并且作为本地接口来使用。如果agent是一个多宿主主机的话，它可以从不同的IP地址获得候选地址。具体的候选地址如何获得取决于网络中peer（会话中另外一个agent）的位置。例如，如果agent支持了两个不同的IP地址的话（一个是内网地址，一个是外网地址），不同的peer就可以通过不同的地址和agent通信。

另外一种候选地址是agent使用STUN或TURN获得的额外的候选地址，这些候选地址主要表现为两种地址形式：NAT公网侧的已转换地址（server-reflexive 候选地址），TURN服务器分配的转发地址（relayed 候选地址）。这里读者一定要注意**两种地址的获取方式**。当使用TURN服务器时，以上两种候选地址都来自于TURN服务器分配地址；当仅使用了STURN服务器的话，agent只能获取到server reflexive地址。以下是候选地址的关系示例图：



候选地址关系

在以上关系图中，两种类型的候选地址都是通过TURN服务器发现获得的。这里的地址端口配对中，大写X表示IP地址，小写x表示UDP端口。当agent对IP地址和端口发送一个（X:x）TURN地址分配请求时，NAT(假设这里有NAT)就会创建一个绑定关系X1':x1'，映射server-reflexive地址到主机候选地址。从本地主机候选地址发送出去的数据包将会通过NAT地址转换变成一个server-reflexive候选地址。同样的道理，从server-reflexive候选地址进入到主机候选地址的数据也是通过NAT地址转换完成。这里，**base**是一个比较重要的概念，需要读者明确，“base"是一个本地主机候选地址，它关联一个给定的server-reflexive候选地址。base指的是一个agent为指定的候选地址发出数据的地址。因此，有时存在一个比较极端的场景，主机候选地址也可以有自己的base地址，这个base 地址和主机候选地址相同。



来自于互联网资源

当agent和TURN服务器存在多个NAT穿越时，TRUN请求会为每个NAT创建一个绑定，但是agent仅发现最外部的server-reflexive候选地址（距离TRUN服务器最近的候选地址）。如果agent不在NAT后的话，base 候选地址和server-reflexive候选地址相同，server-reflexive候选地址就会被移除。

Agent发出的分配请求到达TURN服务器端后，TURN服务器将会从它的本地IP地址Y中分配一个端口y，并且生成一个分配响应，分配响应通知一个agent的转发候选地址。TRUN服务器也会通知一个agent的server-reflexive候选地址X1':x1'，通知的方式是把分配请求中的源传输地址拷贝到分配响应中来实现。TRUN服务器的工作角色类似于一个在L侧和R侧之间的数据转发。如果R侧想对L侧发送数据的话，R侧需要发送数据到Y:y 地址，然后TRUN服务器端前转到X1':x1'候选地址，然后经过NAT后映射到L侧agent。

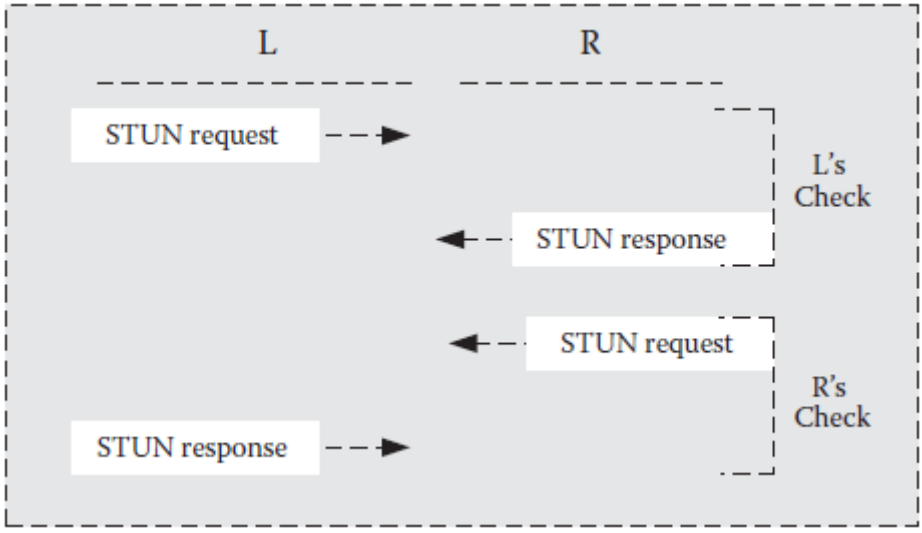
当仅使用了STUN服务器时，agent发送一个STUN绑定请求给它的STUN服务器，STUN将会通知一个agent的server-reflexive候选地址X1':x1'，其通知的方式是绑定请求中的源传输地址拷贝到绑定响应中。

4 连接性检查

一旦L侧agent采集到它所有的候选地址后，它会把这些地址重新按照从最高到最低的排序，然后通过信令通道发送给R侧agent。这些候选地址通过SDP的offer消息的属性参数发送到R侧agent（开始使用offer/answer交互模式）。当R侧agent收到offer消息后，R侧的agent也会执行一个同样的流程来采集候选地址，然后通过响应消息返回自己的候选地址。双方采集发送流程完成以后，每个agent都有自己的完整的候选地址和对待peer的完整候选地址。通过双方候选地址的配对处理，最后产生一个候选配对。候选地址配对产生以后，agent首先需要知道哪个候选地址配对是可以工作的，每个agent会按时设定一系列的检查。每个检查是一个STUN请求响应的事务，每个终端都会执行具体的候选地址配对流程，配对流程检查通过本地候选地址对远端候选地址发送一个STUN请求。连接性检查（Connectivity Checks）的基本原理非常简单：

1. 对候选地址配对进行优先级排序
2. 按照优先级排序顺序对候选配对发送检查请求
3. 确认从其他agent收到检查状态

因此，在执行候选配对检查时需要一个四次握手的处理：



基本候选地址配对检查四次握手处理流程

这里一定要注意，发送STUN请求的目的地和接收源的IP地址和端口，这是完全相同的IP地址和端口，使用此IP地址和端口来传输媒体流（包括RTP和RTCP）。因此，agent会通过数据内容多路分解STUN/RTP/RTCP相关数据，而不使用其接收端口来分解STUN/RTP/RTCP数据。相对于使用端口的方式来说，多路分解方式会更容易处理检查状态，特别是针对RTP和RTCP的数据。因为，STUN绑定请求是用来执行连接检查的，在STUN响应中包含了agent的已转译的传输地址，这个地址是agent和对端peer之间的NAT公网侧地址。如果这个传输地址和agent已学习过的候选地址不同的话，agent就会知道这个候选地址是一个新的地址（PEER REFLEXIVE CANDIDATE），这个新地址和其他候选地址一样，也是ICE测试过的地址。

作为一种优化方式，只要R侧获得了L侧的检查消息，R侧会在同一候选配对中按时对L侧发送一个连接检查消息。这样的话，ICE就会加快发现有效候选流程处理时间，这个过程也称之为“TRIGGERED CHECK”。在双方握手完成以后，双方都知道对对端可以接收或者发送端对端消息。到此为止，连接检查结束。

5 候选地址排序

根据连接检查的介绍，读者可能已经注意到了，前面我们讨论的候选配对查询算法还有一定的问题，假设候选配对存在的话，无论查询方式是何种顺序，查询流程会一直查询直到找到这一对候选配对。显然，查询的顺序肯定影响候选配对的查询结果。因此，为了快速生成候选配对结果，候选地址需要经过排序处理，最后，排序后的候选配对结果就是一个check list。关于排序算法笔者在发送初始offer的章节进行讨论。基本上，排序算法需要遵守两个基本原则：

1. 每个agent给它的候选地址一个优先级数字标识，此优先级标识会随候选地址发送到对端peer。
2. 本地和远端优先级合并，每个agent针对候选配对来说有一个同样的排序。

其中，第二个原则非常重要。如果双方L侧agent和R侧agent的都在NAT后的话，为了确保ICE工作，必须特别注意第二个原则。我们经常可以看到，NAT是不会允许一台外部主机发送数据进入到NAT后的网络环境中，只有在NAT后的agent通过NAT发送数据给这台主机后才被允许交互，外部数据才能进入到NAT后的环境中。这里要注意，直到agent双方都已通过自己相关的NAT穿越发送check信息后，双向的ICE check才能最终成功。agent需要通过check list才能启动工作，所以，它需要周期性地发送一个STUN请求获得列表中的候选配对。这个处理过程称之为 “ORDINARY CHECKS”。

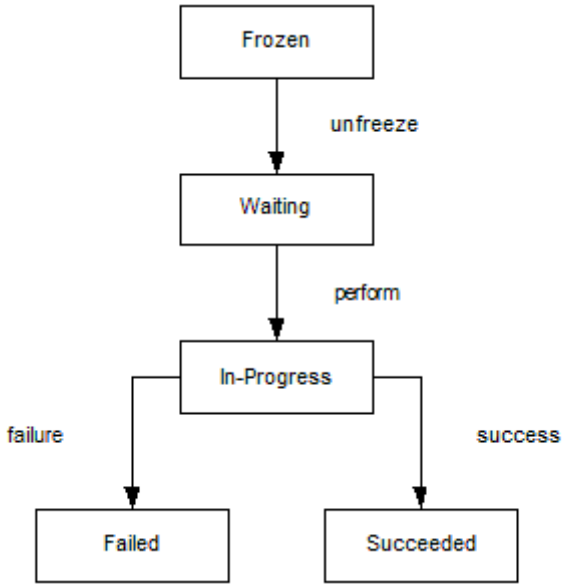
通常情况下，优先级算法的设计是为了同类候选地址直接获得同样的优先级，和一些非直接处理的方式相比，优先级算法在处理候选配对是可能更加高效。因此，通过优先级算法的处理流程可以快速高效地实现直接路由访问，路由处理路径中仅需要几个媒体转发和几个NAT转发。如果采用非直接处理的方式的话，候选配对可能需要更多媒体转发和NAT转发，经过越多的转发就会导致越多的不可控因素。所以，这样的方式不是一种好的推荐的方式。但是，agent都有自己的决定权来优化算法。

6 锁定候选配对

前面的讨论中，我们仅仅描述了一种场景，那就是agents想使用一个组件模块（COMPONENT）创建一个媒体会话。这里，COMPONENT是一个媒体流。在典型的应用环境中，agents实际需要创建一个连接来支持更多的数据流程。网络环境中的很多属性和组件具有非常大的相似度。通常情况下，如果一个组件模块可以工作的话，我们会借用同样的信息来决定最近的候选地址。ICE使用这样的处理方式来处理候选配对流程，这种算法简称为frozen candidates或者锁定候选算法。每个候选地址都和自己的一个属性相关联，我们称这个属性为“FOUNDATION”。如果它们被认为是“相似”的话，我们就会认为他们具有相同的FOUNDATION属性，这表示它们有相同的类型，并且都是从同一host candidate，同样的STUN服务器，使用同样的协议获得的类型数据。否则，这样的候选地址就是不同的。同样的道理，候选配对也有FOUNDATION属性。这个候选配对的FOUNDATION就是通过合并候选地址的FOUNDATION构成。在初始阶段，只有针对FOUNDATION是唯一状态的候选配对进行测试。其他的候选配对会被标识为“frozen” 或者锁定状态。当候选配对的连接检查是成

功状态的话，其他带相同FOUNDATION属性的候选配对被unfrozen或者解封。通过这样的方式就可以避免了重复的连接性检查，增加检查的效率。但是，事实上，这种方式也有失败的可能。frozen candidates是ICE处理流程的一部分，ICE有优先级的算法会自动使用正确的顺序确保正确的候选地址被解封。

这里笔者再花费一点时间再一点补充说明。在笔者接触到的资源中，很多关于WebRTC/ICE的技术讨论，以及相关的官方中，包括RFC5245本身也没有非常完整地解释清楚“frozen”的定义。这里，笔者试图借用一定的示例来进一步明确说明frozen candidates的具体含义。首先，我们来解释一下在我们讨论的场景中具体的frozen定义。如果我们单纯从字面意思, 笔者认为可以理解frozen为封冻或者锁定的含义。接下来读者就容易理解frozen candidates的完整含义。从前面的解释，我们可以看到frozen candidates就是一种候选地址的状态。



来自于互联网资源

每个在check list中的候选配对都有自己的状态，具体的状态处理经过以下几个步骤：

- **Frozen** – 处于锁定状态，等待检查。
- **Waiting** – 从check list中选择最高优先级的候选配对进行处理。
- **In-Progress** – 为这个配对发送check请求，事务在处理流程中。
- **Succeeded** – 配对检查成功
- **Failed** – 配对检查失败

7 安全检查

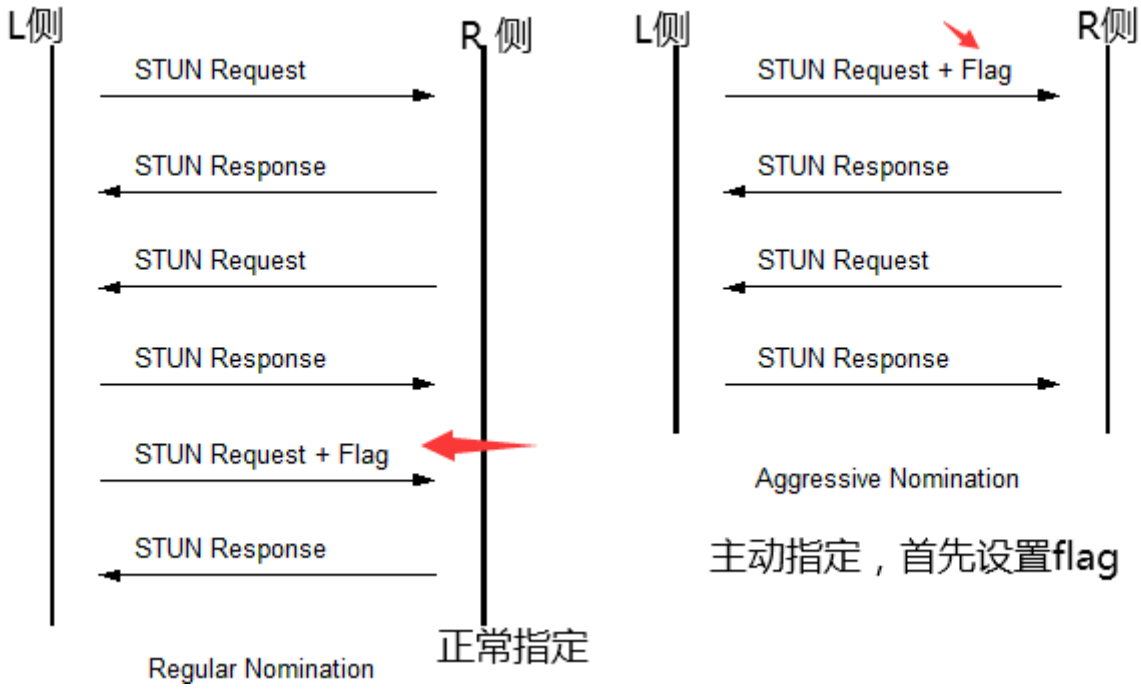
大家知道，ICE的目的是发现两个agent之间的地址，通过这个地址来发送媒体流数据。当然，ICE必须确保这个地址不能出现安全问题，也必须避免把媒体数据发送到错误的地址。每个

STUN的连接性检查都要经过消息认证码（MAC）的计算，消息认证码在信令通道中使用密钥交换的方式进行计算。因为MAC提供了消息完整性和数据原始认证，因此它可以杜绝攻击者伪造和修改连接性检查消息内容。一个比较典型的例子就是SIP的分叉呼叫处理。如果SIP呼叫用户正在使用ICE，并且此呼叫进行了分叉呼叫处理的话，ICE处理就会在每个独立的接收方之间进行。这种使用场景中，在信令通道中的密钥交换帮助每个ICE交换和其分叉的接收方进行交换处理。如果读者不了解分叉呼叫的内容，可以查阅本微信号的历史文章-关于分叉呼叫流程的处理。

8 完成ICE创建

通过前面的介绍，我们知道ICE检查是需要按照顺序处理的。ICE首先处理优先级最高的候选配对，然后再处理低优先级的候选配对。只要每个模块的媒体流检查都成功以后，ICE通过声明一个“成功”表示候选配对成功，整个ICE的创建就完成了。这种操作方式确实也是比较合理的，我们会在后续的文章中介绍具体的算法。但是，任何事情都有其两面性，通过这种按照优先级高低来执行候选配对检查的算法需要考虑一定的风险。如果最高优先级的候选配对产生了数据丢失的问题，这样就会耗费了稍长时间完成配对检查。这种情况下，ICE可能需要花费稍长时间，但是可能会产生比较好的结果。从根本上来说，RFC5245官方规定的优先级算法不一定会产生“优化”的结果。如果其算法的目的是选择低时延的媒体路径的话，使用了转发（转发可能是高延时）作为一种建议提示的话，这种转发建议实际上就没有多大关系。实际场景中可能使用往返时延（RTT）的衡量指标的方式可能比优先级的处理方式更好，在演示中场景中，低优先级的候选配可能比高优先级的候选配对取得更低的时延。

标识成功配对以后，接下来，ICE就会指定agent的功能角色。其中一个agent称之为CONTROLLING AGENT（主控agent），另外一个agnet是CONTROLLED AGENT（被控agent）。主控agent就会从**有效的**候选配对中指定一个配对来应对媒体处理。主控agent通过两种方式来实现指定候选配对：使用regular nomination或者使用aggressive nomination的方式来指定候选配对。



来自于互联网资源

使用regular nomination正常指定候选配对的话，主控方agent会一直发送请求消息，让check处理进行处理，直到找到至少一个有效的候选配对可以支持媒体。然后选择其有效的候选配对。接下来主控agent仍然会继续发送第二个STUN请求消息，但是，在第二个STUN请求中，主控agent会附加一个flag，通知对端，主控agent已经指定了一个候选配对，将要使用那个候选配对来处理媒体流。当携带flag的这个STUN事务完成以后，双方都取消了进一步的check流程。ICE将使用最终指定的那个有效配对发送媒体。ICE正在使用的这对候选配对称之为“SELECTED PAIR”，成为已选配对。所以，读者一定要明白，ICE是真正使用selected pair来进行媒体处理的，前面所有的候选配对都是为ICE最后选择配对进行准备。

上面，笔者介绍了正常指定配对的方式，这里我们再介绍一下aggressive nomination的使用方式。aggressive nomination的方式相对比较激进或主动一点。主控agent使用aggressive nomination时，主控agent会在**每个STUN请求中附加一个flag**，一旦找到第一个成功的配对的话，ICE就会结束其他的检查流程，然后使用其配对处理媒体，主控agent**不会再发送第二个STUN请求**。此selected pair具有最高的优先级。所以，我们通过两种方式的介绍，我们可以看出，**aggressive nomination检查速度快，但是缺少灵活性；regular nomination则处理速度慢，但是可能会找到低时延的媒体路径。**

一旦所有的媒体处理完成后，如果媒体中“m=”和“c=”行中的候选地址（默认候选）不能匹配selected pair（已选配对）的话，主控端会发送一个更新offer消息。如果ICE结束后，为了支持agent的媒体流，任何一方agent都可以在任何时间重新启动ICE。重新启动ICE可以通过agent发送一个更新offer的方式来处理。

9 ICE 轻量级部署讨论

为了让呼叫支持ICE，双方agent必须都使用ICE。但是，在实际环境中，某些agent本身就带了公网地址，它可以从任何通信端接收数据或者进行通信。为了让这类终端能够方便地支持ICE，ICE定义了一种特别的部署方式，称之为lite implementation，或者轻量级部署方式。当然，相对于lite implementation的支持方式，ICE支持正常的full implementation全部署方式。因为这类带国外地址的终端本身带有公网地址，所以，轻量级的部署方式会减少很多中间处理环节。首先，它不采集候选地址，它也不包含支持媒体的主机候选地址（一般的内网地址）。另外，虽然它们需要返回响应消息，但是，这类agent不会生成连接检查或运行状态机。当两个轻量级agent连接时无需发送检查请求响应消息；可是，当使用轻量级agent和全部署方式连接时，这时，全部署方式的agent会变为主控agent来控制检查流程，轻量级agent就会变为被控agent。笔者将会在后续章节或者文章中讨论这两种部署方式的具体细节。读者需要特别注意，轻量级的部署是针对全部署方式的一种补充方式，是在RFC5245中稍晚时间增加的功能支持。针对那些支持公网地址的终端来说，如果可以实现全部署方式的话，RFC5245推荐使用全部署方式。

以上都是关于ICE的热身流程，包括了基本的概念以及大概的处理流程。后续文章中，笔者将继续介绍ICE处理流程和具体细节，从真正的第一步处理流程开始-初始化offer的处理，包括初始化offer中的Full Implementation 要求和其具体步骤。

参考资料：

https://anyconnect.com/stun-turn-ice/
https://tools.ietf.org/id/draft-ietf-ice-rfc5245bis-13.html
https://tools.ietf.org/html/rfc8445
https://en.wikipedia.org/wiki/Interactive_Connectivity_Establishment
https://www.ietfjournal.org/interactive-connectivity-establishment/
https://ietf.org/documents/144/IETF_ICE_intro_92.pdf





关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC技术文档: www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

完整SIP/SDP媒体协商概论-ICE初始offer发送详解

原创 james.zhu Asterisk开源派

在前面的章节中，我们主要讨论了ICE概览，介绍了ICE的基本处理流程和候选地址配对的算法概论和轻量级ICE部署（Lite Implementations）的讨论。和前面介绍中讨论的SIP中offer的处理一样，在此文章中，笔者也将首先介绍ICE处理过程中初始offer的发送处理。因为轻量级的ICE部署场景不是RFC5245推荐的场景，本身协商也忽略了很多检查流程，所以笔者还是按照规范的的重点内容来讨论全部署场景（Full Implementations）中关于初始offer的处理，结尾部分将讨论轻量级ICE部署的场景和SDP解码。

1 全部署场景处理要求

在全部署场景中（Full Implementations），初始offer的处理大概需要经过五个主要的步骤：候选地址获取->候选地址优先级处理->移除冗余候选地址->选择默认候选地址，最后计算发送SDP offer。下面，笔者将会一步步介绍其中最主要的前四个步骤。

首先，笔者介绍一下候选地址获取或者采集（Gathering Candidates）。在日常生活环境中，如果我们计划出去旅游的话，我们一般都会提前准备旅游线路的一些背景资料，例如交通信息，旅游资料方便我们找到最佳的旅游路线，在最低成本的前提下获得最佳的旅游体验。同样，候选地址获取也是一样的。当agent之间的通信即将开始时，agent需要获取候选地址。agent或者offer提供方可以通过操作界面的提示或外部的初始请求来发起一个offer消息。每个候选地址就是一个传输地址，它也包括其类型和base。候选地址的base实际上也是一种候选地址（一个基准地址），当agent使用候选地址时，agent

必须从此地址发送。在RFC5245中，规范定义了四种候选地址类型，它们分别是host candidates，server reflexive candidates，peer reflexive candidates和relayed candidates。其中，server reflexive candidates是通过STUN或者TURN服务器获得，relayed candidates是通过TURN服务器获得，peer reflexive candidates则是通过ICE协商过程中获得（它事实上是一种连接性检查的结果，不做讨论）。关于以上四种类型的定义，笔者在前面很多历史文章中结合一些图例有一些介绍，这里为了能够更好配合SDP的介绍，保持关于SDP内容讨论的完整性，笔者再花费一点时间重新梳理一次前三种候选地址类型。

Host candidates(主机候选地址)是首先需要获取的地址。主机地址是通过端口和IP地址绑定获得。这里的IP地址可以此主机所属的物理网口IP地址，虚拟地址或者VPN地址。agent想调用UDP媒体流的话（也可以通过拓展支持TCP），agent应该获得本地主机支持的一个候选地址，这个候选地址是针对每个媒体流构件的，这些媒体流通过主机所支持的IP地址来传输。agent通过绑定一个具体的IP地址和端口来获得每个候选地址。每个媒体构件有一个component ID，基于RTP的媒体流，它本身就有一个ID，这个ID是1，RTCP的ID是2。如果agent使用了RTCP的话，它必须获取一个候选地址；如果agent使用RTP和RTCP的话，agent有k个IP地址的话，它必须以2*K的主机候选地址来最终地址计算方式。针对每个主机候选地址来说，base基准地址是候选地址自己本身。

Agent除了需要获得主机候选地址以外，agent应该获得Server Reflexive（反射地址）和Relayed Candidates（转发地址）两种候选地址。读者注意，这里使用的是**应该**而不是需要或者**必须**，使用的要求取决于提供者的环境变量要求。在有一些使用场景中，Server Reflexive和Relayed Candidates都是和公网绑定的，因此agent在一个封闭网络环

境中，或者从来不连接公网的话，没有必要使用Server Reflexive 和 Relayed Candidates地址。如果agent支持了双网络协议或者支持多宿主地址的话，agent应该使用全部署方式来处理候选地址。部署TURN服务器的成本是非常高的，需要考虑实际场景。当使用ICE时，agent双方都在NAT后，它们需要一个地址和端口映射处理时，用户才需要考虑使用NAT部署。在后续部署使用过程中，可能一些用户会把使用NAT地址映射作为一种用户场景来满足agent的支持能力，因此，通常这种可选的边缘处理的用户场景仅作为一种可选方案，可能也不会经常使用。因此，如果agent不采集Server Reflexive（反射地址）和 Relayed Candidates（转发地址）的话，RFC5245规范推荐关闭这种部署文件配置。如果将来网络环境发生了变化，agent可以通过配置开启采集地址的功能。这里**再次重复一次**笔者前面提到的废话。如果agent正在采集Server Reflexive（反射地址）和 Relayed Candidates（转发地址）的话，agent需要使用TURN服务器。如果agent仅采集反射地址的话，agent使用STUN服务器。部署场景决定以后，agent开始其候选配对的处理流程，agent可以通过界面配置方式或者其他的侦测手段（通常是通过GUI界面配置STUN/TURN服务器地址），通过STUN或者TURN服务器对其每个主机候选地址进行配对。如果已配置好了STUN或者TURN服务器的话，规范推荐配置一个domain名称，使用DNS处理流程来发现STUN服务器和TURN服务器地址。

这里的DNS处理流程遵**从两个规范（RFC5389和RFC5766）**，分别实现对STURN服务器的查询和TURN服务器的查询。其中，查询STUN服务器地址的流程是通过Service Records (SRV)，使用 **“stun”** 服务来完成的，具体的流程按照RFC 5389的DNS流程实现。查询TURN服务器地址的流程是通过SRV，使用**“turn”**服务来完成，具体的流程是按照RFC5766规范DNS流程实现。在具体的应用场景中，agent可能会通过学习从SRV查询中获得多个STUN或者TURN地址。根

据RFC5245规范的说明，为了提升ICE的处理性能，在一个具体的会话中，对于所有候选地址来说，agent应该**仅使用单个STUN或者TURN地址**。查询的结果就是通过STUN或TURN服务器的一系列主机候选配对。agent然后从一系列配对中选择一个配对，从主机候选地址对STUN或者TRUN服务器端发送绑定或分配请求。特别注意，对STUN服务器来说，发送绑定请求是不需要签权的，响应中任何服务器属性（ALTERNATESERVER）都会被忽略掉，同时agent必须支持向后兼容的模式支持绑定请求（Binding request），此绑定请求在RFC5389中说明。分配请求（Allocate requests）应该需要通过签权认证流程，agent端可以设定一定的安全机制来实现签权流程。

在ICE采集候选地址阶段，ICE设置了一个定时器（*Ta*），每Ta毫秒后，agent就会生成一个新的STUN或TURN事务。这个事务可以是针对前一个事务的重试，在这个新事务中可以携带一定的错误消息（例如，认证错误）。这个事务也可以是一个新的事务，新的事务可支持新的主机候选配对和STUN/TURN服务器配对。为了保证其ICE采集流程的稳定性，不影响其他定时器的使用（例如，STUN重传定时器RTO的），agent不应该在每Ta毫秒内过于频繁生成新的事务。关于这两种定时器的使用方式，我们在后续章节会涉及，这里不再深入讨论。发送了绑定请求或分配请求后，agent将会收到一个绑定或分配请求的响应消息。如果收到的是一个成功的分配响应消息的话，消息中会包含Server Reflexive地址（反射地址，从映射地址中获得）和 Relayed Candidates（转发地址）。其中，转发候选地址是在响应消息的XOR-RELAYED-ADDRESS属性设置中。如果分配请求被拒绝的话，那是因为服务器端没有可提供的资源来支持这个请求，agent应该对服务器端发送一个绑定请求来获得反射候选地址，绑定请求将会对agent提供一个反射候选地址（也是从映射地址中获得）。反射候选地址的基准地址是一个主机候选地址，这个主机候选地址是发送绑定请求和分

配请求的地址。转发候选地址的基准地址是候选地址本身。对主机候选地址来说，如果转发候选地址是确认状态（很少发生的极端情况），那么，这个转发候选地址必须要丢弃。

除了采集反射候选地址和转发候选地址以外，最后，agent还要对每个候选地址分配一个foundation。Foundation是一个身份标识符，它在会话中使用。Foundation ID 用来决定两个候选地址是否相同。计算foundation目前没有特定的说法，它仅针对配对的计算，保持其唯一性。如果要保证两个候选地址具有相同foundation ID的话，以下**四个条件需要都为真**：

1. 候选地址具有同样的类型（主机候选地址，转发候选地址，反射候选地址或者peer 反射地址）
2. 它们的基准地址具有同样的IP地址（端口可以不同）
3. 对反射候选地址和转发候选地址来说，通过STUN��者TRUN服务器获得这些地址，这些地址必须具有同样的IP地址
4. 通过同样的传输协议（TCP/UDP，或者其他）获得候选地址

反之亦然，如果以上其中一个条件不为真的话，则说明候选地址具有不同的foundation。

采集到反射候选地址和转发地址以后，为了保证ICE的正常工作，这种绑定关系需要一定的维护机制来保证此地址的连接性。因此，反射地址和转发地址一旦分配以后，这些地址必须保持一个存活状态。在后续章节中，笔者将继续介绍关于释放候选地址话题。对于通过绑定请求学习到的反射候选地址来说，这种绑定关系必须通过对服务器端发送其他的额外请求才能继续维持。分配请求可以通过刷新事务的方式来实现，具体的处理方式，读者可以参考RFC5766-7章节。刷新请求也同样刷新了反射候选地址。

到此为止，笔者讨论了采集候选地址所关注的几个主要话题（主机候选地址，反射地址，转发地址，foundation计算和候选地址的状态维护）。接下来，笔者将继续讨论关于针对候选地址优先级排序以及其计算方式和指导原则。

为了保证ICE的传输取得最佳的性能，候选地址需要进行优先级的处理。优先级处理针对每个候选地址指定了一个优先级标识。传输媒体流的每个候选地址必须有一个唯一的优先级，此优先级必须是一个正数，取值范围在1到(2**31 - 1)之间。ICE将使用此优先级取值来决定检查连接性的依据，和对候选地址进行修改偏好的选择参数。Agent将使用计算公式来计算其优先级，计算公式所使用的参数根据规范的指导原则进行。如果agent使用不同的计算公式的话，双方agent对连接性检查流程可能出现不协调的情况，因此，ICE将会耗费更长的时间汇聚数据交互。下面，笔者将针对计算公式和具体的指导原则进行说明。

$$\begin{aligned}\text{priority} = {} & (2 \wedge 24) * (\text{type preference}) + (2 \wedge 8) * (\text{local preference}) \\ & + (2 \wedge 0) * (256 - \text{componed ID})\end{aligned}$$

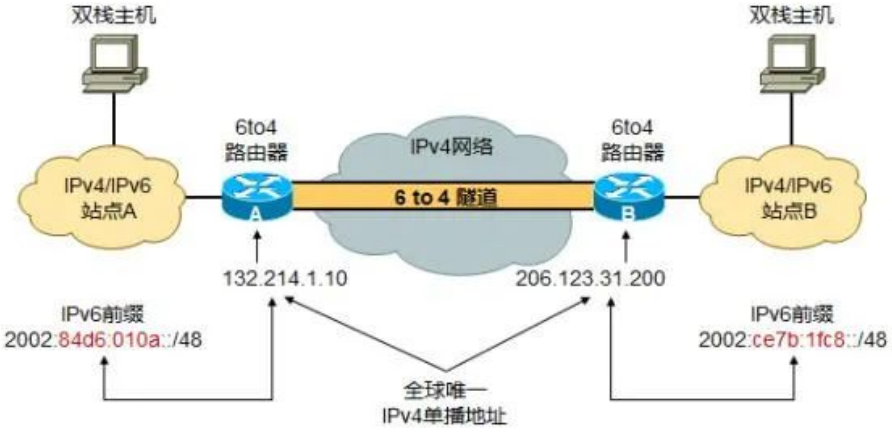
候选地址优先级计算

通过以上公式可以看出，使用格式计算优先级时，永久性的结果计算关联了三个主要的参数（针对每个候选地址类型的偏好，本地IP地址和component ID）。其中，每个候选地址类型的偏好取决于候选地址的类型（host candidates，server reflexive candidates，peer reflexive candidates 和 relayed candidates）。如果agent是一台多宿主的主机，其偏好取决于主机的IP地址。类型偏好（type preference）必须是整数，取值范围从0到126范围内，表示四种候选地址的偏好。126是最高偏好取值，0是最低偏好取值。如果对候选地址类型的偏好设置为0，则表示这个类型的候选地址将为最后的一种选择。同样类型偏好的候选地址必须要确认其相同性，不同

的类型偏好的也需要确认其不同。以上四种候选地址类型的优先级也具有不同的判断级别。其中，peer reflexive candidates的类型偏好优先级必须高于其他反射候选地址类型偏好。本地偏好（local preference）必须是整数，取值范围从0到65535范围内。它表示的偏好是针对具体的IP地址的，从这个具体的IP地址获得候选地址，agent是一台多宿主主机。65535是最高偏好取值，0是最低偏好取值。当本地主机只有单个IP地址时，偏好取值应该设置为65535。通常情况下，如果针对一个具体的媒体流的特别构件支持了多个候选地址的话，此特别构件的多个候选地址具有相同的类型的话，针对每个候选地址的本地偏好必须是唯一的。在RFC 5245规范中，以上这种情况仅发生在多宿主主机环境中。因为多宿主主机是一个双栈主机，本地偏好应该等同于IP地址的优先值。关于优先值的取值读者可以参考RFC3484-2.1，因为现在很多场景中已经部署了IPv6,，因此，可能在某些场景中，IPv6的优先级高于IPv4的优先级。具体的优先值计算取决于具体的场景中。component ID是候选地址的component ID，它的取值范围必须在1到256之间。介绍完计算公式以后，笔者接下来需要讨论选择类型偏好和本地偏好的**四个指导原则或标准**。

第一个指导原则是选择类型偏好和本地偏好需要根据一定的标准。类型偏好和本地偏好取值是主要标准，具体来说，就是媒体的中间介质的使用，例如TURN服务器，VPN或者NAT。在使用这些媒体中间介质时，如果媒体发送到这些介质的候选地址，在收到媒体之前，媒体需要首先被发送到中间介质的候选地址。转发候选地址就是其中一种候选地址类型，它涉及了媒体中间介质。另外一种媒体中间介质就是通过VPN接口获得的主机候选地址。显然，当媒体通过媒体中间介质以后，它会增加发送和接收方之间的时延。同时，因为媒体经过了多个路由器hops，增加了丢包的风险。当然，因为媒体需要进出媒体中间介质的服务器，服务提供商也需要相应增加部署成本。如果用户觉得笔者前面说的这些因素是非常重要的，因为考虑到这

些风险的重要性，转发候选地址的偏好设置就应该要低于本地偏好设置。因此，针对偏好取值的取值可能需要做一定的调整。推荐的办法是，本地候选地址偏好设置为126，反射候选地址偏好设置为100，peer reflexive candidates设置为110，转发候选地址的偏好设置为0。进一步说，如果agent是一台多宿主主机，它本身带多个IP地址的话，从VPN接口获得的主机候选地址的本地偏好设置为0。



来自于互联网资源

选择偏好的第二个指导原则是基于IP组选择方式。ICE可以在IPv4和IPv6网络环境中工作。ICE提供了一种工作机制，可以保证双栈主机选择IPv6，但是万一IPv6网络出现故障时(例如，在6to4路由器转发失败-RFC3056)，它也允许主机回退到IPv4的网络环境中。ICE也可以帮助主机获得原生的IPv6地址和6to4地址。如果是我们以上所说的这种情况的话，相对比较高的本地偏好将会关联到IPv6的地址，然后是6to4的地址，最后才是IPv4的地址。这样安排的目的是允许agent可以马上优先使用原生IPv6地址，如果出现连接问题或者对端agent没有启用原生IPv6时，可以退回到6to4的地址继续和对端通信。

第二个原则选择偏好的原则是基于主机IP地址的类型。第三个原则是基本上是基于第二个原则的基础上，针对网络最重要的问题（安全机制）的选择。因此，根据安全机制选择偏好也是一种非常重要的指导原则。现在国际上很多公司的员工都是远程办公（特别是WebRTC终端方式），如果远程办公员工提供

家庭私人网络访问企业内部网络时，员工端希望通过企业内部网络访问语音流量，通过企业通信系统呼出到其他外部目的地。这样的话，员工私人网络和企业网络之间就需要一个VPN的连接或者SBC的连接（笔者已经介绍过很多关于SBC的使用，这里仅指VPN）。如果是这样的部署场景的话，和其他后续地址相比，VPN地址将具有比较高的本地偏好优先级。

第四个选择偏好的指导原则是关于网络拓扑意识。企业网络的拓扑设计也是选择偏好时需要关注的地方。候选地址的处理如果可以灵活地充分利用网络优势也可以优化地址的选择。对于后续地址来说，它可以利用其中间介质的多种已存网络架构实现偏好选择。在某些网络环境中，如果agent可以预设或者动态发现中间介质，这些中间代理介质可能是最佳（或最近）的路径地址包括候选地址的话，agent应该设定此候选地址的本地偏好为比较高的优先级。

前面，笔者介绍了候选地址采集和候选地址的优先级处理。接下来，我们继续介绍优先级处理以后的另外一个话题，这就是关于候选地址**冗余移除**的问题。为了保证agent端本身存储和管理的问题，agent会移除一些冗余的候选地址。如何判断候选地址是重复或是一个冗余地址呢？根据RFC5245的说明，如果一个候选地址的传输地址等于其他候选地址的传输地址，并且此候选地址的基准地址等于其他候选地址的基准地址，那么此候选地址就是一个冗余候选地址。另外，如果候选地址的传输地址相同，但是它们的基准地址不同，这些候选地址不是冗余候选地址。很多情况下，当agent没有在NAT背后时，反射候选地址和主机候选地址是冗余或者重复的，agent应该设置一个比较低的偏好优先级移除这些冗余候选地址。

采集到候选地址以后，接下来就需要考虑如何设置一个默认的候选地址实现媒体流的进一步传输处理。如果一个候选地址被看作是一个默认的候选地址的话，从非-ICE用户端来说，它将

作为一个媒体目标。这个媒体目标称之为DEFAULT DESTINATION（默认目的地）。如果当agent和一个ICE-aware peer（能够感知到ICE的远端）通信时，如果ICE算法没有选择候选地址的话，ICE处理流程完成以后，agent要求一个updated offer/answer 来修复或者纠正SDP，这样的话，媒体默认的目的地地址将会匹配ICE已选的候选地址。当然，如果ICE选择了默认的候选地址，agent就没有必要发送更新的offer/answer。

Agent 必须选择一组候选地址，每个在使用的媒体流的每个构件的一个候选地址作为默认的候选地址。如果端口不是0的话，说明媒体流正在使用其构件端口。这个结论我们在前面的讨论中已经说明。接下来处理中，尽管a=inactive状态或者bandwidth设置为0，媒体仍然会置于使用状态。

RFC5245规范推荐选择默认的候选地址是基于候选地址的概率，具体来说，这些候选地址和对端peer已经关联在一起的概率，或者它们之间的一起工作的概率来决定。规范推荐的默认候选地址是，转发候选地址（如果有转发地址的话），反射候选地址（如果有反射候选地址的话）和最后主机候选地址。

2 轻量级部署要求

前面所讨论的都是基于全部署场景的内容。轻量级部署（Lite Implementation）相对比较简单，它仅使用了主机候选地址。轻量级部署必须为每个媒体流的每个构件分配0个或者一个IPv4的地址。轻量级部署它可能分配0个或者一个IPv6地址，但是，主机不能使用多余1个以上的IPv6地址。因为很多时候，每个媒体流的每个构件可以支持多个IPv4候选地址，如果agent支持多个IPv4地址的话，它必须从分配的候选地址中选择其中一个地址。如果主机支持的双栈地址的话，RFC5245推荐分配一个IPv4候选地址和一个全局IPv6地址。在轻量级的部署场景中，ICE不能用来动态选择一定范围内的候选地址。在完整的ICE流

程中，通过连接性检查才能真正决定使用这个地址或者那个地址。因此，规范也不推荐从一个特定的网络区域内包含一个以上的候选地址。

每个构件/模块都会设定一个ID，我们称之为component ID或者模块ID。对于RTP媒体流来说，RTP自己的ID为1，RTCP为2。如果agent正在使用RTCP的话，它必须首先获得一个候选地址。

每个候选地址会设定一个foundation。两个从不同IP地址获得的两个候选地址，这两个候选地址的foundation肯定是不同的，如果从相同的IP地址获得的候选地址的话，foundation看到是相同的。这里的foundation计算方式和全部署场景的要求有所不同（同时也要求检查端口）。对于优先级的计算公式来说，对每个IP地址来说仅依靠一个简单的整数递增是不够的。另外，针对同样的媒体流，在所有候选地址中，每个候选地址必须设定一个唯一的优先级标识。优先级的计算公式如下：

$$\text{priority} = (2 \wedge 24) * (126) + (2 \wedge 8) * (\text{IP precedence}) + (2 \wedge 0) * (256 - \text{componed ID})$$

轻量级部署场景中优先级计算公式

在以上格式中，如果主机仅有IPv4地址的话，IP precedence设置为65535。如果主机是IPv6地址或者是双栈地址的话，IP precedence应该是RFC3484中的precedence值。

接下来，agent将会为每个媒体流的每个构件选择一个默认的候选地址。如果主机地址仅支持IPv4地址，主机将仅对每个媒体流的每个构件选择一个候选地址，因此，这个候选地址是默认地址。如果主机支持支持IPv6或者双栈地址，默认地址的选择取决于本地策略。默认候选地址的可能是一个经常用来和对端peer通信的候选地址（参考前面章节的第四种指导原则）。简单来说，默认候选地址更多倾向于使用以前和对端通信成功的候选地址。如果主机仅支持IPv6的地址，那就是全局的IPv6

地址。对支持双栈的主机来说，RFC5245规范推荐使用IPv4地址作为默认候选地址。

3 SDP解码

关于SDP的解码处理流程，全局部署场景和轻量级的部署场景的流程是一样的。Agent将针对每个媒体包含一个m行来表示它将要使用这个媒体。SDP中的媒体顺序和ICE相关。ICE将首先执行连接性检查第一个m行检查，接下来媒体流会进行传输。如果有媒体流的话，首先agent将媒体流置于SDP中，然后处理最重要的媒体流。

针对每个媒体流，每个候选地址将设置一个候选地址属性。关于此属性的构建需要遵从一定的规则，具体的规则参考RFC5245-15。属性将会负责传递**IP地址和端口**和候选地址所使用的传输协议，另外还有配合对端工作的ICE的**一些属性**：priority，foundation和component ID。除了以上这些属性参数因为，属性中还提供了关于候选地址的**问题排查和其他函数功能**，例如，候选地址类型和相关的传输地址。

基于agent之间的STUN连接性检查中使用的认证方式是短期认证机制，具体关于认证的处理在规范RFC5389中定义。相对于短期认证方式，RFC5389还定义了长期认证的方式。短期认证设定了一个时间限定，它仅在每个时间范围内有效。长期认证是通过订阅的方式实现认证。RFC5389-10章节有非常完整的介绍，读者可以查阅此章节细节获得更多内容。短期认证的处理机制是依赖于终端和服务器端之间通过协议使用用户和密码交互的方式实现。在ICE部署场景中，终端和服务器端则通过offer/answer的模式进行交互。安全用户名称是agent用户名称，通过冒号隔离。每个agent为了发送和接收消息，它使用密码来计算消息的完整性。用户名称和用户密码的交互通过ICE的ice-ufrag和ice-pwd属性来实现。用户名称和密码是为了保证其终端的认证以外，用户名称还有另外一个重要作用。

Agent为了提供了针对媒体的安全设置，用户名称提供了针对媒体流的歧义和相关性检查。关于STUN用户名称的重要性的讨论，读者可以查阅RFC5245-B.4附录内容。

如果agent是一个轻量级的部署终端的话，agent必须在SDP中包含一个“ice-lite”的会话级属性。如果agent是全场景部署的终端的话，则一定不要包含此属性。

在SDP中添加默认的候选地址作为一个默认媒体目的地地址。RTP和RTCP的添加方式有所不同。如果媒体是基于RTP的媒体的话，把RTP候选地址中的IP地址和端口存入SDP中的c行和m行来完成。如果agent使用了RTCP的话，它必须使用a=rtcp属性对RTCP候选地址进行解码。具体RTCP的解码参考RFC3605-2.1章节。如果agent没有使用RTCP的话，agent必须说明未使用RTCP，具体的说明方式是通过b=RS:0和b=RR:0来表示。此SDP拓展是在RFC3556-2章节中声明。

当agent和一个非ICE端进行通信时，传输地址将是默认媒体目的地地址的话，这个传输地址必须作为候选地址出现在SDP中，可以以一个或者多个a=candidate行来表示。

ICE可提供拓展支持，拓展实现方式可通过在offer或answer中包含一系列的令牌来实现，agent使用其令牌可以确认ICE的拓展。具体来说，如果agent支持ICE拓展，它必须包含一令牌，使用此令牌定义这个拓展，令牌定义在SDP中使用ice-options选项属性。以下是一个具体的包含ICE消息的SDP示例：

```
v=0
o=jdoe 2890844526 2890842807 IN IP4 10.0.1.1
s=
c=IN IP4 192.0.2.3
t=0 0
a=ice-pwd:asd88fgpdd777uzjYhagZg
```

```
a=ice-ufrag:8hhY  // agnet 用户名称
m=audio 45664 RTP/AVP 0
b=RS:0  // 使用全部署场景
b=RR:0
a=rtpmap:0 PCMU/8000
a=candidate:1 1 UDP 2130706431 10.0.1.1 8998 typ h
a=candidate:2 1 UDP 1694498815 192.0.2.3 45664 typ
   10.0.1.1 rport 8998
```

一旦agent已发送了offer或者answer消息，它必须准备接收在每个候选地址上的STUN和媒体数据包。在全部署场景和轻量级部署场景中，媒体发送的流程有一定区别，更多关于不同部署场景中发送媒体包的讨论将在后续文章中做介绍。媒体包可被发送到一个候选地址中，此候选地址是在以前的offer或answer消息中出现的媒体默认目的地地址。

笔者通过以上章节的内容，重点介绍了发送初始化offer的一些流程，主要包括三个部分的内容，其中包括了全部署场景中的要求（采集候选地址，候选地址排序，移除冗余候选地址，选择默认候选地址），轻量级部署要求，SDP解码。

在下一个章节中，笔者将介绍接收初始化offer的处理流程。

参考资料：

https://tools.ietf.org/html/draft-ietf-mmusic-ice-sip-sdp-39
https://www.rfc-editor.org/rfc/rfc5245
https://www.rfc-editor.org/rfc/rfc8445
https://www.rfc-editor.org/rfc/rfc3264





关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC 技术文档：www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文　　在看

完整SIP/SDP媒体协商概论-ICE初始offer接收详解

原创 james.zhu Asterisk开源派

在前面的章节中，笔者具体讨论了关于发送初始offer的细节。这里，我们将讨论接收初始offer的一些具体内容。关于接收初始offer的流程主要包括几个部分的讨论：验证ICE支持，决定主控/被控方角色，采集候选地址，候选地址优先级排序，选择默认的候选地址，SDP解码，检查列表的构建和定时检查。其中，在接收offer的内容中，候选地址采集，候选地址排序，选择默认候选地址和SDP解码和前面关于发送offer的流程非常相似，因此，在本章节中，这些内容的介绍可能相对比较简洁，笔者将花费更多时间在验证ICE，决定接收角色，检查列表构建和定时检查的讨论中。下面，我们将根据agent接收offer的处理步骤，开始讨论这些具体的处理流程。

1 验证ICE支持能力

Agent收到初始offer以后，首先，它需要验证ICE的支持能力。对于每个在SDP中收到的媒体流来说，如果媒体流的每个构件的默认目的地地址出现在了候选地址属性中，agent将会根据RFC5245规范中ICE的处理流程来进行处理。如果我们进一步做具体理解的话，我们可以参考RTP的处理方式来说明。例如，使用RTP时，在c行和m=行的IP地址和端口出现在了候选地址属性参数中；使用RTCP时，RTCP值也会出现在候选地址属性中。如果前面所说的这个条件不成立，或者这些媒体构件没有出现在候选地址属性中的话，agent必须按照另外一个流程来处理SDP（RFC3264）值，而无需按照ICE的处理机制来处理。这个流程就是我们在前面文章中所介绍的关于SIP针对offer/answer交互模式来处理。读者可以查看历史文档来回顾笔者介绍的相关基础内容。如果按照RFC3264处理的话，读者也需要注意几个例外条件：

1. 所有agent必须遵从RFC5245-10中的会话存活保持流程。
2. 如果agent没有根据RFC5245 ICE的处理流程处理的话，是因为有a=候选地址属性，但是这些a=候选属性没有匹配媒体流的默认目的地地址，agent在其answer消息中必须包括一个a=icemismatch属性（无匹配）。
3. 如果默认候选地址是一个从TURN服务器学习获得的转发地址，agent必须在TRUN服务器端创建一个授权许可，这个许可支持SDP中收到的，从对端的peer学习到的IP地址。大家需要注意，如果权限设置有问题的话，对端发过来的初始数据包将会丢失。

2 决定主控/被控制方角色

在上一篇文章中，笔者介绍了主控agent和被控方agent的角色。在会话中，每个agent需要充当一个角色来执行不同的操作流程。主控方负责选择最终候选配对和被控方进行通信，如

果是全部署环境下的agent，这表示ICE使用挑选的候选配对对每个媒体流进行传输，并且，当agent需要时，可基于ICE的选择生成更新的offer消息。如果是轻量级的部署环境中的agnet，选定为主控方agent表示选择了候选地址配对，这个候选配对是基于offer和answer消息中的配对（在IPv4中仅有一对），并且，当agent需要时，生成一个更新的offer消息来反映这个选择。被控方agent被告知使用的候选配对来传输媒体流，并且针对这个单个信息不会生成一个更新offer。下面，笔者将讨论决定双方角色的规则和处理流程对双方的的影响。关于决定双方角色的规则和其流程的影响，事实上，这取决于双方agent所处的部署环境，这里有三种不同的部署环境需要讨论：双方都在全部署场景，双方各自在全部署场景/轻量级部署场景，双方都在轻量级部署场景。

双方都在全部署场景中，如果一个agent生成了一个offer消息，并且启动了ICE处理流程，这个agent必须扮演一个主控方agent的角色。另外一侧agent则必须扮演被控方角色。双方agent将会构建检查列表，运行ICE状态机和连接检查的流程。主控方将会根据全部署场景流程执行处理逻辑，挑选候选配，ICE根据这些候选配对提供进一步的选择，最后双方agent通过更新offer来更新或者结束ICE处理流程。当然，在实际部署场景中，不排除一些特殊使用环境，例如因为其他通信因素，双方的角色认定发生冲突。双方agnet错误地认为自己是主控方或者自己是被控方。因此，为了避免类似情况的发送，每个agent必须选择一个任意号码，RFC5245规范称之为tie-breaker，在连接检查中使用此数值检测修复这种情况，其取值范围一律发布在0和(2**64) - 1之间（它是一个64位的正整数）。

双方一方是全部署场景agent，另外一方是轻量级部署场景agent中，全部署场景的agent必须扮演主控方agent的角色，轻量级agent则必须扮演被控方agent的角色。全部署agent将

会构建检查列表，运行ICE状态机和生成连接检查。主控方将会根据全部署场景流程执行处理逻辑，挑选候选配对，ICE根据这些候选配对提供进一步的选择。轻量级agent将会监听连接检查，接收响应检查消息，按照轻量级部署的结束ICE处理流程，最后对ICE进行结束处理。因为双方的角色不同，从某种程度来说，主控角色一般都是一直处于运行状态。因此，对轻量级部署agent来说，针对每个媒体流来说，ICE处理状态被认为是运行状态，所有ICE流程也是运行状态。

小广告



Agent双方都是轻量级部署的agent的话，如果一个agent生成了offer消息，并且启动了ICE处理流程，这个agent必须扮演一个主控方agent的角色。另外一侧agent则必须扮演被控方角色。这种环境中，双方从来都不会发送连接检查。准确地说，一旦双方的offer/answer交互模式完成以后，每个agent将执行ICE结束处理流程，它们无需经过连接检查的流程。和第一种场景中所描述的一样，同样的角色冲突问题也可能出现在双方都是轻量级部署agent的环境中。它们都可能认为自己是主控方agent或者被控方agent。这种情况下的处理方式和全场景部署中的角色决定的处理方式不同。轻量级部署环境中角色冲突时，双方agent通过在信令中承载的offer/answer交互，和交互消息中所支持的检测能力来确定双方角色。对轻量级部署agent来说，针对每个媒体流来说，ICE处理状态被认为是运行状态，所有ICE流程也是运行状态。

在会话中，一旦双方角色确定以后，除非ICE重新启动，否则，它们将一直持续充当各自的角色。补充说明，因为ICE重新启动

以后，双方需要重新决定各自的角色，如果是全部署场景agent的话，它们需要重新对tie-breaker赋值计算。

3 候选地址采集

关于针对候选地址的采集处理流程，answerer应答方和offerer提供方的处理方式是完全一样的。笔者在前面的文章中已经非常详细地做出了说明。用户可以阅读此文章来了解全部署场景中的场景流程和轻量级部署的要求等内容。根据RFC5245的推荐，提供方收到offer，早于对用户提醒之前马上执行采集流程。当agent启动时，这样的候选地址采集方式就可能开始。

4 候选地址优先级排序

关于针对候选地址的排序处理流程，answerer应答方和offerer提供方的处理方式是完全一样的。笔者在前面的文章中已经非常详细地做出了说明。用户可以阅读此文章来了解全部署场景中的场景流程和轻量级部署的要求等内容。

5 默认候选地址选择

关于针对候选地址的默认候选地址选择的处理流程，answerer应答方和offerer提供方的处理方式是完全一样的。笔者在前面的文章中已经非常详细地做出了说明。用户可以阅读此文章来了解全部署场景中的场景流程和轻量级部署的要求等内容。

6 SDP解码

关于针对SDP解码的处理流程，answerer应答方和offerer提供方的处理方式是完全一样的。笔者在前面的文章中针对全部署场景和轻量级部署场景的规定已经非常详细地做出了说明。用户可以阅读此文章来了解全部署场景中的场景流程和轻量级部署的要求等内容。

7 检查列表构建

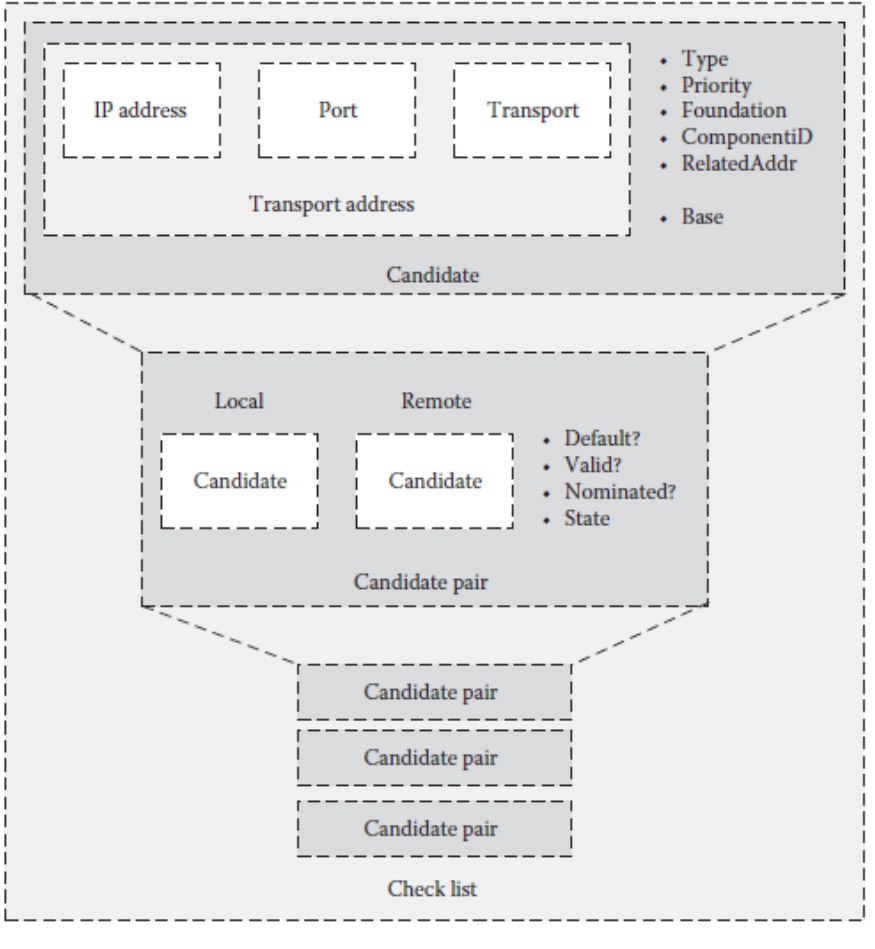
构建检查列表是由全部署场景来实现的，如果是轻量级的部署场景，无需构建检查列表。因为offer/answer交互模式的使用，在媒体使用的过程中需要一个检查列表。为了对媒体流构建一个检查列表，agent需要经过几个必要的步骤来构建检查列表，agent需要经过的步骤是：构建候选地址配对，计算候选配对优先级和排序，优选配对，最后计算配对状态。接下来，笔者将分别讨论这四个主要的步骤。

首先，为了实现对媒体流的支持，agent选择自己本地候选地址和从对端peer收到的候选地址进行配对处理。这里，本地候选地址称之为LOCAL CANDIDATES，远端的候选地址称之为REMOTE CANDIDATES。选择过程中，agent同时还要考虑安全的问题，为了防止选择的地址被攻击，agent可以设置从offer或answer中接收候选地址的数量。关于STUN被攻击的可能性讨论，请读者参考RFC5245-18，后期文章中我们将讨论这个话题，现在不做讨论。如果本地候选地址和远端候选地址配对成功的话，它们必须具有相同的component ID，和同样的IP地址版本。当然，实际环境中，本地候选地址和远端候选地址也完全可能存在不匹配或者匹配不成功的可能，或者，远端候选地址和本地候选地址匹配不成功的可能。有时也可能发生这样的问题，例如，针对一个媒体流来说，agent没有包含候选地址来支持此媒体流的所有构件模块。如果是这样的情况的话，此媒体流的构件数量就会受到影响而减少，并且会认为这个数量等于双方agent所要求的构件最低数量，此最低数量值针对媒体流的所有component构件，并且来源于双方agent所提供的最大component ID（关于ID取值范围读者可参考前面的介绍和历史文档）。

除了上面所介绍的一些情况以外，还有几个比较特殊的情况和读者说明。在涉及到RTP/RTCP的使用场景中有可能发生这样的情况，一方agent提供了RTCP的候选地址，但是对端可能没

有提供类似地址。有时，offer消息提供方可在同一端口多路复用RTP/RTCP数据，并且通过SDP属性中指示了这样的实现方式（多路复用RTP/RTCP参考RFC5761-5和RFC8035）。但是，如果answerer执行了多路复用RTP/RTCP的话，offerer则不知道answerer执行了这样的流程，offerer就会按照默认的设置方式使用各自独立的RTP和RTCP，这样处理的结果就会导致每个媒体流中在offer消息中包含两个components。answerer方执行了多路复用RTP/RTCP，对每个候选地址来说，它将仅包含单个component。因此，此component将是RTP/RTCP mux的合并值。如果此候选地址只有单个component的话，ICE结束执行候选地址配对。毫无疑问，如果配对中本地候选地址和远端候选地址都是默认候选地址时，这个候选配对被认为是默认的候选配对。

如果agent双方不是ICE感知的agent的话，媒体流构件使用此默认候选配对传输媒体流。读者可以参考以下示例来理解check list（检查列表）核心概念和与其他模块之间的关系。



check list列表和其他模块之间的关系汇总

构建候选地址配对完成以后，需要进行后续地址配对的优先级计算。优先级计算是根据以下格式来计算的。其中，G表示主控方agent提供的候选优先级，D表示被控制方提供的候选地址优先级。这里，**G>D?1:0**是一个表达式，如果G大于D，则取值为1，否则为0。

```
pair priority = 2 ^ 32 *MIN(G,D) + 2 *MAX(G,D) + (G
```

关于以上优先级的计算，很多开源项目有类似的处理方式，读者可以参考一些开源项目来做进一步了解。以下一段代码是一个配对计算源代码示例，读者可以参考：

```
// PairPriority computes Pair Priority as in RFC 8445
func PairPriority(controlling, controlled int) int64 {
var (
g = int64(controlling)
d=int64(controlled) )
// pair priority = 2^32*MIN(G,D) + 2*MAX(G,D) + (G>D?
1:0) v := (1<<32)*min(g, d) + 2*max(g, d)
if g > d {
v++ }
return v }

https://github.com/gortc/ice/blob/master/pair.go
```

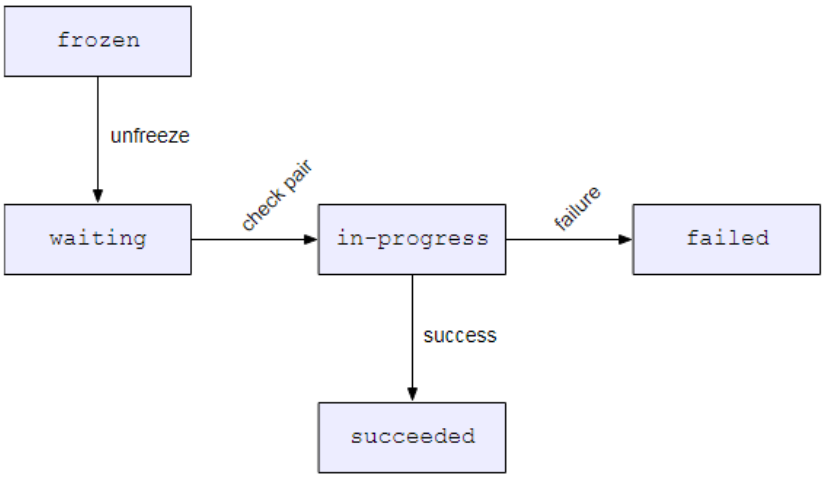
一旦优先级被设定以后，agent就会按照顺序对候选地址配对进行排序。排序的规则按照优先级顺序递减的方式进行。如果两组候选地址配对有相同的优先级，它们两组配对的实现可以任意排序。

获得了候选地址配对排序以后，此优选候选地址会生成一个排序后的配对列表。ICE将会按照排序列表逐一进行连接检查。在

每个检查流程将会涉及发送请求的流程，agent需要从本地候选地址发送检查请求到远端候选地址。这里注意，因为agent不能直接从反射地址发送请求，它仅能从base基准地址发送请求，所以agent需要通过排序后的配对列表来发送请求。对每个配对来说，如果本地候选地址是一个反射地址的话，base基准地址必须替换这个反射候选地址。一旦替换流程完成后，agent必须过滤或筛选此列表。过滤配对列表的流程通过移除配对的方式来实现。具体来说，如果它（需要移除的配对）的本地候选地址/远端候选地址和优先级列表中较高优先级的一对配对相同的话，则需要移除配对相同的较低优先级的配对。

另外，为了安全的考虑，防止STUN服务器被攻击，agent必须限制连接检查数量，在一定的数值设置环境中，agent的检查将会覆盖所有连接列表的地址，这个特定数值必须是可配置的数值。规范RFC5245推荐默认数值是100。候选配对列表一直保持低于100的限定设置，一些低优先级的候选配对将被强制丢弃。在可能的情况下，RFC5245推荐尽量使用比较低的设置限定，在实际生产环境中也可能看到一些用户针对配对检查设置了最大检查限定。要求支持可配置的限定设置也是为系统提供了一个工具，如果发现问题以后，可以通过此限定值来排查问题。

完成了候选配对地址的挑选以后，检查列表需要进行状态计算。每个候选配对支持了一个foundation和一个state（状态）。实际上，这里的foundation是合并了本地候选地址的foundation和远端的候选地址的fundation而生成的一个新的fundation。一旦开始计算foundation时，每个配对将会设定一个状态值，根据检查结果的不同，候选配对需要经过五个可能或潜在的状态计算（历史文章也有介绍这五个计算步骤）。


frozen
unfreeze
waiting
check pair
in-progress
failure
failed
success
succeeded

候选配对状态计算

ICE开始运行时，一个候选配对将迁移到以下任何一种状态。笔者再次重复一次每个状态的任务然后介绍具体流程处理：

- Frozen：处于锁定状态，等待检查

- Waiting：检查还没有启动，等待从check list中选择最高优先级的候选配对进行处理

- In-Progress：为这个配对发送check请求，事务在处理流程中

- Succeeded：配对检查成功，生成成功结果

- Failed：配对检查失败，既没有生成任何响应也没有生成任何还原响应

在检查列表中的每个候选配对的初始状态需要经过一个状态计算。其状态计算需要按序经过以下几个步骤：

1. agent将会把每个检查列表中所有的候选配对设置为Frozen 封冻状态。

2. agent将会为第一个媒体流查询检查列表（第一个媒体流出现在SDP offer和answer中的第一个m行中）。然后针对此媒体流做状态设定处理。对所有具有同样foundation的配对，agent将会设置一个比较低级别component ID的配对状态，设定这些配对进入等待状态。如果有多个类似这样的配对，agent则首先使用具有最高优先级的配对。

这里有两个特定的列表称谓需要读者注意。其中检查列表中的一部分配对会进入等待状态，另外一部分则进入到封冻状态。如果检查列表中**至少有一对配对是在等待状态**的，这样的列表称之为**活动检查列表**。如果检查列表中**所有配对**都是封冻状态的，这样的列表称之为**封冻检查列表**。

除了检查列表中的配对检查有状态存在，检查列表自己本身也关联一个状态，针对正在工作的媒体流，这个状态用来捕捉ICE检查的状态。检查列表具有三种状态：

1. 运行状态：针对正在运行的媒体流，ICE检查状态也在运行中。

2. 完成状态：在这个状态下，ICE检查已经生成了一个经过挑选的配对支持媒体流构件模块。接下来，ICE成功完成处理任务，开始发送媒体流。

3. 失败状态：在这个状态下，针对此媒体流的支持，ICE检查还没有成功。

作为一个offer/answer交互的结果，检查列表首先构建的就会被ICE迁移到运行状态中。ICE处理流程覆盖所有的媒体流过程中，因此，ICE也和这个流程本身有一个关联绑定的状态。当ICE运行时，这个状态等于运行状态。当ICE处理流程完成后，这个状态将是完成状态，如果ICE处理流程失败的话，这个状态就是失败状态。关于以上几个状态之间切换的规则笔者将在定时检查的讨论中做更多说明。

8 定时检查

前面我们一直在讨论关于check的流程，check是由全部署场景生成的，所以，如果agent是轻量级的部署方式的话，可以跳过此内容讨论。Agent执行两种check，一种是ordinary checks，另外一种是triggered checks。两种check都是由一个定时器来控制，针对媒体流数据，定时器会周期性地触发check生成事件。

除了定时器以外，agent也会维护一个先进先出的队列，这个队列称之为triggered check队列，队列维护候选配对，如果有check的机会的话，这个队列将会发送配对进行检查。当定时器触发以后，agent将会从triggered checks队列顶部移除一个配对，对这个配对执行连接检查，最后把这对候选配对的状态设置为正在处理的状态（In-Progress）。如果在triggered checks队列中没有配对的话，agent将会发送ordinary checks。

一旦完成构建检测列表计算的流程，agent将会针对每个active check list设置一个定时器。每Ta*N秒触发一次定时器，这里的N是active check list的数量（初始阶段，至少有一个active check list）。Ta和RTO这两个定时器会在未来的讨论中加以介绍，这里不再展开讨论。Ta乘以N将允许整个check的吞吐量发布到所有active check list中。在实际部署环境中，这个定时器触发的频率可以低于上面的设置，同时，也应该考虑定时器传播的问题，尽量不要同时对每个媒体发布定时器。第一个定时器马上触发后，agent在这一瞬间（offer/answer交互模式已完成）执行连接检查，然后在Ta秒后执行下一个检查（因为在第一个定时器触发时只有一个active check list）。

如果定时器触发以后，agent没有triggered checks需要发送的话，它必须按照以下规则选择一个ordinary checks：

1. 找到在等待状态的check list中最高优先级的配对（注意以下两种状态下配对处理流程）。
2. **如果有这样的配对的话**：首先，从本地候选配对发送一个STUN check请求到远端候选配对。此STUN check请求将会进行相关处理。关于STUN check请求的处理流程，笔者在后续文章中介绍。然后对候选配对的状态进行设置，设置其状态为In-Progress状态。
3. **如果没有这样的配对的话**：agent需要找到**在封冻状态**的check list中最高优先级的配对。如果**有这样的配对**的话，对此配对解除封冻状态，然后对此配对执行check流程，使其状态切换为In-Progress状态。如果**没有这样的配对**的话，结束针对此check list的定时器。

配对流程完成以后，需要保证其检查数据的完整性。如果要计算check消息的完整性，agent需要使用远端用户名称和密码来完成计算。远端用户名称和密码是agent学习远端peer发送的SDP中的用户名称和密码获得。

在接下来的章节中，笔者将继续分享初始应答接收的话题（验证ICE支持，决定角色等话题）。

参考资料：

https://www.rfc-editor.org/rfc/rfc5245
https://www.rfc-editor.org/rfc/rfc8445
https://developer.mozilla.org/en-US/docs/Web/API/RTCIceCandidatePairStats/state
https://github.com/gortc/ice/blob/master/pair.go



关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC 技术文档：www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文

在看

完整SIP/SDP媒体协商概论-初始应答接收详解

原创 james.zhu Asterisk开源派

此文章将讨论agent的应答接收流程。agent收到应答从远端peer以后，它需要按照一定的流程来进行处理。具体的处理流程包括验证ICE支持能力，决定双方的角色，构建检查列表，最后执行ordinary checks。

特别说明，如果使用SIP协议进行协商的话，当发生SIP分叉处理时，一个单个offer可能会生成多个answer消息。在这种情况下，ICE针对每个answer进行完全独立并行处理，把这个offer和其每个answer的合并体看作一个独立的offer/answer交互模式。合并的offer/answer交互模式生成自己独立的候选配对，检查列表，状态计算等属性。其中，当释放候选地址时，这些独立的answer可能会和其他answer产生相互影响。关于这个影响的处理流程，读者可参考RFC5245-8。

下面，笔者将会按照初始应答的接收流程来一步步介绍这些具体的细节。

1 验证ICE支持能力

应答方的ICE支持能力的验证和前面笔者介绍的offer中的验证流程基本一样，一个另外就是offerer提供方从来不会在SDP中生成一个a=ice-mismatch属性。

在一些使用场景中，answer可能忽略媒体流中的a=candidate属性，但是会包含a=ice-mismatch属性，此属性用来支持一个或多个媒体流。如果是这样的设置的话，对offerer来说，

answerer提供ICE支持，但是在此会话中并没有使用ICE处理流程。会话中没有使用ICE处理的原因是针对媒体构件来说，因为中间信令修改了默认目的地地址，但是没有修改相应的候选地址属性。这样的情况是完全可能发生的，可能会导致安全问题。这里涉及了关于ICE的安全问题，笔者在未来文章中会进行讨论，这里不再介绍。另外，如果类似这样的场景发生的话，RFC5245并没有提供一个明确的指导说明来说明agent如何处理这样的失败场景。

2 决定主控/被控方角色

关于应答中角色决定的处理流程，读者可以参考offer中的处理流程，流程完全一致，无特别说明。

3 构建检查列表

构建检查列表也是针对全场景部署agent来定义的，关于应答中构建检查列表的处理流程，读者可以参考offer中的处理流程，流程完全一致，无特别说明。

4 执行 ordinary checks

执行ordinary check的流程也是针对全场景部署agent来说的，读者可以参考offer中的处理流程，流程完全一致，无特别说明。

在接下来章节，笔者将讨论关于连接性检查的具体细节，包括STUN客户端流程流程和服务器端流程。

参考资料：

https://www.rfc-editor.org/rfc/rfc8445
https://developer.mozilla.org/en-US/docs/Web/API/RTCIceCandidatePairStats/state
https://github.com/gortc/ice/blob/master/pair.go





关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC 技术文档：www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文

在看

完整SIP/SDP媒体协商概论-STUN连接检查-服务器端流程

原创 james.zhu Asterisk开源派

在上一篇文章中，笔者讨论了STUN 连接检查的客户端的处理流程。接下来，在本章节，笔者继续讨论关于服务器端的处理流程。

客户端发送绑定请求后，对端agent必须准备好接收绑定请求，这个绑定请求是发送到了每个候选地址的基准地址。绑定请求包含在自己最近的offer或者answer消息中。因此，即使agent是一个轻量级部署场景的agent，它也要求部署方案维持一个状态来处理服务器端的流程。

接收绑定请求时，agent必须使用短期安全机制对请求进行认证和消息完整性检查。Agent必须考虑用户名称的有效性，如果用户名称含有两个用户名称构成的（通过冒号：分割），其中第一个用户名称等于一个用户名称，这个用户在此会话中，由agent在offer或者answer生成的名称。关于名称构成的讨论，读者可以查阅上一篇文章。有时也存在其他的可能性，例如，提供方offerer在收到对端peer的answer之前一个收到一个绑定请求。如果这样的情况发生，agent必须马上生成一个响应消息，响应消息中包含映射地址的计算数据。在这一时间点，agent已经具备了足够的信息生成响应消息。因此，agent就不需要对端peer的密码。一旦agent收到answer以后，这个answer消息就会马上通过几个步骤进行处理，包括检查修复角色冲突的流程，计算映射地址的流程，通过学习获得peer反射候选地址，Triggered Checks和更新推荐标识。提醒读者，以上这些流程都是针对全部署场景agent来说的。还有一些场景

中，在收到answer之前，收到了多个STTUN请求，这样就会导致在triggered check 队列中生成多个配对队列。

小广告



Agent一定不能使用ALTERNATE-SERVER机制，也一定不能支持RFC3489规范的向后兼容机制，它必须使用FINGERPRINT机制。

如果agent在媒体数据中（例如在QoS中）使用了区分服务-Diffserv（遵从RFC2475），agent也应该使用同样的标识方式在绑定请求的响应中。因为区分服务不在我们讨论的范围，具体关于Diffserv，读者查阅RFC2475，这里不做进一步讨论。以下几个步骤的讨论将仅涉及全部署场景的agent处理流程。

1 检测修复角色冲突

正常情况下，通过agent的主控/被控方选择，双方可以选择自己的成功的角色。但是，在一些非正常的环境中，例如使用了第三方的呼叫控制，双方都成为同样的角色。下面，笔者将讨论如何检查同一角色冲突问题，以及如何修复同一角色冲突问题。

Agent必须检查绑定请求，其属性可能是ICE- CONTROLLING或ICE-CONTROLLED 属性。如果要检查以上两种属性，agent需要按照以下流程进行：

- 如果在请求中，其属性既不是ICE-CONTROLLING，也不是ICE-CONTROLLED属性的话，说明agent没有遵从

RFC5245规范，这里有角色冲突的问题，但是agent目前执行不了检测。

- 如果agent是一个被控方角色，并且请求中出现了ICE-CONTROLLING属性，接下来根据具体属性内容大小进行判断：

  如果agent的tie-breaker大于或等于ICE-CONTROLLING的内容，此agent生成一个绑定错误响应，错误响应中包含一个ERROR-CODE属性（487-角色冲突），这里，agent角色仍然保持不变。

  如果agent的tie-breaker小于ICE-CONTROLLING的内容，agent切换到主控方agent角色。
- 如果agent是一个主控方agent角色，并且请求中出现了ICE-CONTROLLED属性，接下来根据具体属性内容进行判断：

  如果agent的tie-breaker大于或等于ICE-CONTROLLED的内容，agent切换到被控方角色。

  如果agent的tie-breaker小于ICE-CONTROLLED的内容, 此agent生成一个绑定错误响应，错误响应中包含一个ERROR-CODE属性（487-角色冲突），这里，agent角色仍然保持不变。
- 如果agent是主控方角色agent，并且ICE-CONTROLLING属性出现在了请求中，或者agent是被控方角色agent，并且ICE-CONTROLLED属性出现在了请求中，双方角色无冲突。

修改了agent角色以后，需要对角色的状态进行修复。因为各自角色的优先级是主控方和被控方的主要执行功能，因此agent修改属性不是简单切换角色就完成了工作流程。修改角色需要agent重新计算配对优先级。这个计算方式我们在前面的历史文章中有非常详细地介绍，读者可查阅此文章。agent角色切换或者修改同样也会直接影响agent其他后续的功能执

行，例如，它是否负责选择推荐配对，是否基于ICE结果生成更新offer的消息。

假设STUN服务器端对绑定请求生成了成功的响应的话，甚至于agent角色也发生了变化，agent可以继续执行其余服务器端的步骤：计算映射地址，通过学习获得peer反射候选地址等流程。

2 计算映射地址

对于在转发候选地址收到的请求来说，被STUN流程（即XOR-MAPPED-ADDRESS属性生成）使用的源传输地址是一个传输地址，这个传输地址是被STUN认为的传输地址。读者在计算映射地址是一定要注意，这里涉及了两种绑定请求的传输方式（Data Indication和ChannelData）。如果绑定请求通过Data Indication传输的话，这个源传输地址会出现在Data Indication消息的XOR-MAPPED-ADDRESS属性中。如果绑定请求通过ChannelData传输的话，这个源传输地址是绑定此channel通道的地址。关于Data Indication消息和ChannelData的定义细节，读者查阅以下两个规范草案：

1. Data Indication：https://tools.ietf.org/id/draft-rosenberg-midcom-turn-08.txt
2. ChannelData：https://tools.ietf.org/html/rfc5766-10

3 学习Peer Reflexive Candidates

如果请求中的源传输地址不能匹配任何现存远端候选地址的话，这说明对端的是一个新的peer反射候选地址。这个候选地址需要通过以下方式进行构建：

1. 此候选地址的优先级设置为请求中获得的PRIORITY 属性值。
2. 候选地址类型设置为peer reflexive。

3. 候选地址的foundation设置为一个任意值，这个foundation必须区别于所有其他远端候选地址的foundation。如果在SDP的后续的offer/answer交互中包含了peer reflexive候选地址，这将表示对此候选地址来说，这是一个真实的foundation。
4. 此候选地址的component ID设置为一个component ID，这个ID是为本地候选地址到远端地址（发送请求的地址）。

构建候选地址完成后，这个候选地址将被添加到远端候选地址列表中。但是，agent还不会使用本地候选地址和这个远端候选地址进行配对处理。在triggered check流程会进行本地候选地址和其配对的处理。

4 Triggered Checks处理

下一步，agent将会构建配对，在此配对中，本地候选地址等同于STUN请求接收的传输地址，远端候选地址等同于源传输地址（这是请求发出的地址），这个地址可能是通过学习获得的一个peer reflexive 远端候选地址。本地候选地址将是主机候选地址（这种情况下，请求可能不是通过转发候选地址获得），或者本地候选地址是一个转发候选地址（这种情况下，请求通过转发地址获得）。此本地候选地址从来不会是服务器端反射地址。因为这两个候选地址对agent来说都是可知的，所以，agent可以获得它们的配对，然后计算其候选配对优先级。接下来，这个配对会查询检查列表。通过查询检查列表可能获得其中一个结果：

- **如果配对已在检查列表中**，继续执行以下四种检查：

  如果配对在**等待或封冻状态**，如果检查没有出现在triggered check队列中，把此配对的检查流程加入到triggered check队列中。

  如果配对在**In-Progress 状态**，agent将会取消这个事务处理。这里取消的意思是agent将不在重传请求，将不会

认为缺乏响应是失败的，但是会在事务超时时间内对此响应执行等待流程。另外，通过对agent进行队列排序，此agent将会被加入到triggered check队列中，因此会让agent创建一个新的连接检查，然后，agent必须对此配对创建一个新的连接检查（重新作为一个新的STUN绑定请求事务）。最后，这个配对的状态切换为等待状态。

如果配对状态是**失败状态**，此状态必须切换为等待状态，并且通过把agent加入triggered check队列，由此触发agent创建一个新的连接检查。接下来，agent必须为这一对配对创建一个新的检查连接（重新作为一个新的STUN绑定请求事务）。

如果配对状态是**成功状态**，agent则无需执行进一步处理步骤。

当双方agent在NAT背后时，通过以上步骤处理来支持ICE的快速处理。

- **如果配对不在检查列表时**，需要进行执行以下流程：

  基于配对优先级，配对会插入到检查列表中

  配对状态设置为等待状态

  配对会加入到triggered check队列中

加入到triggered check队列以后，triggered check队列将会被发送出去。当triggered check队列发送以后，其构建和处理的流程按照前面讨论的发送请求来处理。具体关于发送请求的详情，读者可参与笔者上一篇文章，或者查阅RFC5245-7.1.2，这里不再重复。这些流程要求agent获得一个传输地址，部分用户名（username fragment），对端peer密码。这里读者还要再次注意用户名称和密码的设置。远端候选地址的用户名称（username fragment）等于收到的绑定请求中，冒号后面的USERNAME名称，agent使用此用户名称检查从peer收到的SIP消息（可能从多个分叉中检查），然后找到此

用户名称（username fragment），然后通过此用户名称选择相应的密码。关于用户名称和密码构成的处理方式，读者可以参考上一篇文章中的介绍。

5 更新推荐Flag标识

如果必要，配对的推荐方式标识也是需要更新的。如果agent收到了一个绑定请求，绑定请求中已有一个USE-CANDIDATE设置属性，并且这个agent是一个主控方agent，agent将会查看配对状态（根据上一个讨论中的计算方式），并且继续进行判断处理：

- 如果这个配对状态是成功状态，这表示由这对配对生成的检查流程生成了一个成功的响应。这会引起agent的一个更新处理。当成功响应收到以后，agent会构建一对有效配对。关于构建有效配对的流程，读者可以参考笔者的客户端流程处理的文章。构建配对后，agent会将配对的推荐标识设置为true值。这样的设置方式可能会结束此媒体流的ICE处理流程。
- 如果配对状态是In-Progress状态的话，如果agent的检查流程生成了成功的结果，当响应抵达时，随之生成的有效配对已确定了的推荐标识设置。当响应抵达时，这样的设置方式可能会结束此媒体流的ICE处理流程。关于ICE结束处理流程的讨论，笔者将在下一篇文章做做详细说明。

6 针对Lite部署的额外流程处理讨论

本文章中以上讨论的都是基于全场景部署的agent的讨论。轻量级的部署场景agent的处理相对比较简单，使用的场景也不是非常普遍，因此没有太多的约定条件（没有优先级，foundation等计算设置，没有队列检查等流程）。这里，笔者针对轻量级部署场景的agent做一些简单说明，希望读者引起注意。如果收到的check消息中包含了USE-CANDIDATE设置属性，agent将需要构建一个候选配对。其中候选配对中，本地候选地址等于传输地址（收到请求的地址），远端候选地址

等于一个源传输地址（注意，这个地址是收到请求的源传输地址）。构建成的候选配对设定一个任意优先级，然后推送到有效候选列表中（valid list）。Agent然后设置这对候选配对的推荐标识为true。针对媒体流的每个构件模块，如果这个有效候选列表包含了一对候选配对，媒体流的ICE处理流程将被视作完成状态。

到此为止，结合上一篇文章中关于STUN客户端处理流程介绍，笔者已经完成了关于ICE连接检查的讨论（客户端处理流程和本章节的服务器端处理流程）。在接下来的章节中，笔者将重点讨论关于ICE结束处理的流程。

参考资料：
https://tools.ietf.org/html/rfc5766
https://www.rfc-editor.org/rfc/rfc8445



关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC技术文档: www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文

在看

完整SIP/SDP媒体协商概论-STUN连接性检查-客户端流程

原创 james.zhu Asterisk开源派

在前面章节中，无论是关于发送offer还是关于接收answer消息的讨论中都没有离开一个重要的步骤，那就是执行连接性检查（connectivity checks）。因为侧重点的不同，笔者在前面的章节没有详细介绍连接性检查的具体细节，在这个章节，我们重点讨论连接检查的具体内容和执行流程。

所有ICE的部署需要符合RFC5389中STUN的规范（已经更新为RFC8489）。大家知道，全部署场景的agent才是ICE部署中核心内容。全部署场景agent需要扮演STUN客户端（生成检查）和STUN服务器端（接收检查）的两种角色。轻量级的agent则作为服务器端只能接收检查（check）。因此，我们所讨论的内容将以SUTN客户端生成流程和服务器端接收流程两个部分的内容进行讨论。其中，STUN客户端流程主要包括：

- 创建权限支持转发候选地址
- 发送请求
- 处理响应

服务器端主要流程包括：

- 全场景部署agent的其他额外步骤处理
- 轻量级部署agent的其他额外步骤处理

笔者会根据以上的内容逐一进行讨论。首先，笔者将介绍关于STUN客户端的流程处理。

1 STUN客户端流程总览

这里，笔者首先说明一下关于STUN客户端的发送流程。STUN客户端发送连接检查（connectivity checks），确认连接检查是一个ordinary还是triggered check。另外，笔者说明一下，以下讨论的流程仅支持全部署场景agent。

小广告



为了保证安全，笔者在前面的文章中也提到过，转发也需要一个权限要求。如果使用relayed 本地候选地址发送连接检查的话，如果以前没有创建转发权限的话，客户端必须首先需要创建一个转发权限，允许通过relayed本地候选地址进行转发。如果已经有已创建的转发权限的话，agent可以使用这个权限。如果需要创建转发权限的话，agent转发权限需要根据一定的流程来实现权限处理。具体的权限创建流程读者可以参与RFC5766-8和9章节。创建好的权限必须支持远端候选地址。一种情况需要特别处理，agent在正常情况下，发送CreatePermission请求进行连接检查创建时，RFC5245规范推荐agent要延迟TURN通道创建，直到ICE完成后再开始创建。一旦权限创建完成以后，agent必须维护权限的活动状态，一直到ICE结束此流程。

2 发送请求

在STUN客户端处理流程中，本地候选地址对远端候选地址发送绑定请求（Binding request），发送此请求后生成一个check。关于绑定请求的具体构建和生成，读者可以参与RFC8489。连接检查的安全策略必须使用STUN短期安全机制来实现。关于STUN短期安全机制（用户名称/密码+时间限定）和长期安全机制笔者在以前也做过介绍，读者也可以查阅RFC8489-9.1章节了解更多细节。注意，不能使用RFC3489实现向后兼容的支持能力。连接检查必须使用FINGERPRINT机制来实现STUN消息区分。关于FINGERPRINT机制的定义，读者可以查阅RFC8489-7。

ICE通过定义四个新属性拓展了对STUN支持，这四个属性分别是**PRIORITY，USE-CANDIDATE，ICE-CONTROLLED，和ICE-CONTROLLING**。在接下来的章节中笔者分别介绍这四个新定义的拓展属性。注意，这四个STUN拓展的新属性仅支持ICE的连接检查。

3 ICE定义的四个新STUN拓展说明

如果agent发送绑定请求时，它必须在其绑定请求中包含一个PRIORITY拓展属性。agent必须设定这个PRIORITY等于候选优先级排序中设定的那个权限设置。对于反射候选地址来说，这个优先级可以通过检查结果的学习来获得。除了反射候选地址中的偏好类型设置为此偏好设置以外，此优先级值计算和本地候选地址配对计算也是一样的。

在绑定请求中，被控方agent可以包含一个USE-CANDIDATE。但是，主控方一定不能在绑定请求中包含此拓展属性。此STUN拓展属性USE-CANDIDATE表示的意思是被控方agent希望退出此构件中的检查，使用来自于此构件中的候选配对执行检查。这里涉及了一个配对推荐的具体流程，笔

者在未来的文章中将具体讨论此指导原则和此STUN拓展的使用。

其余两个拓展属性根据角色不同选择包含不同的属性。具体来说，如果agent是在一个主控方角色中的话，在agent发送绑定请求时，它必须在请求中包含ICE-CONTROLLED拓展属性。如果是在被控方agent中的话，它必须在绑定请求中包含一个ICE-CONTROLLING拓展属性。当然，两种角色可能会因为会话不同其内容也可能不同，具体关于其角色决定的内容处理，笔者在前面的文章中有非常详细说明。

4 构建安全信息

除了agent发送绑定请求中需要各自包含所需要的拓展属性以外，agent同时也需要发送一定的安全信息来实现安全验证。绑定请求作为一种连接检查，它也必须使用STUN短期安全机制。STUN短期安全机制需要用户名称和密码。安全策略中的用户名称是本地agent的用户名称和远端peer的用户名称的合并名称（通过括号分开），密码是远端peer提供的密码。这里的策略设置比较迷惑，读者一定要注意。



连接检查用户安全构件组合

具体连接检查的处理方式如上图示例，根据连接检查的方向不同，用户名称和密码的选择组合是不同的。如果从L侧发起，L端agent需要对R端agent进行连接检查的话，它的用户名称组合方式是（RFRAG：LFRAG），密码是RPASS；反之亦然，如果从R端发起连接检查，其用户名称组合是(LFRAG：RFRAG)，密码是LPASS密码。返回响应时，响应处理使用的用户名称/密码和请求中的相同（USERNAME属性不会出现）。

5 区分服务处理

如果agent在媒体数据中（例如在QoS中）使用了区分服务-Diffserv（遵从RFC2475），agent也应该对连接检查使用同样的标识方式。因为区分服务不在我们讨论的范围，具体关于Diffserv，读者查阅RFC2475，这里不做进一步讨论。

6 处理响应

当收到响应以后，这个响应需要使用事务ID和其绑定请求关联在一起。这个绑定关联的处理流程在RFC5389或RFC8489中定义。然后agent把响应绑定到候选配对上，这个候选配对是绑定请求以前发送的候选配对。针对具体的STUN使用中关于失败响应和成功响应的不同场景，这个部分定义了额外的步骤来处理这些流程。下面，我们专门针对失败响应处理和成功响应处理做进一步的介绍。

7 失败响应场景和成功响应场景详解

失败响应有很多种。如果STUN事务生成一个487错误码，这表示agent角色冲突。agent需要检查在请求绑定中是否包含了前面所说的拓展属性ICE-CONTROLLED或ICE-CONTROLLING。如果请求包含的是ICE-CONTROLLED属性，agent还没有完成角色切换的话，它必须切换到主控方角色。如果请求包含的是ICE-CONTROLLING属性，agent还没有完成角色切换的话，它必须切换到被控方角色。一旦agent完成切换的话，它必须对生成487错误的候选配对进行入队处理，然后这个队列进入到triggered check queue中。然后将此配对状态设置为等待状态。当triggered check发送以后，它将包含一个ICE-CONTROLLED或ICE-CONTROLLING属性，这个属性反映它新的角色。这里注意，tie-breaker的值一定不能重选。

角色切换以后，需要进一步的计算。因为不同角色具有不同的功能，因此agent需要重新计算配对优先级（前面文章有介绍）。另外，角色切换还会影响agent所负责的其他功能，例如基于ICE结果选择推荐配对，和生成更新的offer消息。除了ICMP错误以外，因为环境不同，可能STUN事务还能生成其他的错误响应。Agent也可以支持针对连接检查的ICMP错误接收的工作。如果STUN事务生成ICMP错误，agent将会设置此配对为错误状态。如果STUN事务生成的错误是不可恢复的错误或者超时错误的话，agent也会设置此配对的状态为错误状

态。关于不可恢复的错误，读者可以查阅RFC5389-7.3.4章节关于错误的详解。

Agent必须检查源IP地址和响应端口是否等同于目的地地址和端口（目的地地址和端口是绑定请求发送过去的地址和端口），并且响应中的目的地地址和端口是否匹配源地址和端口（源地址和端口是从绑定请求中发送的）。换句话说，在请求和响应中的源地址和目的地传输地址是对称的。如果这两组地址不对称，agent将会设置此配对是失败状态。

以上笔者讨论的是检查失败响应的处理。现在，笔者讨论成功响应的处理。如果检查成功的话，检查成功，以下所有条件必须为真：

- STUN事务生成成功响应。
- 响应中的源IP地址和端口等同于目的地IP地址和端口（绑定请求中发送的）。
- 响应中的目的地IP地址和端口匹配源IP地址和端口（从绑定请求发送的）。

获得成功响应以后，agent需要��一步对响应消息和候选配对进行处理。其流程需要经过四个步骤的处理。笔者继续分别介绍这几个流程。

第一个步骤是发现peer的反射候选地址。Agent需要从响应中检查映射地址。如果传输地址不能匹配agent获知的任何本地候选地址，映射地址表示一个新候选地址，此地址就是peer反射候选地址。此反射候选地址和其他的候选地址一样，同样具有类型，基准地址，优先级和foundation。这四个属性的计算方式如下：

- 它的类型等同于peer的反射地址。
- 它的基准地址等同于候选配对中的本地候选地址，从地址来自于STUN检查发送地址。

- 它的优先级设置为绑定请求中的PRIORITY属性值。
- 它的foundation是通过foundation计算获得。

计算以后，peer反射候选地址（peer reflexive candidate）会添加到媒体流的本地候选地址列表中。对此媒体流来说，其用户名称和密码和其他本地候选地址的相同。但是，peer反射候选地址不会和其他远端候选地址配对。在此流程中，也没有必要进行反射地址和远端地址的配对处理。一对有效的配对会立刻通过构建有效配对的流程进行处理（此章节第二个步骤介绍）。如果agent希望peer反射候选地址和其他远端候选进行配对的话（这个远端候选地址不在将生成的有效配对内），agent可以生成一个更新的offer，在此offer中包含peer反射候选地址。通过这样的方式，agent可以完成此peer反射候选地址和其他远端候选地址配对的流程。

发现了反射候选地址以后，第二步的流程就是构建有效配对。agent可以开始构建一对候选配对。候选配对中，它的本地候选地址等同于响应中的映射地址，它的远端候选地址等同于绑定请求中发送的目的地地址。因为它们的有效性已经通过STUN连接检查验证，这样的配对称之为有效配对。读者需要注意，有效配对可能来自于不同的配对。具体来说，有效配对可能等同于检查生成的配对，可能等同于检查列表中不同的配对，或也可能是一对当前不在检查列表中的配对。如果这个配对是来自于检查流程生成的配对或者在当前检查列表的配对，它也可以被添加到有效列表中（VALID LIST）。agent为每个媒体流维护此列表状态。在ICE处理流程启动时，这个列表是为空状态，检查流程开始执行后在列表中逐渐增加配对，最后生成一个有效候选配对。

Agent经常也会遇到一些特殊状态-配对不在任何检查列表中。为什么会出现这样的情况呢？我们可以回顾一下这样的情景，检查列表有一对配对，其本地候选地址从来就不是一个反射候选地址。这种配对已经获得了它们的本地候选地址（这些地址已经转换成了服务器端反射候选地址的基准地址），如果这些配对重叠的话，需要对这些配对进行过滤筛选处理。当针对STUN检查的响应返回时，如果两个agent之间存在一个NAT时，映射地址将会是一个反射地址。这种情况下，有效配对将有一个本地候选地址，这个地址不能匹配检查列表中的配对的任何候选地址。

如果一对配对不在任何检查列表中的话，还要首先进行关于优先级的计算处理。Agent将会为此配对计算优先级。优先级计算基于每个候选地址优先级，其计算方式根据构建检查列表的流程来进行。本地候选地址的优先级基于其类型来决定。如果它**不是peer reflexive**，优先级等同于SDP中候选地址指示的优先级。如果**它是peer reflexive**，优先级等同于agent完成的绑定请求中的PRIORITY属性值。远端候选地址的优先级来自于对端peer的SDP中。如果没有出现远端候选地址，检查流程必须**启动一个triggered check**来生成一个远端候选地址。这种情况下，优先级来自于triggered check完成的绑定请求中的PRIORITY属性值。然后，这个有效配对被添加到有效列表中（VALID LIST）。

完成有效配对的构建，agent还要进行第三步处理流程。这个步骤就是更新有效配对的状态。Agent设置配对状态，这个状态是一个生成检查成功的流程。这里一定要注意，作为一种响应结果，这里的配对（生成检查流程）和构建有效配对是不同的处理方式（前面介绍过）。检查的成功可能引起其他检查的状态也发生改变。Agent必须按照以下两个步骤来执行：

1. 针对同样媒体流和同样foundation，agent修改所有其他封冻状态的配对到一个等待状态。通常来说，也不总是这

样，它们的其他配对将有不同的component IDs。

2. 对此媒体流的每个构件来说，如果在有效列表中有一对配对，这个检查的成功可能对其他媒体流关闭封冻检查。注意，按照这一步的流程执行的话，对每个媒体流的构件来说，不仅是第一次有效列表配对在这种情况下所需要考虑按照这些流程执行，每一个后续检查获得的检查成功结果获得的配对都要添加到有效列表中。agent按照顺序对其他媒体流查询检查列表：

   如果检查列表是在活动状态，agent修改检查列表中所有封冻状态的配对（它们的foundation匹配了有效列表中等待状态的配对的foundation值）。

   如果检查列表是封冻状态，并**且在检查列表中至少有一对配对**，它的foundation匹配了有效列表的配对的foundation，在检查列表中所有配对中，如果其foundation匹配了有效列表中的一对配对的foundation的话，所有列表中配对的状态会设置为等待状态。这样的处理结果会导致检查列表变为活动状态，ordinary checks开始为其工作。具体细节查阅笔者历史文档中的定时检查设置。

   如果检查列表是封冻状态，并且检查列表中没有配对，它的foundation匹配有效列表中的配对的foundation的话，agent将执行以下处理流程，agent将会对所有的配对分组设置，所有配对有同样的foundation，并且，针对每个组设置，具有最低component ID的则设置其配对状态为等待状态，如果有一个以上这样的配对的话，则启用最高优先级的配对。

Agent完成了更新配对状态以后，最后agent将会执行第四步进行**更新推荐配对**处理。如果agent是一个被控方agent，它已经在绑定请求中包含了USE- CANDIDATE属性的话，从check生成的有效配对已经有一个推荐配对设置flag，这个设置为true状态。如果此配对的优先级在所有标识的配对中具有最高的优先级，这个标识则表示此媒体流或所有媒体流将使用这一对有效配对。如果agent是一个主控方agent，这个响应可能是triggered check的结果，这个triggered check在响应中返回到请求，此请求自己包含一个USE-CANDIDATE属性。这样的情况就是一个更新推荐配对示例，它可能导致对这个配对(这个配对是从原始请求学习获得)进行推荐flag设置。

以上介绍了失败响应和成功响应的处理。在处理响应的流程中，除了对失败响应和成功响应进行处理以外，还要对检查列表和定时器更新进行处理。无论检查是否是成功还是失败，最后事务完成需要**更新检查列表和定时器的状态**。如果所有在检查列表中的配对是失败或者成功的状态：

- 针对每个媒体构件来说，在有效列表中没有一对配对，检查列表的这个状态设置为**失败状态**。
- 对每个封冻的检查列表，agent以同样的foundation对所有配对进行分组
- 并且针对每个组，把带**最低component ID的**配对的状态设置为等待状态。如果有超过一个以上这样的配对，则使用具有最高优先级的配对。

如果在检查列表中没有任何配对在封冻或者等待状态的话，这个检查列表则不再认为是活动的检查列表，并且针对ordinary checks，检查列表不会在定时器计算中计入N的值。这里N是指活动检查列表数量。具体就是流程，读者可以查阅笔者上一篇历史文档关于设定定时检查的讨论。

本章节重点介绍了关于ICE连接检查中的STUN客户端的处理流程。为了避免让读者引起歧义，方便读者阅读，笔者将STUN服务器端处理流程独立分为另外一篇文章发布，关于STUN服务器端的处理流程将在下一篇文章中加以介绍。STUN服务器端主要流程包括两个场景的处理流程：首先是关于全场景部署agent的其他额外步骤处理（检测和修复角色冲突，计算映射地址，通过学习获得peer反射候选地址，Triggered Checks讨论和更新推荐配对标识），然后是关于轻量级部署agent的其他额外步骤处理。

参考资料：

https://www.rfc-editor.org/rfc/rfc5389
https://www.rfc-editor.org/rfc/rfc8489
https://www.rfc-editor.org/rfc/rfc5766
https://ir.nctu.edu.tw/bitstream/11536/43505/1/655201.pdf
https://www.cisco.com/c/en/us/td/docs/solutions/PA/ICE/icepa125.html
https://dev.w3.org/2011/webrtc/editor/archives/20130830/webrtc.html



关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC技术文档: www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文

在看

完整SIP/SDP媒体协商概论-后续offer处理-answer生成

原创 james.zhu Asterisk开源派

在上一篇文章中，笔者介绍了后续offer中关于offer生成的讨论。今天，笔者将继续介绍peer接收offer和生成answer响应的细节，主要涵盖细节包括：

1. 整体部署场景讨论（检测ICE重新启动，移除媒体流，增加媒体流）
2. 全场景部署讨论：无remote-candidates属性状态下，ICE运行时现存媒体流处理和ICE完成后现存媒体流处理，带remote-candidates现存媒体流处理。
3. 轻量级场景部署讨论：ICE运行时现存媒体流处理和ICE完成后现存媒体流处理。

读者注意，在这里讨论的offer/answer已经涉及到了后续offer的内容，也就是更新的offer，所以有时会和初始offer时的一些属性进行对比处理，读者一定不要迷惑。

1 整体部署场景讨论

当agent在现存会话中收到后续offer时，无论前面的offer/answer交互验证是否成功，agent必须重新检查一次验证ICE支持能力（前面的文章中介绍过接收初始offer关于ICE能力支持验证流程）。虽然，前面offer/answer交互可能导致ICE不能使用，不过，这种处理结果可以作为后续offer/answer交互的结果使用。和上一篇文章中所讨论的应用，agent接收offer时，对于整体部署场景来说，仍然需要执行常规的三个步骤的处理：检测ICE重新启动，添加媒体流处理，移除媒体流处理。现在，我们分别加以介绍。

小广告



用户属性发生了修改，agent需要重新检测并且重新启动ICE。具体来说，agent和前面收到的SDP中属性对比，如果agent在收到的更新offer消息中包含了一个已修改的属性（可能是a=ice-ufrag 或者a=ice-pwd），针对此媒体流，agent需要重新启动ICE。如果所有的媒体流需要重新启动的话，所有ICE也需要重新启动。如果一个媒体流的ICE重新启动的话，agent会在answer中执行两种修改处理：

- Agent必须修改answer中的a=ice-ufrag和a=ice-pwd。
- Agent可能在answer消息中修改其部署层级。

针对媒体流，agent会修改SDP中其他属性设置，属性设置和和初始answer中的处理流程相同。针对此媒体流，在接下来的处理流程中，候选地址组会根据实际情况，可能会包括部分参

数，无参数，或者包括前面的候选地址参数，并且可能包括全新采集的候选地址组。

如果agent收到一个offer，offer中包含了一个新的媒体流，针对此媒体流，就像agent在初始offer包含的媒体流一样，agent必须在answer中设置相关的域值。在answer设置以后就会导致ICE重新启动。

如果agent收到一个offer，offer中包含了一个媒体流，其端口设置为0的话，agent**一定不能**为此媒体流在answer中包含任何候选地址属性，并且**不应该**包含任何和ICE定义的相关属性，例如foundation和component-id等。

2 全场景部署环境处理流程

这个章节将对在全场景部署环境中，在不同ICE运行状态下对现存媒体流进行的处理做详细说明。这里要注意这三个参数环境，用户名称（ice- ufrag），密码（ ice-pwd）和部署级别。除了agent已经从offer消息中检测到ICE重新启动之外，用户名称（ice- ufrag），密码（ ice-pwd）和部署级别必须维持不变。如果agent想修改其中一个属性值的话，agent通过生成一个新的offer来重新启动ICE。agent不能在answer消息重新启动ICE处理流程。其他的流程取决于此媒体流的ICE运行状态。笔者将结合不同remote-candidates属性出现的以下三种状态进行讨论。

如果针对一个媒体流的ICE的状态**处于运行状态**，并且此媒体流的offer已锁定了remote-candidates属性，构建answer消息的规则和笔者上一篇关于offer生成的流程相同。

如果针对一个媒体流的ICE状态**处于完成状态**，并且此媒体流的offer已锁定了remote-candidates属性，构建answer消息的规则和笔者上一篇关于offer生成的流程相同，**另外**，应答方

answerer 一定不能在answer消息中包括a=remote-candidates属性。

以上两种状态都是锁定了remote-candidates属性的。如果remote-candidates是一种可变状态，出现了竞争条件的话，现存媒体流的处理就需要做另外处理。当agent对端peer对一个媒体流已包含了ICE处理流程的话，主控方agent将会收到一个带a=remote-candidates属性的offer消息。出现在offer中的这个属性用来处理介于offer接收和响应接收之间的竞争条件，通知answerer接收方ICE将会选择一个候选地址。关于竞争状态的流程，读者可以查阅RFC5246-appendix-B.6。接下来关于针对offer携带a=remote-candidates的处理流程完全取决于竞争条件中协商胜方的处理流程。

Agent为媒体流的每个构件生成一个候选配对，它需要远端候选地址和本地候选地址，具体处理流程为：

- 设置远端候选地址等于offerer提供方中的默认目的地地址。例如，针对RTP的m行和c行内容，和RTCP的a=rtcp。
- 设置本地候选地址等于传输地址，此传输地址是支持在offer中a=remote-candidates同一构件。

生成候选配对完成以后，agent看到候选配对会出现在有效列表中。如果有一个配对没有出现在有效列表中的话，说明检查流程失去了竞争条件。这样的配对称之为 "losing pair"（丢失的配对）。接下来，agent会在检查列表中通过远端候选地址查找，查找所有远端候选地址等于丢失配对中的远端候选地址的配对，执行以下流程：

- 如果这些配对中无配对在In-Progress处理状态，并且至少一个配对是失败状态，这可能是因为网络问题导致，例如可能发生了网络隔离问题，严重的网络丢包。这样可能就没有出现remote-candidates，agent应该为此媒体流生成一个answer，然后为此媒体流重新启动ICE流程。

- 如果这些配对中至少有一对配对在In-Progress处理状态，agent应该等待它们的检查流程完成，并且当每个检查完成后，重新执行这部分流程，直到再没有丢失配对需要处理。

一旦完成了没有再需要处理的丢失的配对以后，agent就可以生成一个answer消息。它必须为此媒体设置默认目的地地址，默认目的地地址为remote-candidates（来自于offer）中的候选地址。它必须为此每个候选地址在answer中包括一个候选地址属性，这里的每个候选地址是在offer中remote-candidates属性中。

3 轻量级部署环境处理流程

如果在收到的offer中包含remote-candidates属性，agent为媒体流的每个构件生成一个候选配对，它需要远端候选地址和本地候选地址，具体处理流程为：

- 设置远端候选地址等于offerer提供方中的默认目的地地址。例如，针对RTP的m行和c行内容，和RTCP的a=rtcp。
- 设置本地候选地址等于传输地址，此传输地址是支持在offer中a=remote-candidates同一构件。

针对此媒体流，agent然后把这些候选地址迁移到有效列表中。ICE处理流程状态设置为完成状态。

除了以上设置以外，agent控制角色也是一个问题需要讨论。如果agent认为它自己是主控方agent，而且在offer消息中包含了remote-candidates属性，双方agent都认为自己是主控方agent。这种情况下，双方agent将可能都已经同时对对方发送了更新offer消息。这种极端情况需要负责传输offer/answer交互的信令协议来解决。最后，其中一方agent总是以竞争环境中的胜方出现。在信令协议传输offer/answer交互的流程中，对端peer发送offer之前，胜方agent会首先收到一个

offer消息。胜方agent会充当主控方agent的角色，因此，这里所讨论的应答方answerer必须修改其角色为主控方角色。接下来，如果agent基于一个流程（根据RFC5245-8.2.2）发送更新offer的话，这个agent就是一个主控方agent。

除了以上控制角色修改以外，构建answer的执行流程中关于有效列表中的修改和ICE状态修改和构建offer流程的相同。读者可以查阅上一篇文章做进一步了解。

本章节介绍了在更新的offer中关于接收offer消息和生成answer消息的处理过程。接下来的章节笔者将介绍在后续offer/answer交互中check更新和有效列表更新的细节。

阅读原文

在看

完整SIP/SDP媒体协商概论-后续offer处理-Offer

原创 james.zhu Asterisk开源派

根据RFC3264的规范，agent的任何一方都可以生成后续offer。在笔者前面的文章中，笔者讨论了关于ICE结束处理的流程的规则，这个结束ICE流程的规则会导致被控方agent发送一个更新offer。具体来说，当ICE从默认配对中已选择了不同的配对时，在ICE结束流程中，被控方agent会发送一个更新offer消息。但是，前面的文章中没有详细介绍agent如何实现更新offer，接收offer，以及最后更新检查连接和有效列表的步骤。如果agent要实现更新offer的处理流程，双方agent需要经过三个主要的流程。Agent需要首先要经过offer消息构建，然后对端peer接收更新的offer，生成更新的answer消息，最后更新连接检查和有效列表。

因为更新offer的内容比较庞杂，为了让读者有一个比较好的阅读体验，笔者计划将上述内容分为三个部分来介绍。本文章介绍第一个部分的内容（生成后续offer），在以后的文章中分别介绍第二部分和第三部分的内容。

本文章主要涵盖关于生成后续offer的内容：

1. 整体部署场景讨论（ICE 重新启动，移除媒体流，增加媒体流）
2. 全场景部署讨论：ICE运行时现存媒体流处理和ICE完成后现存媒体流处理。
3. 轻量级场景部署讨论：ICE运行时现存媒体流处理和ICE完成后现存媒体流处理。

在讨论后续offer/answer交互模式时，首先需要关注的是关于如何生成更新的offer消息。在生成更新offer消息中，RFC5245分别规定了两种常见的处理流程（全场景部署场景和轻量级部署场景）。接下来，笔者会首先对整体部署场景进行讨论，然后分别对两种部署场景中关于不同ICE运行状态环境中现存媒体流的处理进行讨论。

注意：关于ICE重新启动的细节，RFC5245和RFC8445规范有非常某些区别，笔者这里以RFC5245的规范作为讨论基础。如果对最新的规范有兴趣的话，可以查阅RFC8445的细节。

1 整体部署场景讨论

在讨论后续offer/answer交互模式时，首先需要关注的是关于如何生成更新的offer消息。在生成更新offer消息中，针对agent，RFC5245分别规定了两种常见的处理流程（全场景部署场景和轻量级部署场景）。在介绍这两种部署场景的处理流程之前，我们首先介绍三个和整体部署场景相关的内容。第一个是关于重新启动ICE流程的讨论。

小广告



Agent可以对现存的媒体流重新启动ICE处理流程。重新启动ICE处理流程会导致前面的ICE处理流程被刷新，并且还要重新启动检查流程。注意，重新启动ICE处理流程和重新创建一个新会话之间有不同之处，在重新启动ICE处理流程的过程中，媒体流仍然会被发送到以前的有效配对目的地。对于一个媒体流来

说，如果有以下两种情况可能发生的话，agent必须重新启动ICE流程：

- 已经生成了offer，此offer的目的是为了修改媒体流目的地。简单来说，就是agent想生成一个更新的offer消息（ICE没有使用这个offer），这样的结果是为媒体构件目的地生成一个新的值。
- Agent正在修改其部署级别。正在情况一般发生在第三方呼叫控制的环境中。对象实体正在使用的信令不是实体接收媒体的信令，并且这个实体已经从媒体 mid-session的目的地修改为另外一个实体，这个实体具有不同的ICE部署环境。

另外一些规则的修改也会引起ICE重新启动，SDP中的c行中IP地址修改为0.0.0.0会引起ICE重新启动。因此，如果需要支持呼叫等待业务功能的话，ICE部署中则不能使用以上机制，它必须使用a=inactive和a=sendonly设置。因为这里涉及了IPv6的网络场景，旧规范RFC3264已经做了更新，关于此要求最新细节，建议读者可以参考RFC6157-4.1章节的介绍。如果agent要为媒体流重新启动ICE流程的话，agent必须在offer消息中修改认证用户需要的两个属性ice-pwd和ice- ufrag。注意，RFC5245规范也允许agent在offer中使用会话级的属性设置，仅为在后续的offer中作为一个媒体级属性提供同样的ice-pwd或ice- ufrag。这样的操作不会修改密码，不会修改其呈现方式，也不会引起ICE重新启动。Agent在SDP中设置了多个其他的属性，这些属性在包含在初始化的offer中。针对此媒体流，在接下来的更新offer的处理流程中，候选地址组会根据实际情况，可能会包括其中一些参数或者没有包含任何参数，或者前面的候选地址，并且可能包括全新采集的候选地址组。候选地址采集的采集通过采集候选地址来执行（关于候选地址场景请参考历史文章）。

根据RFC3264的规范规定，如果agent需要移除媒体的话，它可以设置其端口为0。这个规则在这里也适用。除了agent设置端口为0以外，针对要移除的媒体流，它**一定不能包含**任何候选地址属性，并且**不应该**包括任何ICE相关的属性设置。具体的ICE相关属性，参考如下示例：

```
candidate-attribute   = "candidate" ":" foundation SP component-id SP
                        transport SP
                        priority SP
                        connection-address SP     ;from RFC 4566
                        port         ;port from RFC 4566
                        SP cand-type
                        [SP rel-addr]
                        [SP rel-port]
                        *(SP extension-att-name SP
                             extension-att-value)

foundation            = 1*32ice-char
component-id          = 1*5DIGIT
transport             = "UDP" / transport-extension
transport-extension   = token              ; from RFC 3261
priority              = 1*10DIGIT
cand-type             = "typ" SP candidate-types
candidate-types       = "host" / "srflx" / "prflx" / "relay" / token
rel-addr              = "raddr" SP connection-address
rel-port              = "rport" SP port
extension-att-name    = byte-string    ;from RFC 4566
extension-att-value   = byte-string
ice-char              = ALPHA / DIGIT / "+" / "/"
```

如果agent想增加一个新的媒体流的话，它需要在SDP中为此媒体流设置相关的域值。这些SDP中的相关域值设置需要参考初始化offer中关于SDP解码中的设置。Agent修改了这些媒体流的设置后，ICE处理流程就会开始发送媒体流。

以上内容介绍了整体部署环境中关于ICE重新启动，移除媒体流和删除媒体流的处理流程。接下来，我们将首先针对全场景部署环境中处理流程进行说明。

2 全场景部署环境处理流程

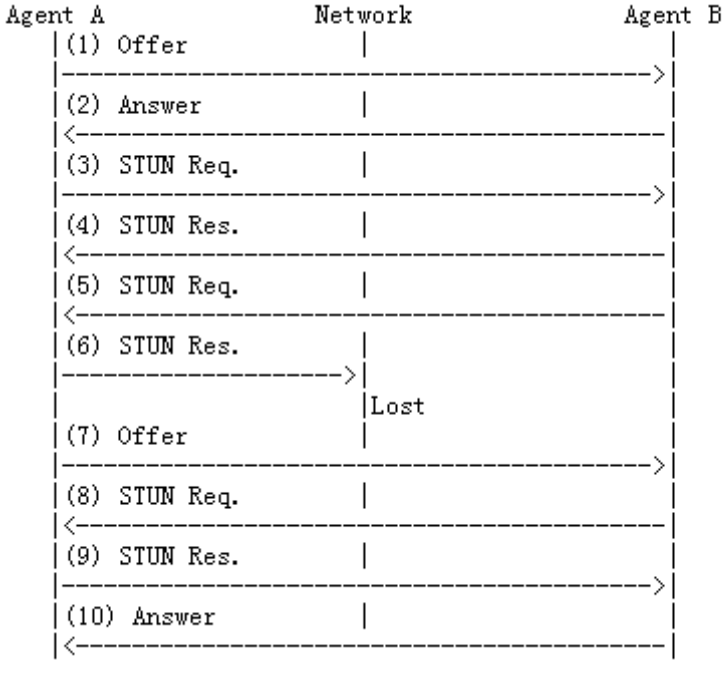
这个章节将对在全场景部署环境中，在不同ICE运行状态下对现存媒体流进行的处理做详细说明。这里要注意这三个参数环境，用户名称（ice- ufrag），密码（ ice-pwd）和部署级别，前面使用的这三个条件没有发生改变。如果agent需要修改其中一个参数条件，ICE就需要重新启动。

如果agent生成了一个更新的offer，在更新offer中包括了一个以前已创建的媒体流，并且ICE检查也在运行状态时，agent需

要根据以下流程执行。针对此媒体流，此agent必须为所有本地候选地址包括候选属性。在SDP标识的候选地址的属性，例如，priority，foundation，type和相关传输地址都**应该保持不变**。候选地址中基础的确认信息，例如，IP 地址，port和传输协议一定**不能发生变化**（如果其中一个发生变化，则说明是一个新的候选地址）。component ID和以前的component ID保持一致。Agent**可以**包括以前offer中没有生成的其他候选地址，但是，自从上一次offer/answer交互以后，对agent已经采集的候选地址来说，需要包括一个peer reflexive candidates。针对媒体来说，就像在初始offer中的一样，肯定有一组候选属性可以匹配默认目的地地址，agent可以为此媒体修改默认目的地地址。

以上是ICE在运行状态时，agent更新offer需要执行的流程。除了ICE处于正在执行状态以外，ICE检查也可能是完成状态。如果agent生成了一个更新的offer，在更新offer中包括了一个以前已创建的媒体流，并且ICE检查在完成状态时，agent需要根据以下流程执行。针对媒体的默认目的地地址（例如，媒体流的m行和c行中的IP地址和端口值）必须是本地候选地址，这个本地候选地址来自于对每个媒体构件的有效列表最高优先级推荐配对。通过这样的处理方式可以“修复”目的地地址，让默认媒体的目的地地址等于ICE为此媒体的已选地址。Agent必须包含一些候选地址的候选属性，这些候选属性匹配了媒体流中每个构件的默认目的地地址，并且agent一定不能包含其他候选地址。另外，如果agent是一个被控方agent的话，它必须为每个媒体流（这些媒体流的检查列表是在ICE完成状态）包含a=remote-candidates属性。这个属性设置中包含远端候选地址，这些地址来自于此媒体流的每个构件所支持的有效列表中的最高优先级配对取值。这里可能存在一种极端情况，这种情况需要尽量避免。被控方agent选择了它的配对，但是更新offer对主控方agent执行了一个序乱的连接检查。这种情况下，对端agent可能不知道哪个配对是有效配对。因此，在更

新 offer 使用 a=remote-candidates 属性避免更新 offer 和 STUN检查协议之间因为请求/响应消息丢失出现的这种极端情况。注意，这种极端情况的处理机制讨论仅在RFC5245的规范中有规定，但是在RFC8445中取消了这种处理来自。相对于RFC5245，RFC8445针对ICE重新启动做了大幅简化处理。



a=remote-candidates 避免配对序乱处理(RFC5245)

3 轻量级部署环境处理流程

这个部分的讨论和前面的讨论应用，笔者也会分别针对两种ICE运行状态下的轻量级部署环境的处理流程进行介绍。

首先说明一下轻量级部署中针对现存媒体流，ICE流程处于运行状态时的处理流程。轻量级部署必须在任何后续的offer中的a=candidate为每个媒体流的每个构件的包含所有的候选地址。候选地址的构建和初始化offer中关于候选地址的构建的流程完全相同。具体的构建流程在前面的历史文章中有专门介绍，读者可以查阅历史文档。轻量级部署中**一定不能**在后续offer中再增加其他的主机候选地址。如果agent想增加一个新的主机候选地址的话，agent必须**重新启动ICE处理流程**。在后续offer中的用户名称（ice- ufrag），密码（ ice-pwd）和部署级别应该和前面offer中的属性保持一致，如果以上三种参数后续offer中发生修改，agent则必须为此媒体流重新启动ICE处理流程。

针对媒体流来说，如果ICE处理状态处于完成状态，此媒体流的默认目的地地址必须设置为此媒体流构件的候选配对（在有效列表中）的远端候选地址。对轻量级部署来说，总是支持一个在有效列表中的单候选配对。另外，agent必须为每个默认目的地包含一个候选地址属性。

如果agent是一个被控方agent的话（仅发生在双方都是轻量级部署的agent），此agent必须为每个媒体流包含一个a=remote-candidates。a=remote-candidates中包含远端候选地址，这个远端候选地址来自于有效列表中的候选配对（每个媒体流的每个构件有一对候选配对）。

在接下来的章节中，笔者将继续介绍针对更新offer中对端agent接收offer和生成answer的处理流程。

参考资料：
https://www.rfc-editor.org/rfc/rfc3264
https://www.rfc-editor.org/rfc/rfc6336
https://www.rfc-editor.org/rfc/rfc8445



关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

最新Asterisk完整中文用户手册详解及免费slack支持：www.asterisk.org.cn

Freepbx/FreeSBC技术文档: www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文 在看

完整SIP/SDP媒体协商概论-关于ICE流程结束处理

原创 james.zhu Asterisk开源派

笔者在前面的两篇文章中分别介绍了STUN客户端处理流程和服务器端处理流程。在本文章中，我们将针对ICE的最后一部分的处理进行总结，这个章节包括ICE的流程结束的处理。ICE流程结束需要从两种部署场景的agent来讨论。一种是全场景部署agent的处理流程，另外一种是轻量级的部署场景agent处理流程。另外，ICE结束流程的最后还要进行对封冻候选地址的处理。和前面的文章一样，笔者这里仍然还是重点讨论全场景部署agent的处理流程，然后针对轻量级场景中agent的部署进行讨论，最后对ICE结束对封冻候选地址进行讨论。

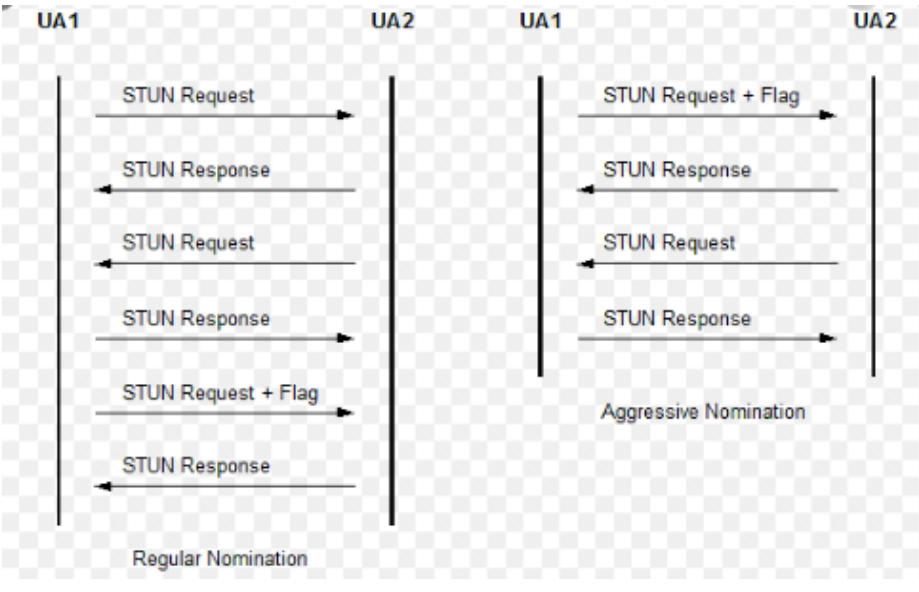
1 全部署场景处理流程

针对全场景部署agent的处理流程中，关于ICE结束的流程涉及了推荐配对和状态机更新两个部分的内容。其中，推荐配对是由被控方agent产生。接下来，笔者会继续讨论推荐配对的处理流程。

2 推荐配对

刚才笔者已经提到，推荐配对是由被控方产生的。ICE通过Regular Nomination（正常推荐方式）或Aggressive Nomination（主动推荐方式）来选择推荐配对。选择何种方式对推荐配对进行算法处理，取决于对端pper的部署场景。如果对端peer支持的是一个轻量级部署场景，agent必须使用Regular Nomination算法来处理。如果对端peer使用了ice-options属性（ICE options），agent不能理解此选项的话，agent必须使用Regular Nomination算法。如果对端peer支持全场景部署，并且没有使用任何ICE选项或者使用了ICE选项

（但是，agent可以理解），agent可以选择使用Regular Nomination或者Aggressive Nomination算法。但是，因为Regular Nomination的稳定性比Aggressive Nomination要好，所以，RFC5245规范推荐使用Regular Nomination算法。下面，笔者分别对这两种算法进行完整介绍。



图片来自于互联网

当推荐配对使用Regular Nomination时，agent会让一些检查流程完成处理，在检查的每个流程中会忽略掉USE-CANDIDATE属性。针对一个媒体流构件来说，一旦一个或多个检查成功完成，成功检查完成后会生成有效配对，有效配对然后添加到有效列表中。Agent会让检查继续执行，直到检查遇到某些评判标准，然后根据某些评判标准选择有效配对。停止检查的评判标准和评估有效配对标准完全取决于逻辑优化本身自己。

当被控方agent选择了一对有效配对时，它会重复这个生成有效配对的检查，并且这次使用USE-CANDIDATE属性。因为前面的检查是成功的，所以，理论上讲，这个检查应该也是一个成功的检查。针对检查的逻辑处理方式会对配对增加一个推荐标识（nominated flag），并且仅支持这个配对。因此，针对每个媒体构件来说，在有效列表中仅有一个单个支持了推荐标识配对，并且，当检查列表的状态切换到完成状态后，ICE会选

择正确配对来针对媒体构件发送和接收媒体流。根据以上介绍，读者可以看出，因为agent可以控制检查停止和检查的评判标准，因此，Regular Nomination具有更大的灵活性。这里只有一个要求，agent最后选择一个配对或者只能选择一个候选配对，然后针对此配对（支持USE-CANDIDATE属性）生成一个检查。通过增加拓展支持，Regular Nomination同时提高了ICE的适应能力，可以支持多种部署场景的变化。Regular Nomination具有更高的稳定性，它可以允许双方agent通过单个配对实现媒体支持，无需其他临时选项支持（Aggressive Nomination需要临时选项支持）。但是，Regular Nomination也有其缺点，因为Regular Nomination需要额外检查完成，因此，Regular Nomination肯定会增加处理时延。通过以上内容的介绍，笔者描述了Regular Nomination算法的处理方式，接下来，笔者介绍一下Aggressive Nomination算法的使用方式。

小广告



当使用Aggressive Nomination算法时，在agent发生的每个检查中，被控方agent会包含一个USE-CANDIDATE属性，针对一个媒体构件的检查中，一旦第一个检查是成功的，这个配对就会被添加到有效列表中，然后设置一个推荐标识。在有效列表中，所有构件的配对都设置了推荐标识后，媒体开始通过最高优先级的推荐标识配对进行传输。但是，因为agent在所有的检查中都包含了USE-CANDIDATE属性，在所有的检查中，有可能部分其他的检查还没有完成，这样就会引起其他有效配对会产生自己的推荐标识设置。因为，ICE总是从有效列表中选择最高优先级推荐标识的候选配对来进行媒体传输。因此，当ICE检查完成时，已选择的配对可能实际上暂时发生了修

改，这样就会导致一系列的临时选择作为一个缓冲（三秒延迟规则，后面讲到），直到ICE检查稳定后，这样的状态才能结束。因为这个问题，因此，相对于Regular Nomination算法来说，Aggressive Nomination缺乏一定的稳定性。但是它不会增加检查延时。

3 更新ICE处理状态

无论是对于主控方agent还是被控方agent来说，ICE的处理状态取决于在有效列表中标识候选配对的当前状态和检查列表的状态。任何时间内，以下五种场景可能发生在这些处理状态中。现在，我们具体介绍一下这五种可能发生的场景。

- 对媒体流来说，如果在有效列表中**没有**已设推荐标识的配对，并且检查列表状态是正在运行中，ICE处理将会继续执行。

- 对媒体流来说，如果在有效列表中至少**有一对**已设推荐标识的配对，并且检查列表状态是正在运行中，则继续进行以下处理流程：
  Agent**必须**移除在检查列表中所有等待状态和封冻状态的配对，并且为同样component（构件）启动一个triggered check queue，作为标识配支持此媒体流。
  如果在检查列表中的一个In-Progress配对作为标识配对支持同样构件的话，如果配对的优先级**低于最低优先级**标识配对，针对此构件来说，agent应该清除检查的重传处理流程。

- 对于至少一个媒体流的每个构件来说，在有效列表中一旦至少有一对标识配对，并且检查列表的状态是正在运行中的状态，需要进一步执行以下流程：
  Agent**必须**为此媒体的检查列表进行状态修改，修改其状态为完成状态。

Agent**必须**继续对任何检查做出响应，这些检查可能仍然是agent用来接收媒体的响应，并且，如果STUN 服务器端流程要求的话，agent必须执行triggered check。
Agent**必须**为检查列表继续重传任何In-Progress 检查。
Agent**可以**开始为媒体流传输媒体，为全场场景部署agent和轻量级场景部署agent发送媒体的处理流程将在后续文章中介绍。

- 一旦每个检查列表的状态是完成状态，要求执行以下流程：
  Agent设置所有的ICE处理状态是完成状态。
  如果agent是正在处于被控状态，此agent会为每个媒体流的每个构件检查最高优先级已标识的候选配对。如果它们中的任何候选配对不同于在最近offer/answer交互消息在的默认候选配对，被控方agent必须生成一个更新的offer。具体的生成方式根据后续offer/answer交互模式的处理流程进行。

  如果被控方agent正在使用Aggressive Nomination，当配对选择时，它可能导致多个更新offers。Agent可以延迟发送此更新的offer，通过一定的时间设置进行控制（RFC5245规范建议设置为一秒钟），这个时间可以保证其已选配对进入稳定状态。
- 如果检查列表的状态是失败状态，ICE将不能为完成针对媒体流的处理。正确的处理方式依赖于对其他媒体流的检查列表状态：
  如果所有的检查列表的状态是失败状态，所有ICE处理的状态将认为是失败状态，并且，agent应该认为此会话是一个失败的会话，agent不应该重新启动ICE处理流程，被控方agent应该结束整个会话。
  针对其他媒体流来说，如果在检查列表中至少有一个检查的状态是完成状态，被控方agent应该在其更新的offer中

从此会话中移除此失败的媒体流。

针对其他媒体流来说，如果在检查列表中没有任何检查是完成状态，但是，至少有一个检查是正在运行状态，agent应该让ICE进行执行。

到此为止，关于ICE结束处理中全场景部署agent的处理流程就已经结束。接下来，笔者将讨论针对轻量级部署场景agent对ICE结束流程的处理。

4 轻量级部署场景处理流程

针对轻量级部署场景agent来说，ICE结束流程相对比较直接。这里有两个情况需要注意：

1. 部署场景是轻量级的部署场景，对端peer是全场景部署场景
2. 部署场景是轻量级部署场景，对端peer也是轻量级部署场景

ICE结束处理会给agent带来一个后续影响，agent能够清除任何已分配的候选地址（ICE没有使用这些候选地址）。关于清除已分配候选地址的处理方式，笔者在本文章的后续部分介绍。

如果对端peer是全部署场景的话，这种情况下，agent将会收到peer的连接检查。当agent已经收到来一个连接检查，这个检查中包含了针对一个媒体流的每个构件所支持的USE-CANDIDATE属性，此媒体流的ICE处理状态将要从正在运行状态迁移到完成状态。当针对所有媒体流的ICE处理状态是完成状态的话，所有ICE处理状态都是完成状态。轻量级部署从来不自己决定针对媒体流的ICE流程失败处理，而是宁愿让全场景部署的agent来决定。在后续的offer中，针对失败的媒体流，全场景部署将做出决定，移除或重新启动这些失败的媒体流。

如果对端peer是轻量级peer的话，ICE结束处理的流程相对比较复杂一些。一旦offer/answer交互模式完成后，双方agent

都会检查它们的候选地址和它的peer的候选地址。针对每个媒体流，每个agent将会使用自己候选地址和对端peer的候选地址进行配对处理。当它们具有同样的component，使用同样的传输协议（RFC5245使用UDP传输协议），并且来自于同一IP地址类型（IPv4或IPV6），那么这两个候选地址就会配对。在生成配对后，针对每个媒体流构件中的配对数量有两种处理方式：

- 如果在每个媒体构件中，有一个单个配对的话，此配对会添加到有效列表中。如果一个媒体的所有构件只有一个配对的话，针对此媒体流的ICE处理状态将会设置为完成状态。如果所有媒体流的ICE处理状态是完成状态的话，所有ICE处理状态将会设置为完成状态。这种部署环境经常会发生在IPv4的网络环境中。
- 如果在每个构件中，有多于一个以上的配对的话，需要执行以下几个步骤：

  Agent必须基于本地策略选择一个配对。因为，这种情况仅发生在IPv6的环境中，RFC5245推荐agent需要根据RFC6724的处理流程选择一个单个配对。

  Agent会在有效列表���为每个构件添加此已选配对。然后允许开始发送媒体。这里要注意，但是在实际环境中，可能双方agent已选择了不同的配对。

  为了保持双方agent的配对的一致性，被控方agent必须发送一个更新的offer，在这个更新的offer中包含一个remote-candidates属性设置。关于更新offer的发送处理流程，笔者在将来的讨论中介绍。

  当offer发送以后，agent一定不能更新ICE处理状态。针对所有媒体流，如果此后续offer完成处理，主控方agent必须修改ICE处理流程状态为完成状态，并且设置所有ICE处理状态为完成状态。主控方agent的状态设置则基于轻量级部署场景的流程来进行处理。

5 封冻候选地址

在ICE结束处理的流程中，包括两种对后选地址的封冻处理。一种是全场景部署中封冻处理，另外一种是轻量级的部署场景封冻处理。接下来，笔者分别对其流程进行说明。

首先介绍全场景中关于封冻处理的流程。ICE结束处理的流程要求agent继续监听STUN请求，一旦对其媒体流的处理完成后，继续对媒体流生成triggered checks。RFC5245介绍了一种规则，规则描述了当agent处于安全状态时，它在一个候选地址（ICE没有选择此候选地址，然后释放候选地址）终止发送或接收检查。

当SIP网络中使用了ICE，并且offer被经过分叉处理发送到多个接收方时，ICE将会使用同样的本地候选地址并行和每个自己的answer进行处理。对所有使用那些候选地址来传输媒体流的peers来说，一旦ICE处理流程状态达到完成状态，agent应该等待**另外三秒钟**，然后agent可以终止对检查的响应，或在那个候选地址生成一个triggered checks。Agent可以在那个等待时间释放候选地址。注意，服务器端反射候选地址的封冻处理从来不会被明确声明，keepalive的缺失会启动封冻处理流程发生。三秒延迟的规则策略可以处理如下场景，具体来说，ICE已经完成后，agent使用了aggressive nomination算法，然后对已选配对进行快速修改。

轻量级部署场景中ICE结束处理相对比较简单。对所有使用那些候选地址来传输媒体流的peers来说，一旦ICE处理流程状态达到完成状态，轻量级部署场景可以释放任何没有被ICE选择的候选地址。

完成了关于ICE的讨论以后，笔者将进一步讨论关于后续offer/answer交互中的offer生成，answer接收，更新offer等细节。

参考资料：
https://www.rfc-editor.org/rfc/rfc6724
https://www.rfc-editor.org/rfc/rfc3484
https://www.rfc-editor.org/rfc/rfc5245.html
https://datatracker.ietf.org/meeting/93/materials/slides-93-mmusic-3
https://bugzilla.mozilla.org/show_bug.cgi?id=1034964



关注微信公众号：asterisk-cn，获得有价值的Asterisk行业分享

Asterisk freepbx FreeSBC技术文档: www.freepbx.org.cn

融合通信/IPPBX商业解决方案：www.hiastar.com

如何使用FreeSBC，qq技术分享群：334023047

阅读原文　　在看

完整SIP/SDP媒体协商概论-ICE选项和keepalives讨论

原创 james.zhu Asterisk开源派

笔者在前面的完整介绍了关于后续offer/answer交互过程中offer接收和answer生成的细节。这里，笔者将介绍后续offer/answer交互中的最后一个部分-ICE选项支持，状态更新以及存活时间的讨论。在更新状态中又涉及了全场景部署的处理流程和轻量级场景的部署流程。现在，我们逐一介绍这些细节。

1 全场景部署处理流程

在全场景部署的更新处理流程中涉及了四个方面的内容需要讨论。首先，我们介绍一下关于ICE重新启动的内容。

在ICE重新启动之前，针对媒体流的每个构件，agent必须记住在有效列表中的最高优先级标识的配对（称之为历史已选配对）。Agent将会继续使用这些配对发送媒体流。发送媒体流的流程在后面的文章中会加以讨论。一旦目的地地址收到提示信息，agent必须刷新有效列表和检查列表中的数据。然后agent重新计算检查列表和其状态。具体处理流程参考历史文章中关于构建检查列表的流程。

如果在offer/answer交互中已添加了一个新的媒体流，agent必须为此新媒体流创建一个新的检查列表，还创建一个新的初始为空的有效列表。具体处理流程参考历史文章中关于构建检查列表的流程。

如果在offer/answer交互要移除一个媒体流或answer拒绝了offer中提供的媒体流的话，agent必须为此媒体流刷新有效列表，必须结束在任何处理过程的STUN事务。agent必须为此媒体流移除检查列表，并且取消任何待处理的ordinary checks。

针对全场景部署中的更新有效列表和检查列表，ICE的处理根据agent的状态和检查列表状态的不同存在很多不同流程。首先说明，除非正在ICE重新启动，否则有效列表是不会受更新offer/answer交互影响。

针对一个媒体流来说，如果agent的状态是正在运行状态，检查列表是已更新状态（如果状态是完成状态，检查列表已无相关性）。为了实现这个要求，agent必须使用计算流程重新计算检查列表。具体处理流程参考历史文章中关于构建检查列表的流程。在计算结果中，如果发现在检查列表中有一对配对已经是全面检查列表中出现的配对的话，其配对状态是Waiting，In-Progress，Succeeded或Failed状态的话，其状态将被拷贝到检查列表中；否则其状态将设置为封冻状态（Frozen）。

小广告



如果检查列表中无任何活动配对（意味着每个检查列表中的配对是封冻状态），full-mode（双方agent都使用了ICE）的agent为第一个媒体流在有效列表中设置第一个配对，设置的第一个配对的状态为等待状态，然后把在检查列表中具有同样component ID和具有同样foundation所有其他配对设置为等待状态。

接下来，agent会逐一执行某个检查列表，最高优先级的配对首先开始执行。如果列表中有一个配对状态是成功状态，其component ID设置为1，然后继续执行以下处理。在同样检查列表中，如果所有其他封冻状态的配对具有同样foundation，并且在这些具有同样foundation配对中，如果它们的component ID不是1的话，agent将把这些封冻的配对的状态设置为等待状态。针对一个具体的检查列表，如果媒体流的每个构件的配对有一对在成功状态，为其他所有媒体流的第一个构件（可能在不同的检查列表中），agent将会把所有具有同

样foundation，其配对状态处于封冻状态的配对的状态进行状态迁移，这些配对的状态从封冻状态设置为等待状态。

2 轻量级场景部署场景

轻量级的部署场景中关于更新列表检查的处理比较简单。如果为一个媒体流重新启动ICE，agent也必须为此媒体流重新启动一个有效列表。Agent也必须记得此媒体流的每个构件的上次有效列表中的配对（称之为历史已选配对）。然后根据流程继续发送媒体流。流程的规则定义在将来的文章中介绍。接下来，针对每个媒体流的ICE处理状态必须修改为正在运行状态。

3 ICE option标识

在实际应用场景中，我们经常遇到一些用户的反馈WebRTC的兼容性问题，自己也经常面临WebRTC的兼容性问题。我们不得不经常更新一些功能支持，同时也不得不不断学习新的知识。其实，我们从RFC5245以及其最新规范8445中就可以看出，这些处理流程在最近几年内进行了很多次修改。现在，我们介绍一个特殊的ICE选项。除了在RFC5245中规定的ICE启动流程以外，在最新的RFC8445中增加了一个ICE选项-ice2 选项。当ICE agent在候选交互中包含了ice2选项以后，表示需要遵从RFC8445规范。在一些特定的���互中使用ice2让对端peer获悉agent也不再使用此交互流程。例如，在RFC8445中已经移除了主动推荐标识的流程（aggressive nomination procedure），如果agent需要通知对端不再使用此交互流程时，可以增加一个ice2选项来表示。

一个agent如果遵从RFC8445的话，在候选地址交互开始时必须通过ice2通知对端peer其交互模式启用了ice2选项。否则，对端可能收到一个未知的ice选项。

关于对ice2的编码和对对端peer的消息传输，读者可以参与RFC4566中的参考规范。在其参考规范中详细说明了ICE-

SIP-SDP的细节。最新的草案参考链接如下：

```
ice-sip-sdp:
https://tools.ietf.org/html/draft-ietf-mmusic-ice-s
```

另外，再次提醒读者，如果要开发最新的SIP和WebRTC的业务应用的话，因为ICE处理流程中SDP的频繁更迭，读者一定要关注最新的RFC8445规范以及关于SDP构建的草案。

4 Keepalives

无论是否使用ICE或者媒体流的状态如何，keepalives是终端保持存活的重要手段。大家知道，为了终端保持状态的活动状态，所有的终端都必须不断向服务器端发送存活消息。针对媒体会话来说，存活消息的目的就是为了保持NAT绑定存活状态。无论媒体流状态是inactive，sendonly，recvonly还是sendrecv状态，并且也不管在线状态，带宽属性设置状态，终端必须发送keepalives。即使周期会话中完全没有使用ICE，终端也必须发送keepalives。终端应该使用对端peer能够支持的格式来发送keepalives。ICE终端允许基于STUN的keepalives支持UDP的流。具体来说，当agent是一个全场景部署的agent，并且和对端peer（轻量级部署场景或全场景部署agent）通信时，它们之间必须使用STUN keepalives。agent能够通过每个媒体会话中属性a=candidate状态决定对端peer支持ICE。如果对端peer不支持ICE的话，keepalives数据包格式的选择是本地部署的事情。根据RFC5245的推荐，keepalives的格式最好是这样的格式-在实际媒体内容缺失的情况在，其格式支持数据可以非常容易发送出去。比较典型的两个例子是RTP No-Op和RTP的舒适噪音处理。如果对端peer不支持任何目前比较采用的keepalives格式的话，agent应该使用一个不正确的版本发送RTP数据包或者其他格式发送（当然，peer可能会丢弃这些错误数据）。

在Tr秒时间内，为了一个媒体构件的处理流程，ICE使用一个候选配对发送数据，如果此候选配对中没有数据发送，agent必须为此配对生成一个keepalives。Tr值应该是可配置的值，默认设置为15秒。Tr值一定不能低于15秒设置。另外一种处理方式是，如果agent通过动态方式可以发现intervening NAT的绑定生命周期的话，agent可以使用这个绑定生命周期来设置Tr值。系统管理人员是在自己可控的网络环境中部署ICE，在网络环境允许的情况下，Tr值应该尽可能的长一点。

如果keepalives使用了STUN，STUN绑定指示需要根据RFC5389规范来执行。此绑定指示不能使用任何认证机制。绑定指示中一个包含FINGERPRINT属性实现多路分用，但是不能包含任何其他的属性。此绑定指示仅用来维持NAT绑定存活处理。绑定指示通过同样的发送媒体的本地和远端候选地址来发送。虽然绑定指示是用来处理keepalives，但是agent仍然也需要准备好接收连接检查。如果agent收到了一个连接检查的话，agent应该根据RFC5389生成一个响应消息，但是，这个处理不会影响ICE的处理。

一旦ICE选择了候选配对准备发送媒体流，或媒体开始传输，无论以上两种方式哪种方式首先发生，agent必须开始keepalives的流程处理。一旦会话结束或媒体流被移除，keepalives也需要马上结束。

这里需要补充一点关于keepalives和Voice Activity Detection (VAD)一些讨论。实际环境中，keepalives在实际数据缺失的情况下发送，所以实际环境中如果没有使用VAD的话，从来不会产生keepavlives消息发送的情况，因此也不会存在带宽增加的可能性。当启用VAD时，keepalives消息是在静音期发送，会在每15秒-20秒之间发送一个单数据包，此数据所占用的网络资源远小于语音数据发送状态下的（每20-30毫

秒之间发送）所需要的网络资源。因此，keepalives的影响不会对环境部署中网络带宽有很大的影响。

读者注意，因为keepalives涉及了多种网络环境的连接，网络设备的部署也非常复杂，简单的一种规范很难完整说明全部的具体场景。笔者可以根据不同的场景做进一步的研究：

- 关于keepalives的优化，读者可以查阅草案的3.4章节：

  https://tools.ietf.org/id/draft-ietf-pcp-optimize-keepalives-00.txt

- 关于keepalives的讨论，一些规范和浏览器支持做了一些调整，读者可以查阅笔者参考资料链接获得详情。RFC5245仅提供了ICE的keepalives的讨论，读者也可以结合RFC6263关注RTP中NAT绑定中的keepavlives的讨论。

参考资料：

https://www.rfc-editor.org/rfc/rfc5389

https://tools.ietf.org/id/draft-ietf-pcp-optimize-keepalives-00.txt

https://www.cisco.com/c/en/us/support/docs/ip/serial-tunnel-stun/16398-50.html

https://tools.ietf.org/html/draft-muthu-behave-consent-freshness-04

https://tools.ietf.org/html/rfc6263

https://www.semanticscholar.org/paper/NAT-Traversal-Techniques-and-UDP-Keep-Alive-Widmer/9f730e024dd212186c7b2ced75c877edad6951f0

https://www.rfc-editor.org/rfc/rfc8445#section-10

https://tools.ietf.org/html/draft-ietf-mmusic-ice-sip-sd



融合通信/IPPBX商业解决方案：www.hiastar.com

最新Asterisk完整中文用户手册详解及免费slack支持：www.asterisk.org.cn

Freepbx/FreeSBC技术文档: www.freepbx.org.cn

如何使用FreeSBC，qq技术分享群：334023047

关注微信公众号：asterisk-cn，获得有价值的通信行业技术分享

阅读原文

在看

完整SIP/SDP媒体协商概论-ICE/SIP呼叫延迟问题初探

原创 james.zhu Asterisk开源派

前面的章节中笔者讨论了ICE中更新状态和ICE选项支持的内容。这里，我们要进一步讨论ICE中媒体处理和配合SIP场景中可能面对的问题。具体讨论在ICE在处理涉及的发送媒体和接收媒体的两个不同的流程。最后，笔者将介绍在ICE环境中，配合SIP使用时，呼叫业务需要处理的流程以及呼叫延迟推荐的优化方式。

首先说明，在ICE环境中，针对媒体/数据处理两种规范有一定的区别。具体来说，在RFC5245规范中规定的是媒体收发处理，但是在其更新的规范RFC8445中讨论的是数据收发处理。这里，规范中使用的定义有所区别，另外，具体的处理流程也有所不同。这里，我们主要是根据RFC5245的规范做详解，没有根据RFC8445规范说明。因此，读者如果需要最新的关于数据收发的处理，应该参考RFC8445和RFC576（以及最新的RFC8656）。

1 ICE环境中发送媒体

针对发送媒体的讨论需要关注三个部分的内容。首先是全部署环境中发送媒体的讨论。在全部署场景中，agent总是使用一对候选配对来发送媒体，我们称之为“selected candidate pair”。Agent将会通过已选候选配对将媒体发送到远端候选地址，并且设置数据包的目的地地址和端口等于远端候选地址, 然后从已选配对的本地候选地址发送媒体。当本地候选地址是server 或者 peer reflexive时，媒体是从基准地址来发送。从转发候选地址发送过来的媒体需要从基准地址通过STUN服务器端发送出去，发送处理流程按照RFC5766（最新规范参考RFC8656）的规范来执行。如果本地候选地址是转发候选地址的话，RFC5245规范推荐，针对远端候选地址，agent需要在STUN服务器端创建一个通道。关于agent创建通道的流程按照RFC5766-11的规范执行。针对媒体流的构件的已选配对有以下三种状态说明：

- 如果此媒体流的检查列表的状态是运行状态，并且因为重启ICE，没有以前的已选配对，那么已选配对为空。
- 如果此媒体流的检查列表的状态是运行状态，并且因为重启ICE，存在一对以前的已选配对，已选配对等于以前的已选配对。

- 如果检查列表的状态是完成状态，针对此媒体构件来说，已选配对等于在有效列表中的最高优先级标识配对。

媒体流的构件影响着媒体流的发送方式，并且已选配对数量状态也决定agent是否可以针对具体构件发送媒体。具体来说，针对一个媒体流中的一个构件中（至少一个构件），如果一个已选配对为空，agent**一定不能**对其他构件发送媒体。针对媒体流的每个构件，已选配对都有一个值的话，agent**可以**为此媒体流的所有构件发送媒体。**注意**，媒体流的一个构件的已选配对可以不等于同样构件中最近offer/answer交互中的默认配对。如果不相等时，选择已选配对发送媒体，而不是使用默认配对发送媒体。当ICE流程首先完成时，如果已选配对不能匹配默认配对，被控agent就会发送一个更新的offer/answer交互消息来纠正这个不同。不过，直到更新offer收到之前，仍然存在一个不匹配结果。进一步来说，在一些极端情况下，在更新offer/answer交互中的默认配对不会有任何匹配。

小广告



以上介绍的是全场景部署环境中关于媒体发送的处理讨论。这里，我们简单介绍在轻量级的部署环境中媒体发送的流程。轻量级部署场景中媒体发送的流程比较简单。在有效列表中没有任何候选配对时，轻量级部署场景中的agent一定不能发送媒体流，直到在有效列表中为媒体流每个构件包含了一个候选配对，agent才能发送媒体流。Agent将会通过候选配对将媒体发送到远端候选地址，设置数据包的目的地地址和端口等于远端候选地址, 从已选配对的本地候选地址发送媒体。

在完整场景部署环境中，ICE支持了一个抖动缓冲机制（jitter buffer）交互。在传输RTP流时，RTP媒体流可以开始使用一个候选地址传输，一定时间后，RTP流可以切换到另外一个候选地址继续传输。虽然以上这种场景不会在ICE中经常发生，但是这种情况一旦发生的话，可能引起语音延迟等问题。具体来说，传输地址切换到新的候选地址可能导致RTTP流使用不同的网络资源属性通过不同的网络路径或者网络环境发送媒体流，这样就会导致RTP延迟的问

题。因此，agent可以使用抖动缓冲机制来优化数据包发送时候选地址切换带来的语音延迟。很多语音编码使用一个标记来标识talkspurt起始段。为了使用抖动缓冲机制的目的，当发送媒体流时，候选地址发生切换时，RFC5245规范推荐发送方marker bit来表示。具体关于marker bit，读者可以参考RFC3550。关于talkspurt和RTP的其他定义，读者可以参考RFC5391，以及参考资料中的其他链接来进行进一步学习，这里不再展开讨论。

2 ICE环境中接收媒体

一旦ICE成功执行了最新的offer/answer交互流程，ICE部署场景中的接收方必须准备接收候选地址每个构件所支持的媒体，包括其RTP和RTCP数据流。根据RFC5245推荐，针对一个具体的媒体流，当agent从新的源地址和目的地IP收到媒体流数据包时，agent应该使用抖动缓冲机制来接收媒体流数据（前面已讨论）。在RFC3550-8的算法中定义了SSRC的冲突检测和重复循环机制，这些算法中使用了一些基础的资源，例如同一SSRC的不同源传输地址中的检测方式。当使用ICE时，可能发生媒体流通过不同候选地址（切换）发送媒体流数据。Agent能够比较智能，可以判断媒体流是否来自于同一对端，此agent的对端可能产生了STUN交互传输的新地址。因此，如果源传输地址发生了改变，但是数据流来自于同一对端peer的话，这种情况不应该被视作为SSRC的冲突。

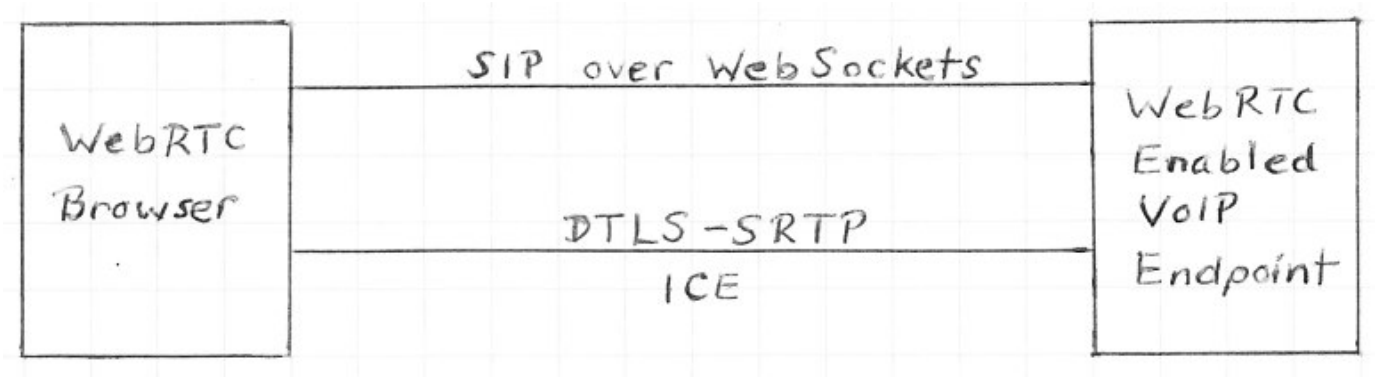
3 ICE配合SIP使用的讨论

前面我们全部讨论的是关于ICE环境中的处理流程。因为目前大部分的语音通信中，SIP协议已经非常普遍，很多应用场景不得不和SIP呼叫集成。在RFC5245中讨论了很多和SIP应用场景的集成，但是在最新的RFC8445中已经移除了关于SIP的使用讨论。因为目前很多应用仍然需要和SIP集成，为了能够让读者对ICE相关技术有一个比较完整的认识，因此笔者认为仍然有必要针对SIP协议的集成进行说明。具体来说，关于ICE部署场景中和SIP结合使用可能涉及几个方面的内容细节，包括SIP信令延迟处理（INVITE中的offer处理/响应中的offer处理），SIP options标签和媒体功能option 标签支持，SIP分寸处理交互，SIP Preconditions交互和SIP第三方呼叫控制交互。下面，笔者针对这些内容做具体说明。

4 关于ICE/SIP延迟原则讨论

根据笔者前面文章中介绍的和本文章前面介绍的内容，ICE处理流程需要在两个终端之间进行一系列，基于STUN的连接检查。这些检查是通过answerer生成的answer消息启动，对端offerer收到answer消息启动offerer的检查。完成这些检查需要**耗费一定的**时间，并且还有一些类似的处理，例如在**answer或offer选择的消息**都会被看作影响用户时延的一些要素条件。

在这些时延的处理中，其中两个比较重要的时延需要用户去认真考虑，它们分别是**post-pickup delay和post-dial delay**。Post-pickup delay表示的延迟是介于用户应答呼叫和用户之间开始进行正式语音通话的时间。Post-dial delay表示的延迟是介于用户输入呼叫目的地地址和成功呼叫对方以后返回的回铃音之间的时间。因为这两个时间分别出现在了初始请求中和响应中，并且涉及到了ICE的连接检查，因此，我们针对这两个时间在各自流程中的细节做进一步的说明。



图片来自于互联网资源

5 INVITE中的offer消息

很多时候，一些用户可能会遇到一些通过WebRTC呼叫延迟的问题。这些问题很多都是和ICE协商处理流程过长相关。现在，我们讨论一个非常具体的问题，那就是post-dial 延迟问题。如果需要降低post-dial 延迟，RFC5245推荐呼叫方开始采集候选地址的流程应该早于初始INVITE发送时间。这个设置方式可以通过界面的设置提供一个提示，此时SIP呼叫正在等待状态，或者通过拨号盘的一个事务活动，电话摘机等方式进行提示处理。因此，ICE可以早于初始请求发送之前采集候选地址，决定延迟时间。

如果应答方answerer在INVITE请求中收到一个offer消息，answerer应该开始采集候选地址，一旦完成采集流程，然后生成一个临时响应消息。这里，ICE要求在SDP中传输的临时响应消息是可靠传输。这种可靠传输可以通过现在的PRACK机制来实现。具体的实现机制可以参考RFC3262。另外，这种传输也可以通过ICE的优化来实现。使用ICE优化的处理方式来实现的话，如果临时响应中包含SDP answer，这个SDP answer开始ICE流程来处理一个或多个媒体流的话，这个临时响应可以实现可靠发送无需按照RFC3262规范执行。如果这样操作，agent通过TU的指数退避算法重新临时响应。具体的定义参考RFC3262。这样的重传处理可以通过两种方式实现（停止STUN绑定或者重传answer消息）。Agent重传必须停止STUN绑定接收流程，针对媒体流的STUN 绑定接收是在SDP中设定。因为这个绑定请求接收表示offerer方已经收到了对端的answer消息。另外，agent如果需要这样的重传，需要在2xx响应

中重新传输answer消息。如果对端peer是一个轻量级的agent的话，就从来不会有STUN绑定请求。如果类似这样的场景发生，agent发送4次18x临时响应以后，然后必须停止重传。这里，有时尽管对端从来没有收到18x临时响应，实际上ICE已开始工作。但是，在实际生产环境中，对一些中间设备或者防火墙穿越来说，发送18x仍然是非常重要的一种验证方式。如果agent在最后重传之前没有收到绑定请求，agent不会认为此会话已结束。

尽管临时响应通过可靠传输进行，但是，当agent能够发送更新offer或者answer时必须遵从一定的规则，规则不能发生修改，规则定义按照RFC3262处理。具体来说，如果INVITE中包含一个offer，同样对应的answer出现在此INVITE的1xx所有响应消息和2xx响应消息中。另外，仅2xx发送以后，更新的offer/answer交互才能执行。如果双方agent都支持PRACK的话，这种优化机制就不应该被使用。注意，优化机制不是支持临时可靠传输的标准技术，这种优化机制具体使用在临时响应传输answer消息的环境中，通过此answer消息开始ICE处理流程。

除了以上的处理延迟的方法以外，另外一种可选方式是agent可以延迟发送一个answer消息，直到收到200 OK以后再发送answer消息。但是，这种处理方式可能带来不好的用户体验，RFC5245不建议推荐。

一旦agent发送answer以后，此agent应该开始连接性检查。一旦针对媒体流的每个构件的候选配对进入到了有效列表中，answerer应答方可以开始在媒体流资源上发送媒体数据。

因为SIP处理流程和ICE的结合，可能会遇到关于其他媒体流发送的问题。根据前面的处理方式，在此流程处理节点之前，任何需要针对呼叫方发送的媒体（例如早期媒体流）一定不能在此处理节点之前发送。因为这个原因，ICE处理应该延迟提示被呼叫方，直到前面说的候选配对进入到有效列表中才开始发送媒体，否则没有任何候选地址可以支持媒体流的每个构件。如果在SIP呼叫的路径上有PSTN网关的应用场景，还要考虑PSTN的setup消息的处理流程带来的延迟。这样就会增加一个post-dial 的延迟（拨号后延迟���。但是，这样可能会带来一个假振铃（ghost rings）的“好的”效果。这种假振铃通常是被呼叫方会听到振铃声，接听电话以后被呼叫方听不到对方任何声音。因此，假振铃也无需要求或使用precondition（因为precondition是取决于本地支持（RFC5027））。除了以上好处以外，另外一个好处就是可以“保证”无爆破音数据产生在第一个媒体的数据包。因此，也可能出现post-pickup delay

为0的“理想”结果。如果agent选择延迟本地提示的方式，一旦提示开始以后，agent应该生成一个180响应消息。

6 Response中的offer消息

前面我们讨论了offer在INVITE中，answer通过临时响应或者200 OK传输的情况。其实，ICE也可以在另外一种环境工作，那就是offer消息出现在响应消息中。这种情况经常出现在第三方呼叫业务（RFC3725）的场景中，agent应该在可靠临时响应中生成自己的offer消息（生成消息的规则必须遵从RFC3262规范），并且不能在INVITE接收中提示用户。answer消息将随PRACK抵达。这样的处理方式就会支持ICE处理流程早于提示消息，因此不会增加呼叫创建时间，也不会产生post-pickup delay。一旦ICE完成以后，当呼叫应答以后，被呼叫方可以提示用户并且生成200 OK。因为offer/answer交互在之前已经完成，此200 OK消息中就不会包含任何SDP消息。

除了以上处理方式以外，另外一种可选方式是agent可以把此offer消息置于200 Ok中（这种情况下，answer是在ACK中）。当这种场景发生时，被呼叫方通过INVITE接收消息提示用户，用户应答以后，ICE处理流程才启动。这样处理的好处是可以降低呼叫创建延迟，但是可能会引起后续的post-pickup delay和出现媒体爆破音问题。

7 SIP options标签和媒体功能标签

RFC5768规范定义了配合ICE使用时的SIP可选标签支持和媒体功能的标签支持。ICE部署使用SIP应该支持可选标签和媒体功能标签细节，使用功能标签处理注册机制来支持兼容性处理流程。具体关于“sip.ice” 功能标签细节，读者可以查阅RFC5768规范。

8 ICE和SIP分叉处理的讨论

ICE可以非常好地配合SIP forking处理流程工作。事实上，ICE可以解决很多和分叉处理关联的问题。如果没有ICE的话，当呼叫被分叉处理后，呼叫方收到多个媒体流数据时，呼叫方不能决定哪个媒体流对应哪个被呼叫方。如果使用了ICE后，媒体传输中包含用户信息，连接性检查会早于媒体传输之前执行，媒体传输过程中包含了具体的用户名称（username fragments），通过这样的方式就会获得媒体流以及对应的被呼叫方信息。因此，只要呼叫方收到了一个answer以后，呼叫方就可以执行关联性处理。

9 ICE和SIP Preconditions讨论

QoS（服务质量）是通信服务的核心要求，RFC3312和RFC4032定义了QoS的规范。但是，QoS只能支持在offer/answer交互中的默认目的地列表中的传输地址。如果ICE修改了传输地址，这个传输地址已接收了媒体流的话，这个更新就会通过更新offer来处理，在更新的offer中修改默认目的地地址匹配ICE已选地址。这样的话，这种修改类似于发生了一个re-INVITE，可以完全视作支持了RFC3312和RFC4032，可以忽略掉背后发生的ICE协商的目的地修改流程。注意，很多情况下，事实上agent不应该提前指示QoS preconditions已匹配的结果，它应该等待一定的时间，直到ICE检查完成，并且ICE已选择了候选配对来传输媒体流以后，agent才能指示QoS preconditions已匹配的结果。

ICE有自己的检查预处理条件(Connectivity Preconditions)的交互规范(RFC5898)。此交互规范提供了更多的功能和语法细节。语法规则如下：

```
precondition-type = "conn" / "sec" / "qos" / token
```

有兴趣的读者可以参考RFC5898了解其具体处理流程。

10 ICE和SIP 第三方业务呼叫的互动讨论

ICE可以支持第三方呼叫中的flow I，III，和IV呼叫流程（在RFC3725中定义了4种3PCC 呼叫流程，读者可以查阅RFC3725做详细了解）。其中，flow I 可以支持ICE工作场景，无控制方支持，无ICE感知支持。在Flow IV中，只要控制方通过ICE属性传递支持，无任何属性修改的话，Flow IV就可以工作。Flow II (**这里不是flow III， 规范拼写错误？**)则完全和ICE不兼容，每个agent认为自己是answerer，因此从来不会生成一个re-INVITE请求。

关于第三方处理流程的后续操作处理，需要按照RFC3725-7来进行，这个流程需要其他ICE的处理支持。具体来说，如果agent收到一个mid-dialog的re-INVITE, 它没有包含offer消息的话，agent必须为此媒体重新启动ICE，然后完成新候选地址采集流程。另外，候选地址列表应该包含当前正在被媒体使用的候选地址。

笔者在本文章中首先介绍了ICE环境中媒体收发流程，然后介绍了offer在INVITE请求和响应中的处理，并且指定介绍了关于涉及了SIP呼叫时面对的呼叫延迟以及优化的推荐处理方式。最后，笔者介绍了关于ICE和SIP第三方呼叫业务的整合规范。

参考资料：

https://www.rfc-editor.org/rfc/rfc8445

https://www.rfc-editor.org/rfc/rfc5766

https://www.rfc-editor.org/rfc/rfc8656

http://sbhunia.me/publications/manuscripts/acity11.pdf

https://www.researchgate.net/publication/224348734_Jitter_Buffer_Analysis

http://genesysguru.com/blog/blog/2009/11/12/voice-quality-jitter-silence-supression-and-talkspurts/

https://tools.ietf.org/html/rfc5391

https://www.rfc-editor.org/rfc/rfc3960

https://www.rfc-editor.org/rfc/rfc5027

https://www.rfc-editor.org/rfc/rfc5898

https://www.slideshare.net/alexpiwi5/deploying-webrtc-in-a-lowlatency-streaming-service



融合通信/IPPBX商业解决方案：www.hiastar.com

最新Asterisk完整中文用户手册详解及免费slack支持：www.asterisk.org.cn

Freepbx/FreeSBC技术文档: www.freepbx.org.cn

如何使用FreeSBC，qq技术分享群：334023047

关注微信公众号：asterisk-cn，获得有价值的通信行业技术分享

阅读原文

在看

完整SIP/SDP媒体协商概论-ICE攻击类型和安全架构讨论

原创 james.zhu Asterisk开源派

为了完成整个SIP/SDP媒体协商概论，笔者花费了很多时间介绍关于WebRTC中的ICE处理流程。通过完整的详解，笔者相信关于ICE处理流程的大部分内容已经解释的比较清楚。在总结ICE部署安全问题之前，重复说明一下关于ICE部署-RFC5245的完整名称：

```
Interactive Connectivity Establishment (ICE):
A Protocol for Network Address Translator (NAT) Tra
Offer/Answer Protocols
```

在针对RFC5245的规范介绍中，笔者通过以下几篇文章详细说明了整个ICE流程处理的过程。如果读者想了解完整ICE部署场景和协议详解的话，读者可以参考以下链接：

完整SIP/SDP媒体协商概论-WebRTC/ICE概览
完整SIP/SDP媒体协商概论-ICE初始offer发送详解
完整SIP/SDP媒体协商概论-ICE初始offer接收详解
完整SIP/SDP媒体协商概论-初始应答接收详解完整SIP/SDP媒体协商概论-STUN连接性检查-客户端流程
完整SIP/SDP媒体协商概论-后续offer处理-Offer
完整SIP/SDP媒体协商概论-STUN连接检查-服务器端流程
完整SIP/SDP媒体协商概论-后续offer处理-answer生成
完整SIP/SDP媒体协商概论-ICE选项和keepalives讨论

在本章节，笔者重点结合RFC8445官方为读者补充一些关于ICE部署操作时应该注意的一些问题。这里先说一点题外话。和RFC3261规范或者其他规范相比，笔者认为RFC5245规范不是

一个“非常好”的规范（可能笔者是错误看法，仅供参考），主要有以下几个方面的原因：

1. 多处定义和流程说明不是非常明晰完整（不具体举例），有个别语言用词不是非常规范，开发人员可能存在很多理解偏差。
2. 因为WebRTC的发展太快，浏览器终端技术和开发语言也发展太快，很多新的其他技术在短短几年发生了很多变化，导致协议规范本身的兼容性出现很多问题。这可能也是WebRTC部署使用存在很多问题的原因。
3. 关于和SIP相关内容调整有一点突然。如果SIP和ICE结合使用时，可能让开发人员引起歧义。因为在RFC8445中移除了此部分的建议，因此开发人员如何实现和SIP具体业务的兼容性支持是一个挑战。

当然，任何技术或者官方制定参与者都有其局限性，而且很多出现的新技术完全可能淘汰旧的技术，特别是一些基于浏览器的技术。因此，我们讨论关于ICE处理流程的规范时也必须要结合RFC8445规范来讨论，除了以上链接所介绍的文章以外，为了保证ICE处理流程讨论的完整性，笔者这里重点针对RFC5245和RFC8445中关于ICE部署时需要注意的几个问题进行补充讨论。接下来的讨论中，笔者将讨论关于ICE部署时需要考虑的问题，ICE安全问题，WebRTC安全架构草案。

1 ICE部署时需要考虑的问题

这里，我们首先简单讨论关于NAT和防火墙的环境问题。当初ICE的设计目标是基于当时现存NAT和防火墙设备而设计的。因此，为了部署ICE也没有必要替换或者修改这些设备。另外，如果ICE部署在VOIP环境中的话包括了NAT和防火墙设备的话，语音质量的控制可能已经不是ICE环境所能够控制的。在两个关于NAT和防火墙处理方面的关键的规范RFC4787和RFC5382中，根据规范建议ICE部署应该可以完美支持NAT设备和其终端。在“遵从NAT规范”的网络环境中（实际上很难实现，很

多场景仍然依赖于STUN，TURN服务器协助），无需STUN服务器端，ICE可以正常工作。其他一些呼叫应用的优化，例如语音质量提升，降低呼叫创建时间，降低带宽占用都取决于网络运营者本身。

首先需要一定的带宽要求支持STUN和TURN服务器。在部署ICE时仍然需要很多和网络环境本身的交互，因此很多因素都需要考虑。在现在比较大型的ICE部署环境中，典型的应用包括了ICE和STUN和TURN服务器的交互（一般应用可以借助于第三方的STUN服务器）。一般来说，这些服务器都部署在数据中心或者其他云服务平台。STUN服务器要求相对比较小的带宽服务。对于每个数据/媒体流的每个构件来说，从每个客户端到STUN服务器端至少将会产生一个或者多个STUN事务。在IPv4的网络环境中，基于语音呼叫的呼叫流程中，每个呼叫在呼叫方和被呼叫方之间至少生成4个事务，包括各自的RTP和RTCP。每个事务是一个请求和一个响应。前期的数据包长度是20 bytes，后期的将达到28 bytes。其实，如果按照每个用户在繁忙时间，平均每个用户呼叫4次的话，10用户差不多需要10×1.7 bps， 一百万个用户也差不多1.4Mbps。 这个数据相对来说仍然是一个比较小的数据。当然，这是仅针对STUN 事务来说的，实际应用环境中还有很多其他因素需要考虑，这里不再讨论。

IP version: IPv4
Packet len (bytes): 108 + ufrag

ms	4	8	12	16
500	1.86k	1.98k	2.11k	2.24k
200	4.64k	4.96k	5.28k	5.6k
100	9.28k	9.92k	10.6k	11.2k
50	18.6k	19.8k	21.1k	22.4k
20	46.4k	49.6k	52.8k	56.0k
10	92.8k	99.2k	105k	112k
5	185k	198k	211k	224k
2	464k	496k	528k	560k
1	928k	992k	1.06M	1.12M

IP version: IPv6
Packet len (bytes): 128 + ufrag

ms	4	8	12	16
500	2.18k	2.3k	2.43k	2.56k
200	5.44k	5.76k	6.08k	6.4k
100	10.9k	11.5k	12.2k	12.8k
50	21.8k	23.0k	24.3k	25.6k
20	54.4k	57.6k	60.8k	64.0k
10	108k	115k	121k	128k
5	217k	230k	243k	256k
2	544k	576k	608k	640k
1	1.09M	1.15M	1.22M	1.28M

Connectivity-Check Bandwidth

来自于Jusin Uberti (Google) 测试结果

相对于STUN来说，TRUN服务器可能需要更多的带宽。在TRUN服务器端除了STUN数据以外，可能还有其他转发的媒体数据。如果呼叫需要通过TRUN转发的话，网络环境还要考虑转发到数据流量，最终处理方式仍然取决于网络环境部署本身，例如，如何实现更加灵活的高并发语音和视频会议转发以及如何处理会议进行中人数动态增加的场景都是对带宽处理的挑战。

除了前面说的STUN和TURN服务器对带宽的要求以外，候选地址采集和连接性检查也对带宽有一定的要求。事实上，场景候选地址和执行连接性检查也非常消耗带宽。当然，这应该是一般的常识，和ICE创建以后相比，前期ICE执行候选地址采集和执行连接性检查需要双方不断发送和接收交互请求响应消息，因此就会产生更多的数据流量。如果ICE检查结束以后，带宽的消耗也相对比较低，重新启动ICE以后，带宽消耗又随着重新交

互流程启动，带宽消耗也逐步增加。开始启动ICE keepalives以后，带宽仅局限于一定的边际范围，相当于总带宽来说仍然是比较小的消耗数量。因此，网络部署环境中需要考虑部分ICE候选地址采集和连接性检查交互的数据带宽，还要考虑媒体流传输所需带宽，其余的ICE keepalvies的数据占比相对比较小，则无需太多考虑。

小广告



除了以上两种需要考虑到因素以外，因为候选地址场景和连接性接触引起的网络拥塞也是一个在网络部署中重要的考虑因素。很多网络拥塞可能来自于终端的访问，包括终端处理性能问题，访问边界的网络环境问题导致网络拥塞。网络部署中，管理人员应该考虑ICE部署所引起的这些问题，并且最好确保ICE部署可以实现灵活调整的功能。当然，如果支持ICE灵活调整功能的话，肯定会同时带来呼叫创建时间增加的风险，不过，这样可以保证ICE的稳定性和实现易部署方式。

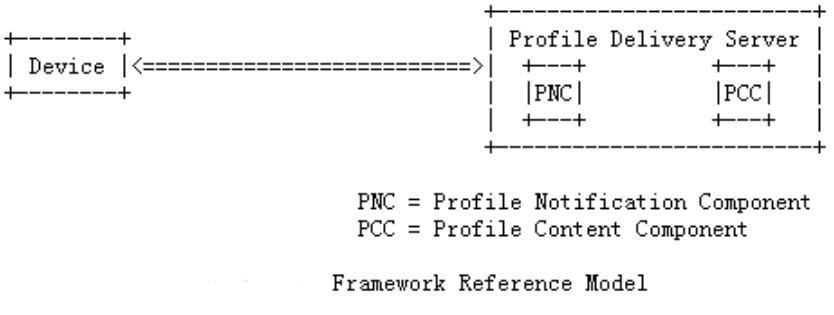
在部署ICE时，用户也要考虑STUN keepalives的设置，避免过长或者过短的keepalives 设置，另外，keepalives的数据无需太多带宽支持，因此此参数应该不会影响网络环境中ICE部署。

在很多部署环境中，用户可能混合使用ICE和lICE-lite。特别说明，ICE-lite的使用场景非常有限，建议用户慎用。

因为很多ICE部署场景是端对端的用户场景，在部署ICE场景时，ICE连接性检查启动端对端的执行流程时涉及了很多的处理流程。这些流程的执行对网络运营人员是一个很大的挑战，因

此网络运营人员就会面对两个比较重要的问题：1）部署ICE时，如何通过工具来排查问题。2）如何通过工具来检测ICE的执行性能。ICE内置功能支持了信令交互的中传输ICE传输的设置，可以检测这些参数和候选地址类型。但是，目前专业的ICE排查工具相对还是比较少，另外，因为ICE需要浏览器客户端的支持，如果浏览器缺乏最新的规范支持，或者WebRTC缺乏最新官方支持的话，其工具可能会影响运维人员的排查效果。除了排查工具因为，另外一种排查手段是系统log或者浏览器的log日志。通过服务器端log和浏览器端的log可以提供丰富的log日志，可以检测到ICE的执行状态。

ICE配合终端使用时，其中一个重要的终端就是SIP终端。ICE的配置依赖于SIP终端的配置信息。配置终端上需要考虑的几个参数包括定时器，TURN服务器的用户安全信息，STUN和TURN主机名等。ICE自己本身不会提供这些配置信息，ICE依赖于附加到ICE的终端认证机制来实现。这种机制管理终端的用户配置信息，例如SIP终端使用的一个架构来传输SIP用户profile。因为篇幅关系，笔者这里不再展开讨论，如果读者有兴趣的话，可以参考RFC6080学习关于”Session Initiation Protocol User Agent Profile Delivery“。

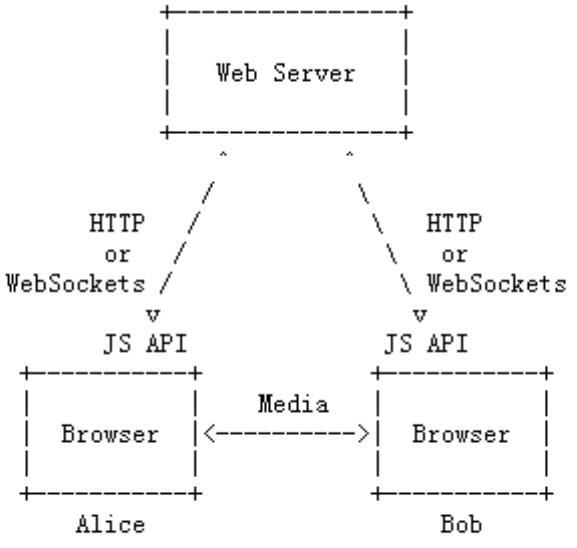


Framework Reference Model

2 ICE安全问题讨论

安全问题一直是通信环境中最重要的问题，并且确实也存在很多的安全漏洞。因为无论在初期的信令协商阶段，还是在后期的语音视频传输阶段都可能存在数据暴露的问题。在ICE部署环境中也存在同样的可能性。在起始阶段中，侦测候选地址的流

程就会暴露客户端和对端peer的源地址，攻击者就会不断监听这些端口和地址来找出攻击的漏洞。一些地址可能是比较敏感的地址，例如通过通过VPN用户的本地网络接口采集到的服务器端反射地址。



A simple WebRTC system

如果在部署ICE时，网络运维人员不能杜绝这些潜在的安全问题，ICE会针对这些地址使用定义机制进行协商和侦测执行流程，最终这些安全机制的核心信息就会被暴露出来。另外，WebRTC本质上来说是为了实现点对点的连接，如果用户双方通过web页面实现连接的话，它们之间的web源数据中可能会出现双方协商的其他相关地址信息，这些信息可能被其他第三方获得来，这样就会导致更多安全问题。为了针对WebRTC的地址安全问题进行优化，Google在2019年提出来一个草案作为一个WebRTC地址的执行要求:

```
1 WebRTC IP Address Handling Requirements
2 https://tools.ietf.org/id/draft-ietf-rtcweb-ip-hand
```

此份草案提出了4个基本规则和4个基本的模式（Enumerate all addresses，Default route associated local addresses，Default route only和Force proxy）。此草案提供了介于ICE和WEBRTC隐私暴露处理方面的要求和基本的原则。

以上背景说明总结了关于ICE中可能存在的安全问题，根据其环境部署，执行连接检测，侦测候选地址等不同阶段，在ICE部署中存在以下几种类型的攻击方式：

连接性检查攻击: 攻击者的目的是对连接性检查进行扰乱，攻击者让agent进入到一个错误的连接性检查的状态中。根据攻击类型的不同，攻击者需要不同的支持能力。在一些攻击场景中，攻击者需要置入到连接性检查的路径中。还有的场景中，攻击者也无需在连接性检查路径上，攻击者具备可以生成STUN连接检查的能力也可以进行攻击。攻击者在连接检查的网络环境以后，它们可以被看作为一个网络实体来控制agent，攻击者可以通过为连接性检查提供**错误的**检查结果或者测试结果进行攻击，具体的几种欺骗类型包括：

- False Invalid：这种类型环境中，攻击者可以对一对agent进行欺骗，让agent认为候选地址是无效的地址。这样会引起agent的错误判断，让agent认为推荐的候选地址是有效地址（攻击者推荐的候选地址，已被注入），或者对呼叫进行干扰，强制候选地址采集失败。为了强制生成一个这样的结果，攻击者需要对端其中一方的agent发送对端连接检查。当连接性检查发送后，攻击者注入一个伪装的响应消息，携带一个不可恢复的错误响应，例如400或者故意丢弃此响应，响应就永远不会返回到agent。另外一种方式是如果候选地址是有效的，初始请求仍然可能会抵达对端agent，并且返回一个成功的响应。攻击者就需要强制数据包和其响应通过DoS的攻击的实体来干扰数据正常收发或者丢弃数据。攻击者通过伪装一个响应消息，使其具有能力通过STUN 短期安全机制生成用户信息。为了保证响应成功处理，攻击者需要密码，此刻，如果候选地址交互信令是加密的，攻击者就不会获得密码，不能获得密码然后就丢弃这个响应。当然，如果信令没有进行加密，攻击者就可能获得用户密码，进行进一步的攻击流程。伪装的Spoofed ICMP Hard

Errors（Type 3, codes 2-4）也可以用来生成False Invalid结果。攻击者也具有一定的能力生成ICMP错误响应消息来欺骗agent，agent会对攻击者暴露多个候选地址和端口信息。这样攻击者通过ICMP伪装的响应消息获得攻击权限。

- False Valid：这种类型的欺骗环境中，攻击者可以对一对agent进行欺骗，当候选地址不是有效地址时，攻击者欺骗agent，认为候选地址配对是有效。这样会引起agent通过配对生成一个会话，最后，agent接收不到任何返回的数据。如何实现强制生成False Valid的具体流程和以上讨论的非常相似。
- False Peer-Reflexive Candidate：这种类型的欺骗场景中，攻击者诱导agent发现一对新的候选地址配对，实际上这一对配对不是agent所期望获得的候选配对。这样可能导致攻击者通过数据转发的方式，让数据进入到DoS目的地，然后进行监听等攻击流程。这种方式的实现是通过伪造请求，伪造响应或伪造转发地址来实现。
- False Valid on False Candidate：这种类型的欺骗采集中，agent已经相信了攻击者的身份，攻击者已经置入了一个候选地址，这个地址实际上不会路由到agent端地址。例如，注入错误的peer-reflexive candidate或者错误的false server-reflexive candidate。当然，如果攻击者注入任何其他的错误地址，攻击者可以进行任何方式的攻击。涉及到具体的安全攻击类型可以参考RFC5389-16。

  以上多种场景可能发生有很多前提条件。除非伪装的候选地址确认了攻击者的身份，否则攻击很难实现。因为双方agent可能在不同的网络环境，结构不同的转发地址，在不同网络，不同终端那个同时协调，并且对攻击者实现了成功认证，这种概率相对比较低。即使攻击者被伪装的候选地址确认为一个正常用户，如果数据路径是加密的，攻

击者也很难实现攻击，但是可能会对连接性检查流程造成干扰。

第二种攻击方式就是此服务器的反射地址进行攻击-**服务器反射地址采集攻击。**因为ICE终端使用STUN绑定请求从STUN服务器端来采集服务器反射候选地址，不会以任何方式来进行签权认证。因此，攻击者可以使用很多方式进行攻击，例如，使用一些攻击方式为客户端提供错误的服务器反射候选地址。攻击者可以和DNS服务达成一个伪数据，这样，DNS查询以后返回一个伪装的STUN服务器地址，这个STUN服务器地址然后对客户端提供一个伪装的服务器反射地址。另外一种方式是通过攻击者监控STUN信息，发现网络漏洞或者其他共享资源，例如WIFI等，然后攻击者注入自己的伪响应消息，客户端接受为有效的信息。还有一种攻击方式是攻击者通过伪造方式，构建一个STUN服务器，STUN服务器返回一个不正确的响应消息，携带错误的映射地址。

第三种方式是对**转发候选地址采集攻击。**攻击者试图干扰转发候选地址采集的消息，强制客户端相信客户端有一个错误的转发候选地址。TURN签权交互的机制是使用的长期安全机制，因此，如果注入伪装的响应消息或者请求消息不会很多成功的注入。另外，不像绑定请求的处理流程，因为为客户端提供转发候选地址时没有使用源IP地址和端口，因此，分配请求不会允许转发攻击（携带了已修改的源IP地址或端口）信息。最后，即使攻击者强制客户端认为其转发地址是无效转发候选地址，连接检查也成功确认其候选地址是成功的配对地址。攻击者仍然需要每次在错误的候选地址发起一个错误的有效结果，成功协调以上的状态对攻击者来说也是一个极大的挑战，因此这种攻击方式的成功概率也非常低。

最后一种攻击方式是**内部攻击。**很多时候，往往内部安全问题是安全措施中最为薄弱的地方。攻击者通过第三方工具注入伪

候选地址或STUN信息。当攻击者在交互流程中是一个已认证用户和有效参与方时，攻击者可以发送攻击信息来配合ICE部署。一些内部软件或者第三方业务和ICE集成时可能会导致安全问题。比较常见的一种攻击方式是STUN放大攻击。攻击者会让agent转发候选地址或者语音数据包到一个目的地地址。攻击者发送大量的候选地址，相应的响应agent收到候选地址消息以后就会启动检查。就会转发到目的地地址，当然，最终这些目的地地址不会返回任何响应消息，agent因此也从来收不到响应消息。放大攻击的方式也可能使用在定时器的设置中。在WebRTC的使用场景中，攻击执行可能已经在后台执行，WebRTC用户可能根本没有意识已经被攻击，因为攻击执行可能通过浏览器访问已经获取到了攻击代码。同时，在每Ta毫秒内，answerer端将会启动一个新的连接性检查, 因为攻击者发送了大量的候选地址（例如提供50个候选地址），重传定时器就会对大量候选地址设置一个非常大的定时器数值。因为网络资源的限定，或者agent端的资源能力，放大攻击的手段可能最终导致处理时延和其他相关流程的延迟。当然，这种放大攻击的手段也可以通过其他的手段来避免，例如，agent终端限制接受候选地址最大数量，或者限制连接性检查的并发数量等方式。很多研究人员也试图通过升级或者拓展相关的协议来避免攻击发生，例如强制发起方发送第一个消息以后，等待一段时间，收到回复响应以后，再进行下一个消息发送。这种思路事实上在ICE部署环境中非常难以实现，ICE不可能区分以下两种场景的处理：

- 无响应消息，因为发起方被启用执行DoS攻击来针对一个可信任的目的地实体（此实体将无任何响应）。
- 无响应消息，因为对发起方来说，IP地址和端口都是不可达状态。

以上这两种情况中，第一种情况下无进一步的检查发送，第二种情况下，在下一个机会中还会再发送一次检查消息。

WebRTC安全初探

在本文章中，笔者重点讨论的是关于ICE的安全问题和其攻击手段的一些详解。但是，因为讨论ICE部署必须结合WebRTC来进行讨论，因此，笔者在本章节结合最近发布的一些WebRTC安全草案简单讨论一下关于WebRTC的业务功能中的一些安全问题。根据最新的关于WebRTC安全的草案和专家针对浏览器安全提出的建议，用户需要考虑以下几个方面的安全问题：

- 对信令加密，对媒体加密。笔者在以前很多文章中针对信令和媒体加密做了很多技术，这里不再讨论。
- 对终端资源进行安全管理。WebRTC需要访问终端麦克风和摄像头资源，所以用户需要对此资源进行充分的安全管理。调用这些资源前必须获得认可。
- 应用/屏幕关系的访问许可。确保对端用户是安全用户，可以确保本地信息不会被泄漏。
- 确保本地IP地址和隐私不会被轻易访问获取。
- 确保WebRTC访问策略返回企业内部的安全访问策略。

在最新的关于WebRTC安全架构的建议中，通过可信任命模式下针对已确认实体和未确认实体进行区别处理，并且增加了针对SIP SDP的拓展说明，WebRTC关于终端安全访问模式细节，通信认可处理，Web安全讨论和IP位置隐私处理等内容。这些最新的细节基本上符合了目前WebRTC中所遇到的各种安全问题，在ICE部署环境中，网络运营人员可以通过此草案来做进一步学习。读者可以通过以下链接做更多了解：

```
1 WebRTC Security Architecture
2 https://tools.ietf.org/html/draft-ietf-rtcweb-secur
```

本章节首先介绍了关于ICE运营环境的要求，然后介绍了关于ICE攻击的类型，最后介绍了WebRTC的安全问题以及关于WebRTC安全架构的简述。

总结，通过以上关于ICE攻击类型和安全条例，结合以前的历史文档，笔者已经基本完成了关于ICE部署的详解讨论。在接下来关于SIP/SDP的概论讨论中，笔者将继续讨论其他和SIP/SDP的内容。

参考资料：
https://tools.ietf.org/id/draft-thomson-tram-turn-bandwidth-01.xml
https://tools.ietf.org/html/draft-wing-tram-turn-mobility-03
https://github.com/coturn/rfc5766-turn-server/
https://bloggeek.me/is-webrtc-safe/
https://webrtc-security.github.io/
https://tools.ietf.org/html/draft-ietf-rtcweb-security-12#section-3



融合通信/IPPBX商业解决方案：www.hiastar.com
最新Asterisk完整中文用户手册详解及免费slack支持：www.asterisk.org.cn
Freepbx/FreeSBC技术文档: www.freepbx.org.cn
如何使用FreeSBC，qq技术分享群：334023047
关注微信公众号：asterisk-cn，获得有价值的通信行业技术分享

阅读原文

在看